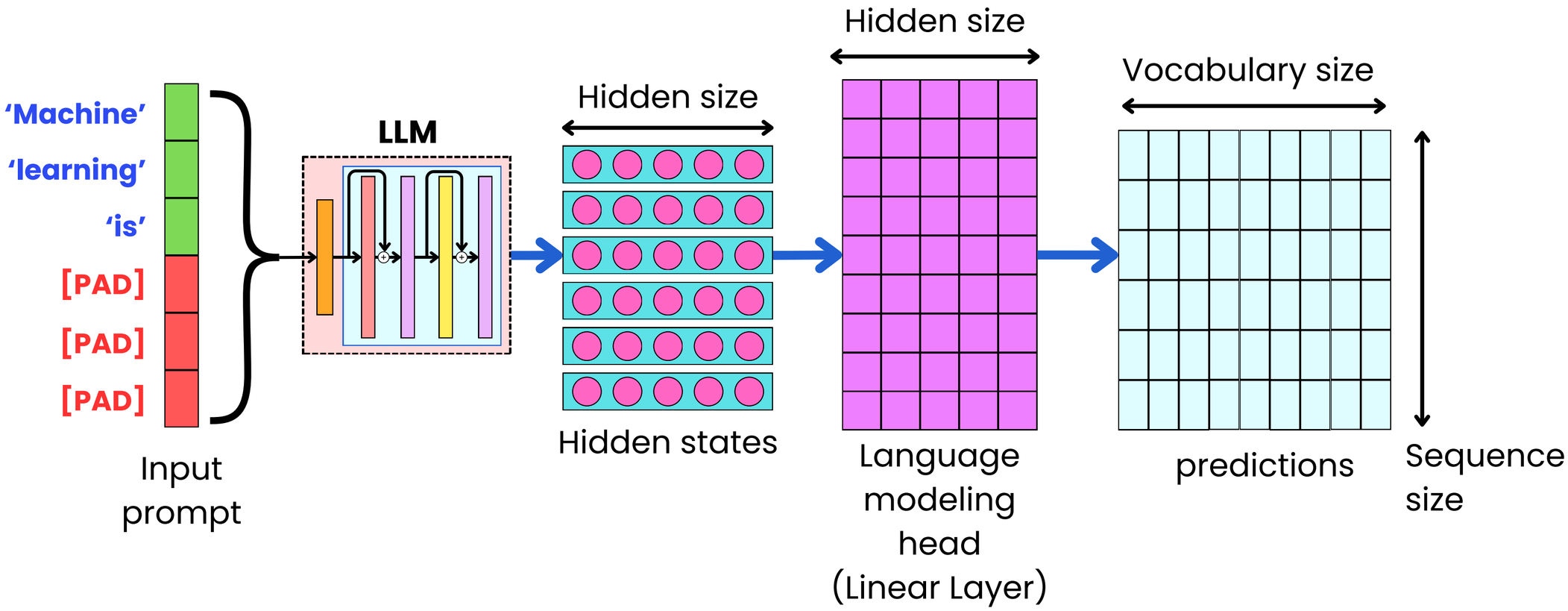

The AiEdge ACADEMY
Fine-Tuning LLMs

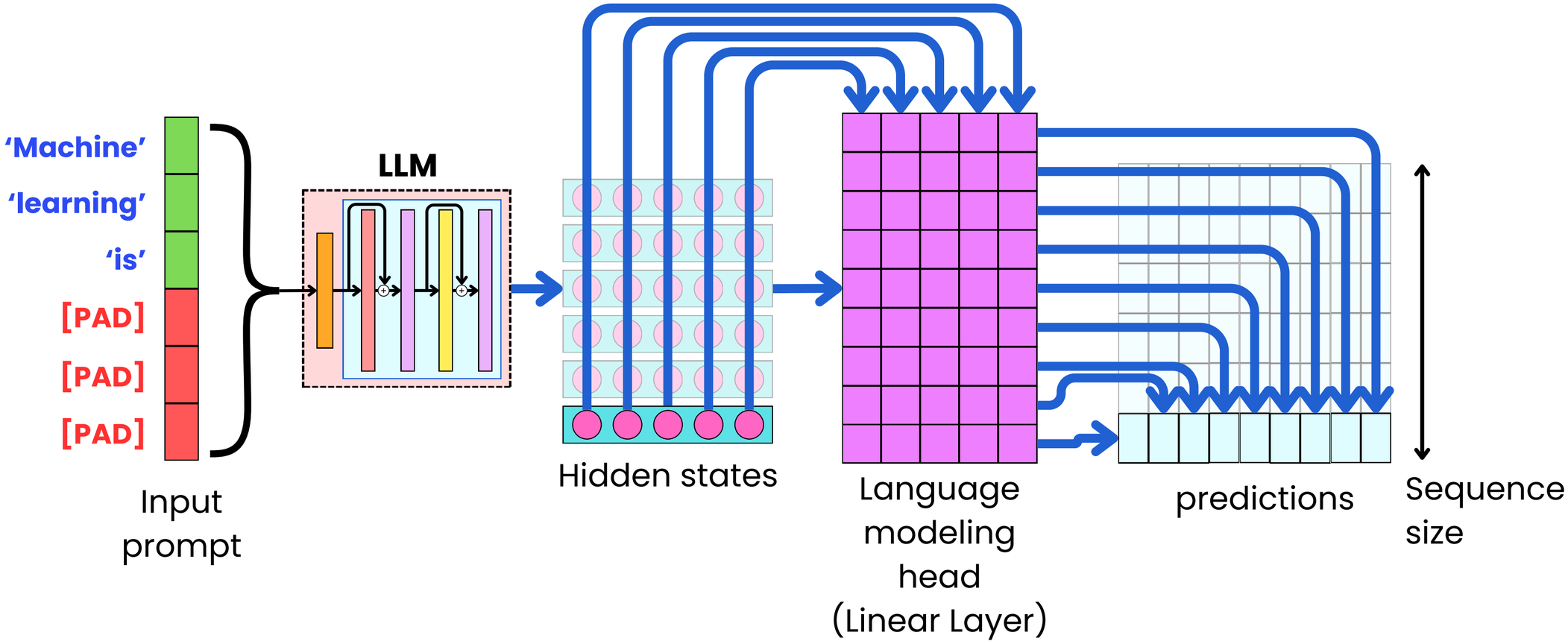
The Different Fine-Tuning Tasks

- For Language modeling
- For Sentence prediction
- For Text classification
- For Token classification
- For Text encoding
- For Multimodal modeling

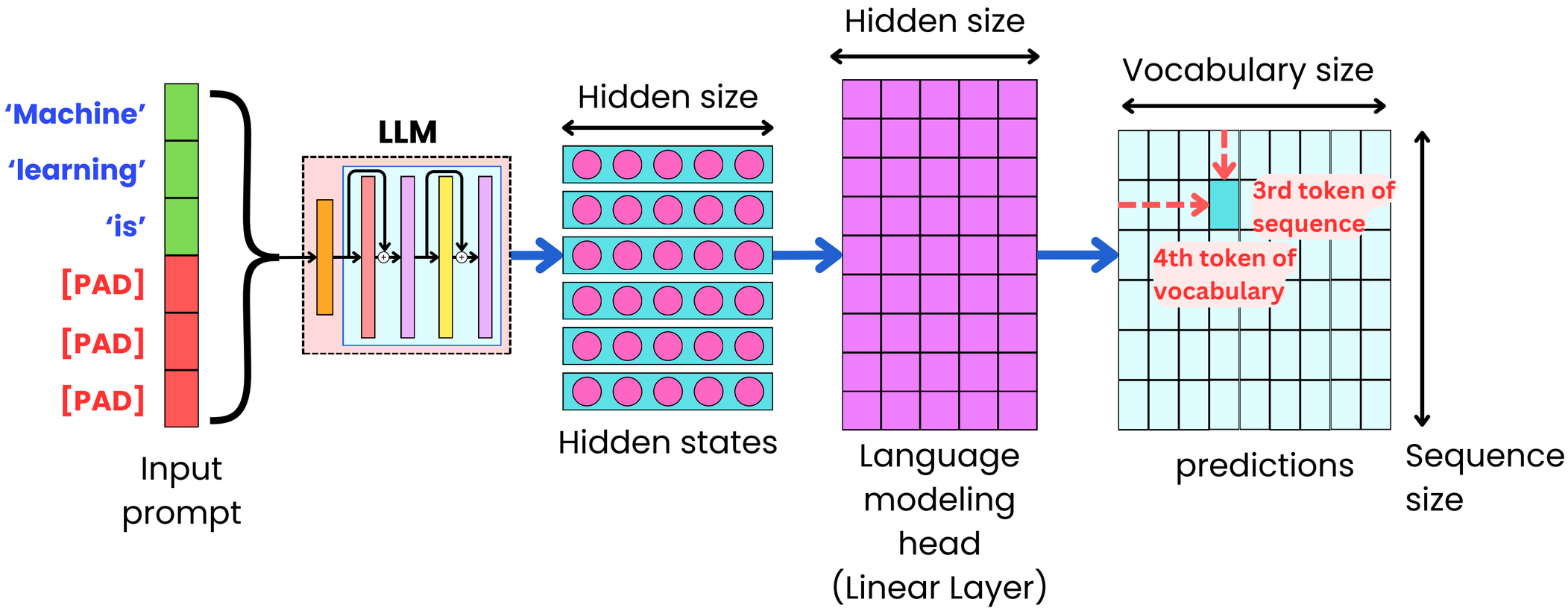
Causal Language Modeling



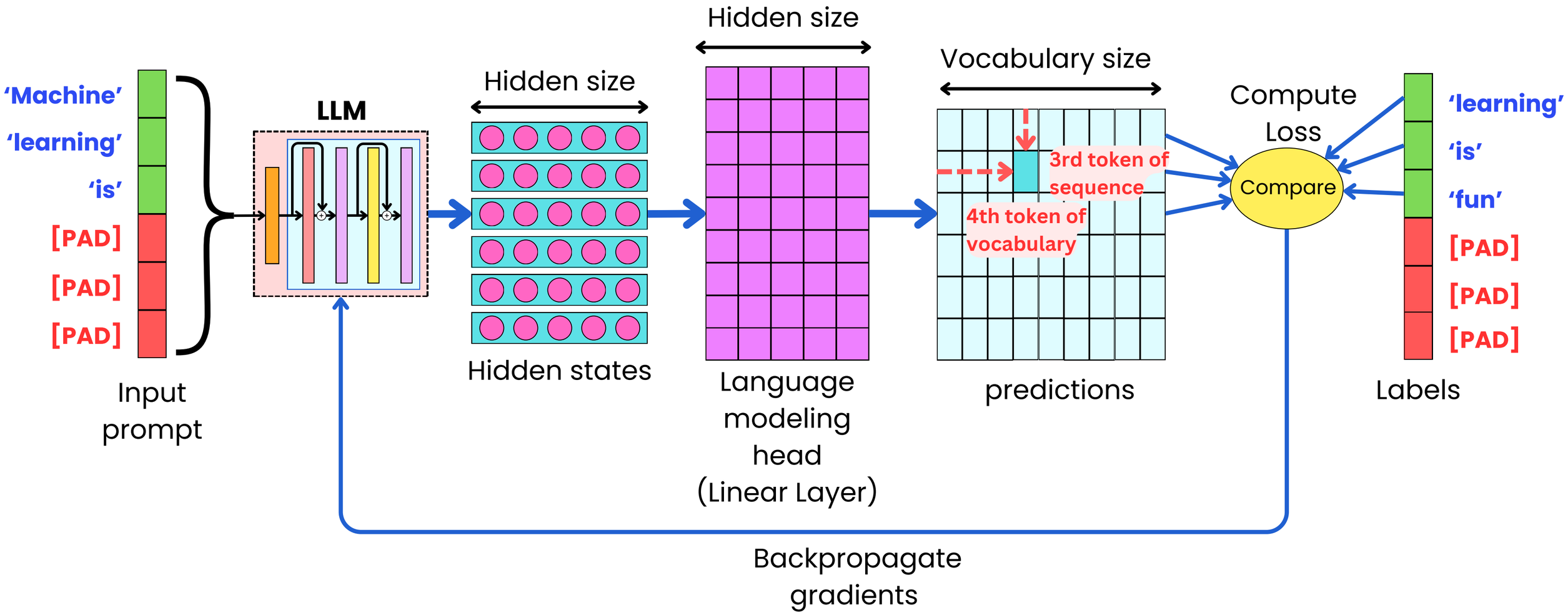
Causal Language Modeling



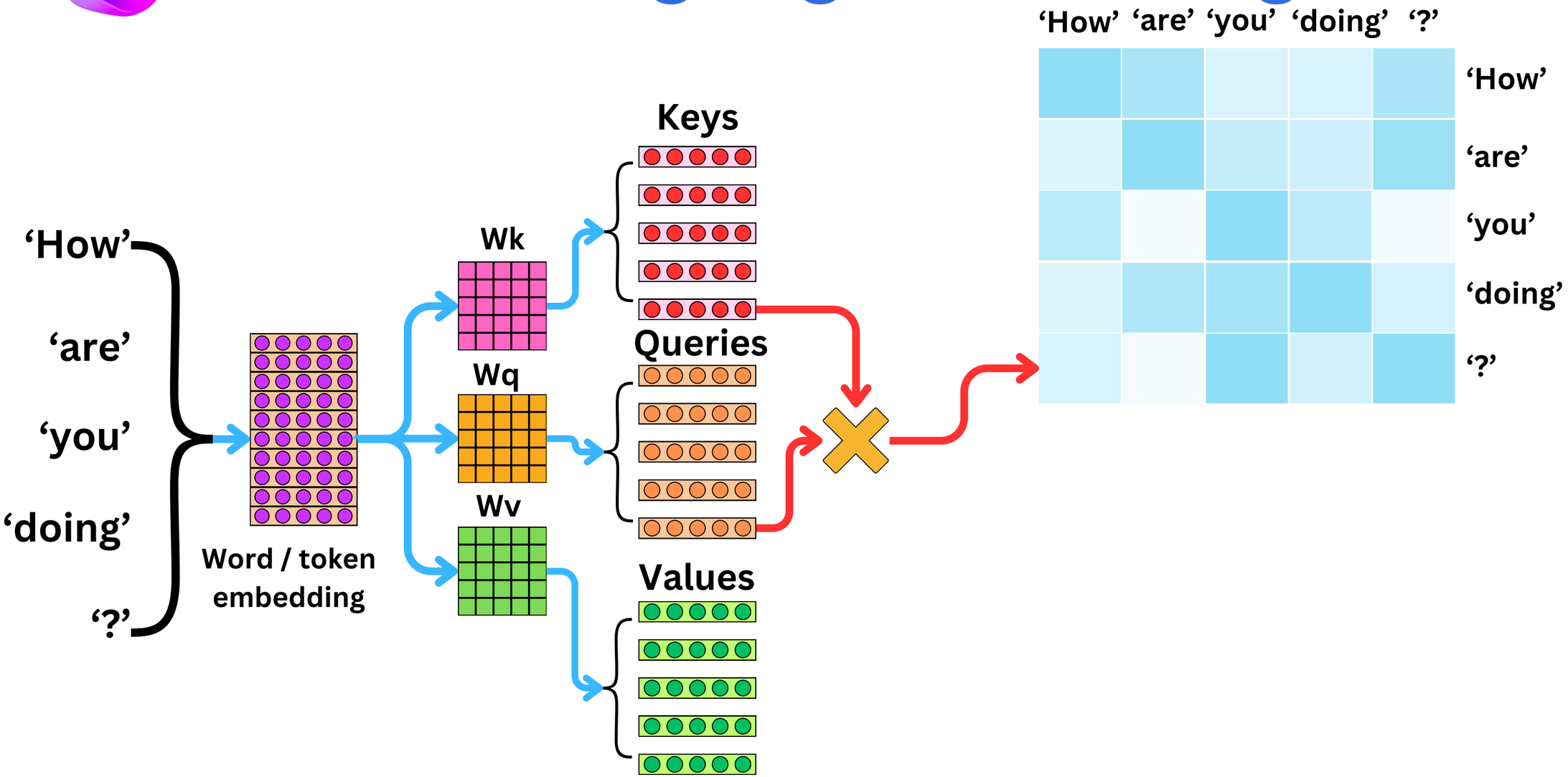
Causal Language Modeling



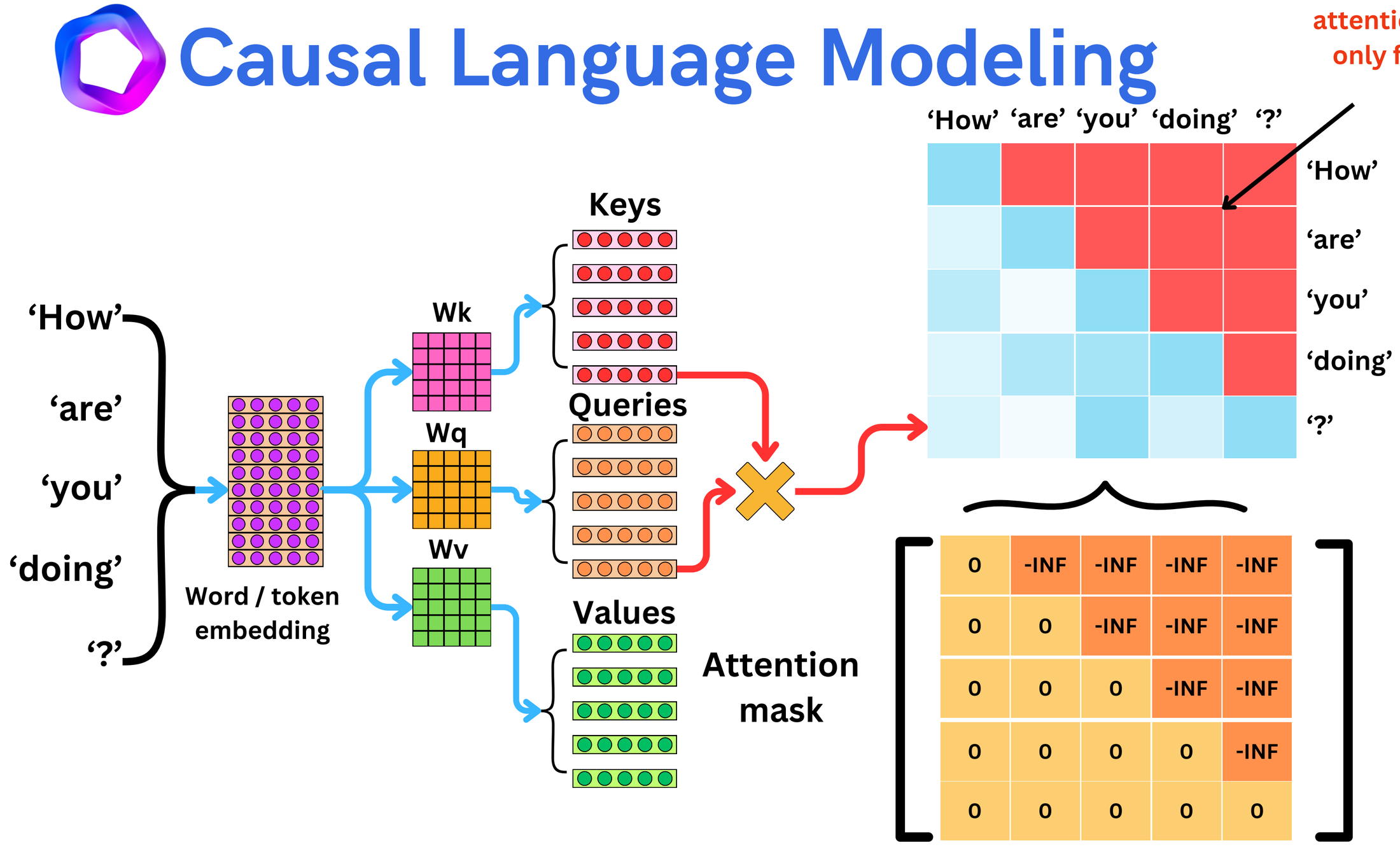
Causal Language Modeling



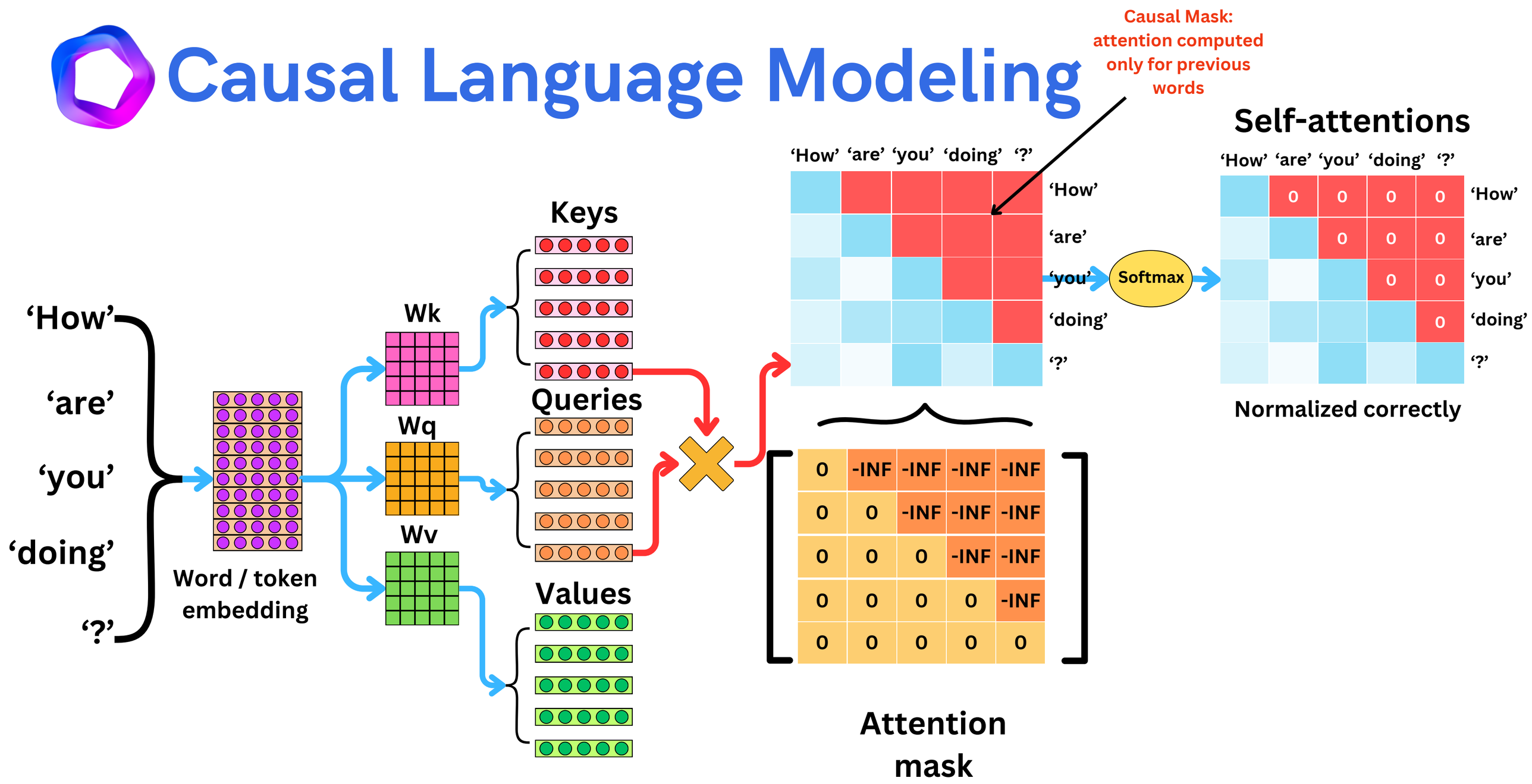
Causal Language Modeling



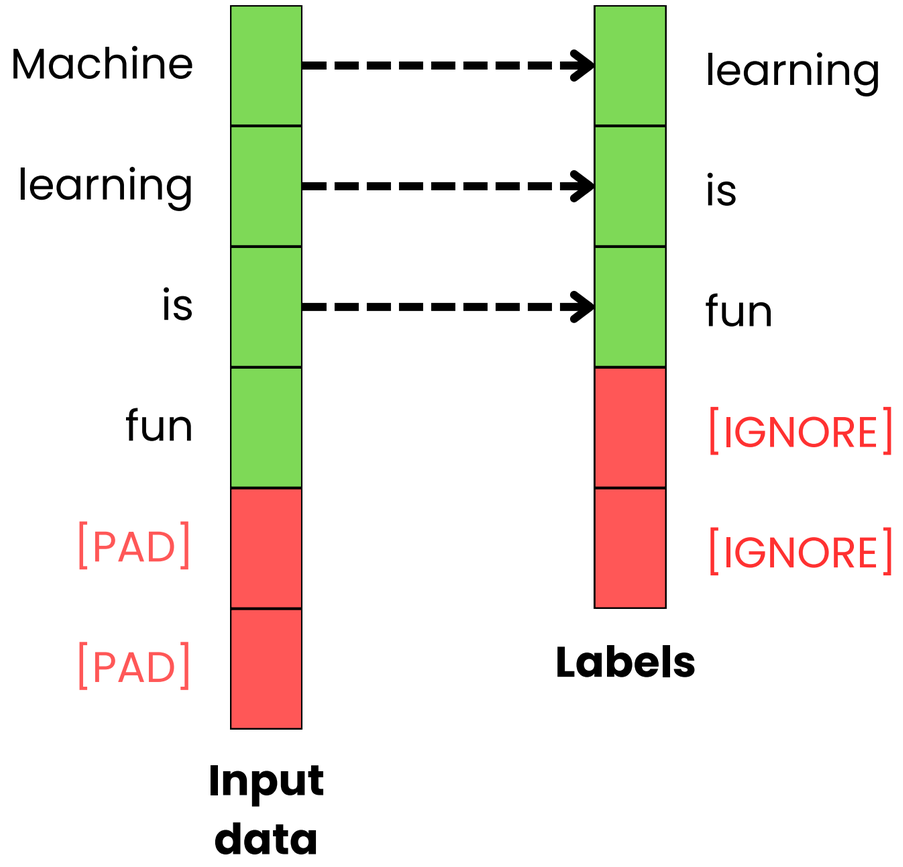
Causal Language Modeling



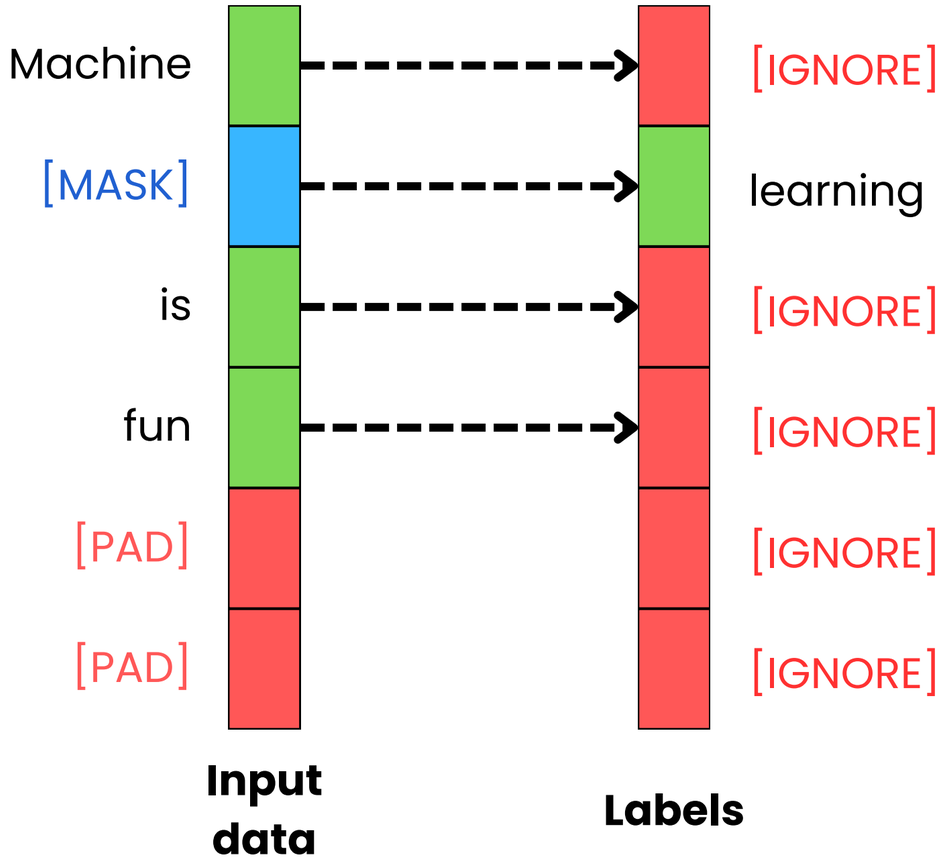
Causal Language Modeling



Masked Language Modeling

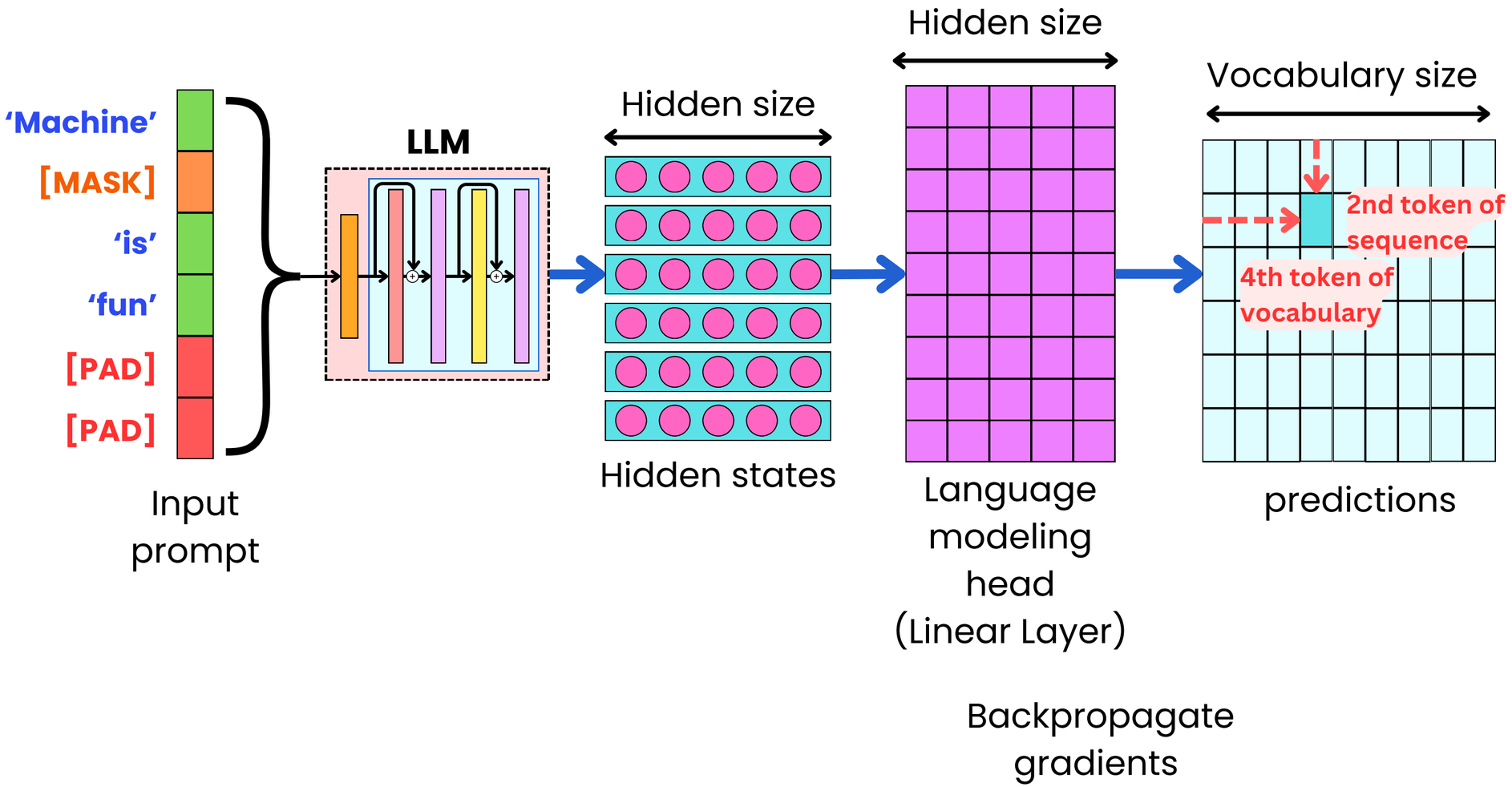


Causal Language Modeling

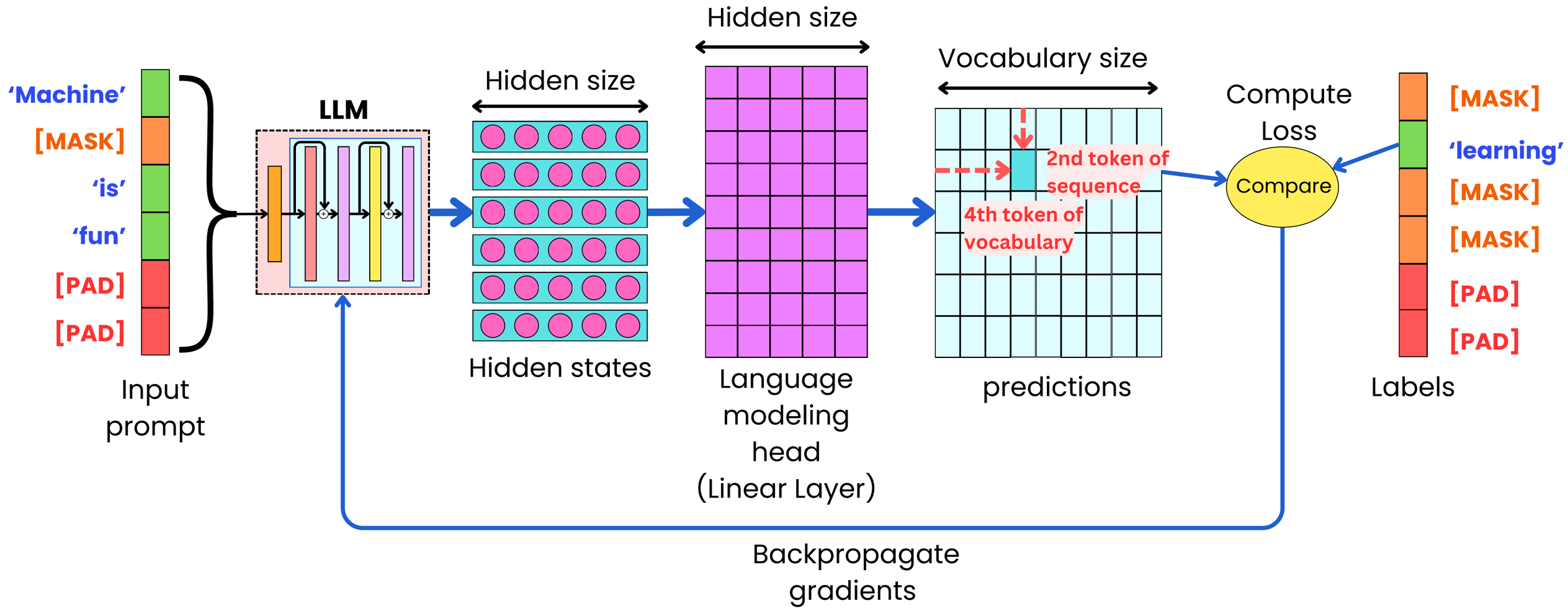


Masked Language Modeling

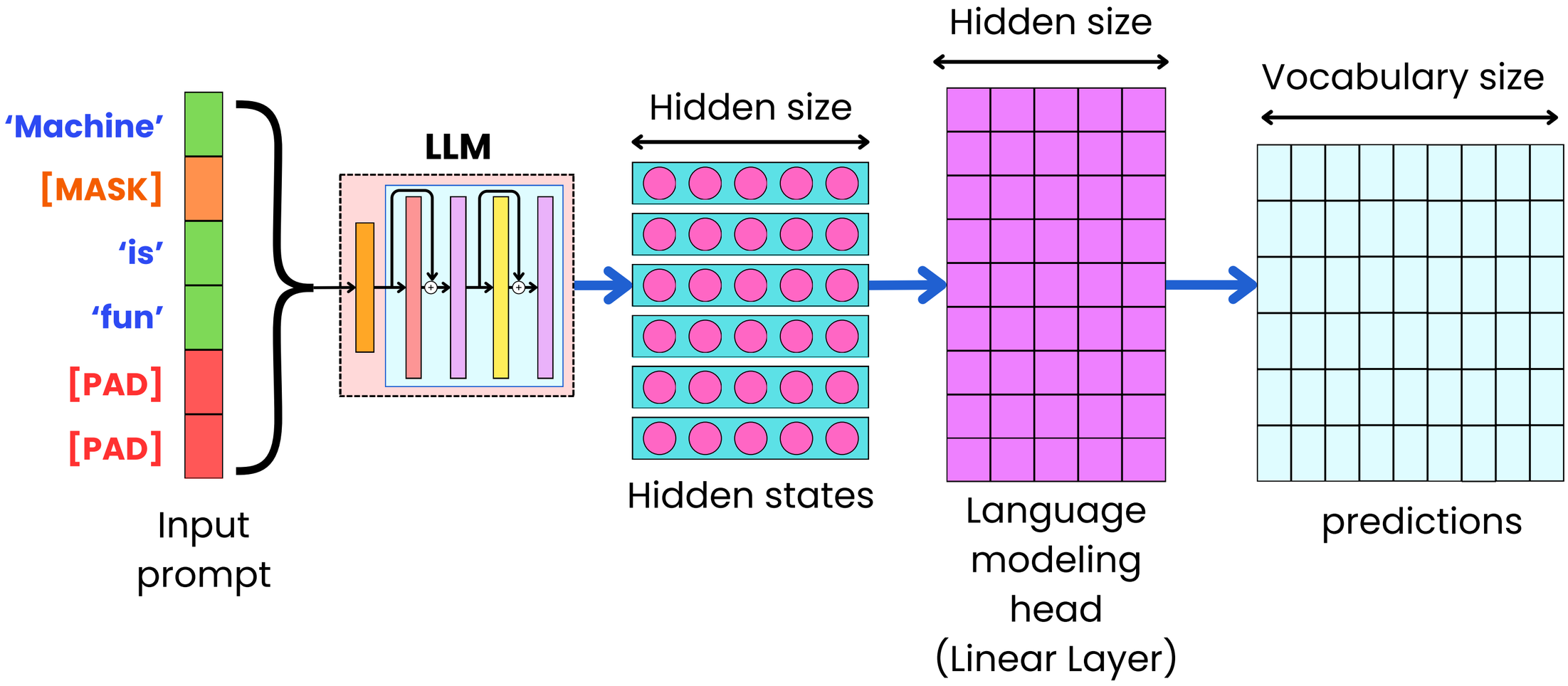
Masked Language Modeling



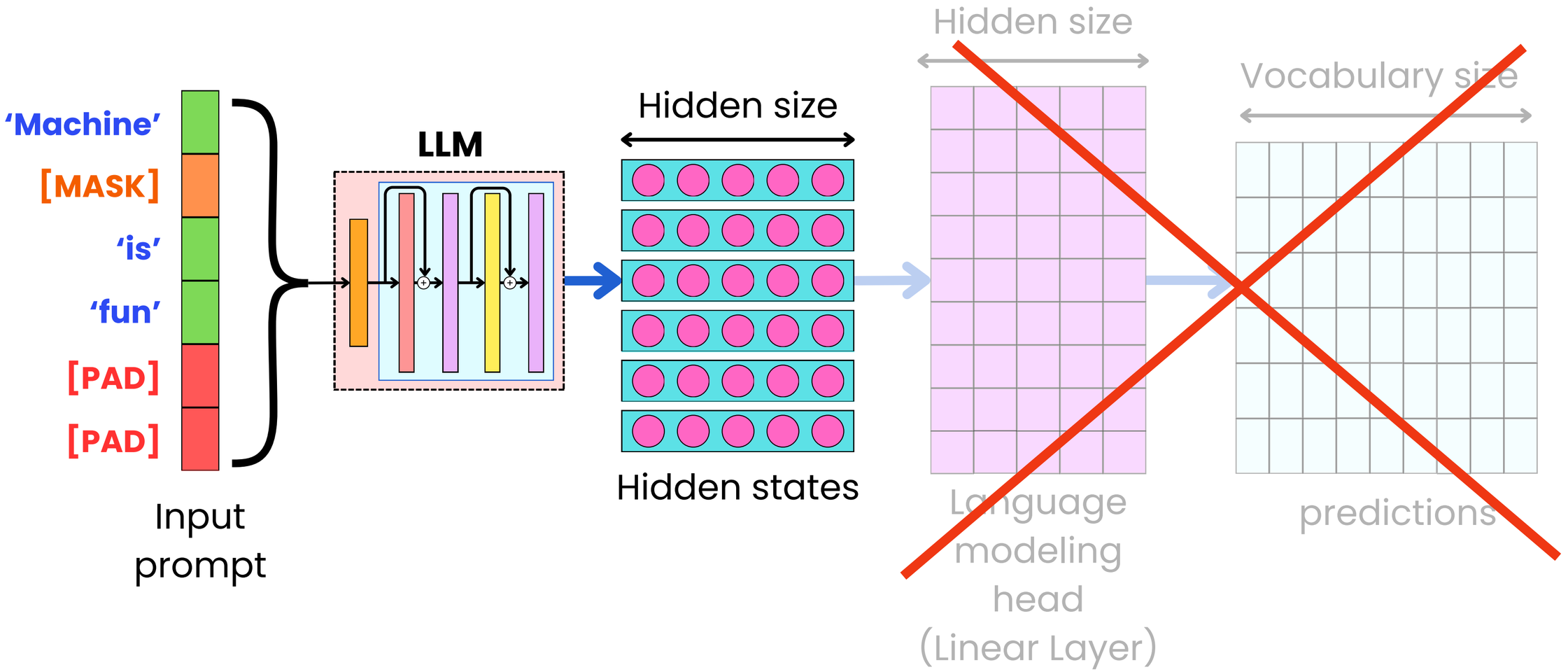
Masked Language Modeling



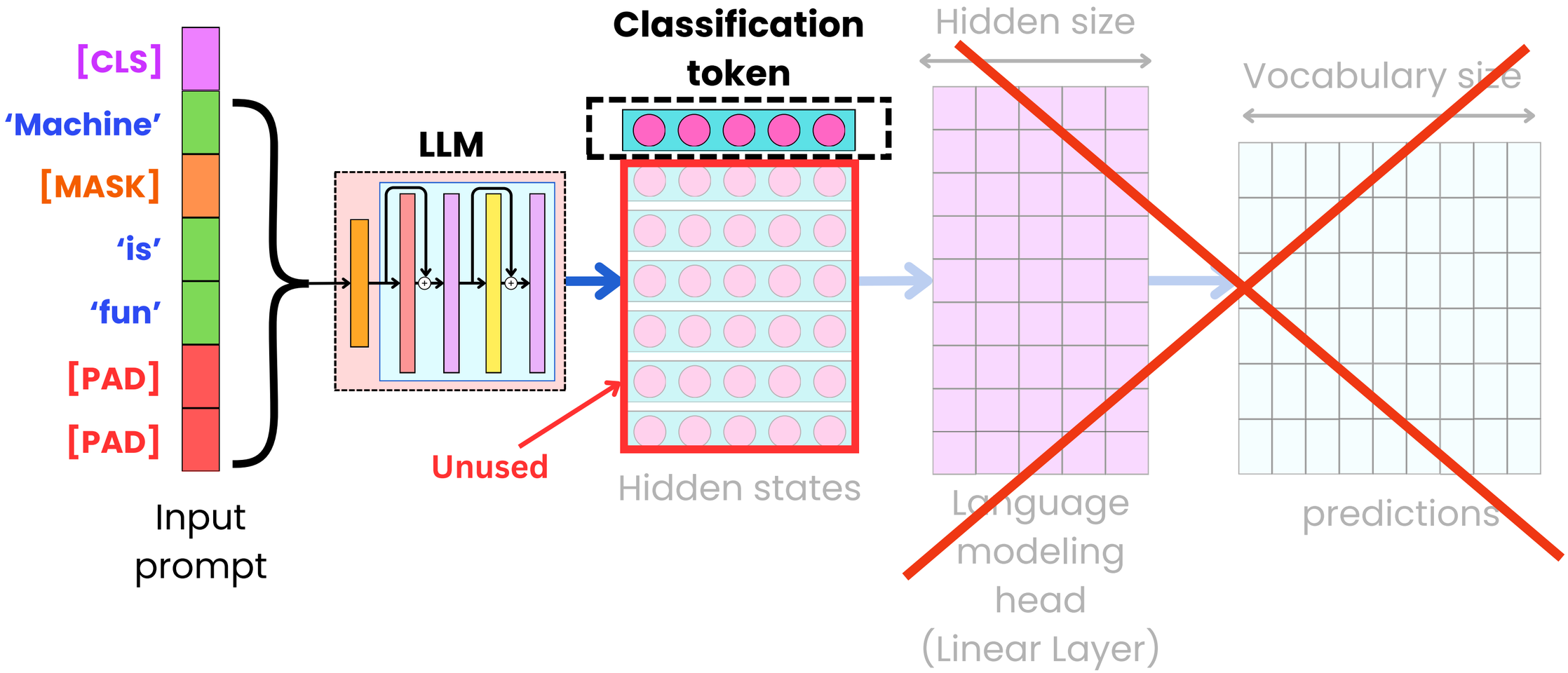
Masked Language Modeling



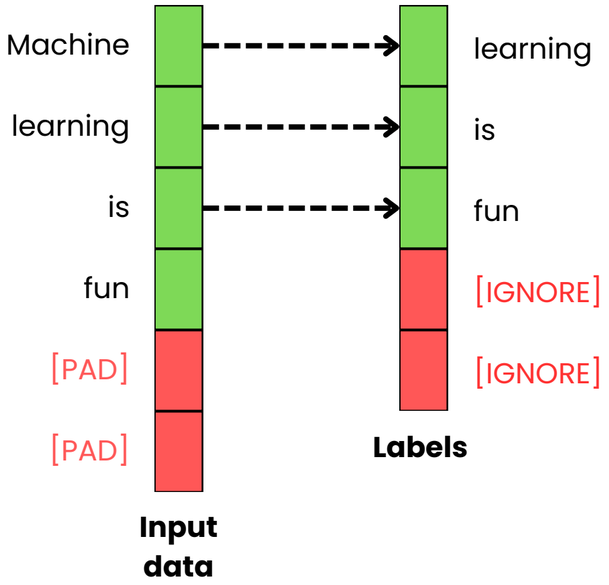
Masked Language Modeling



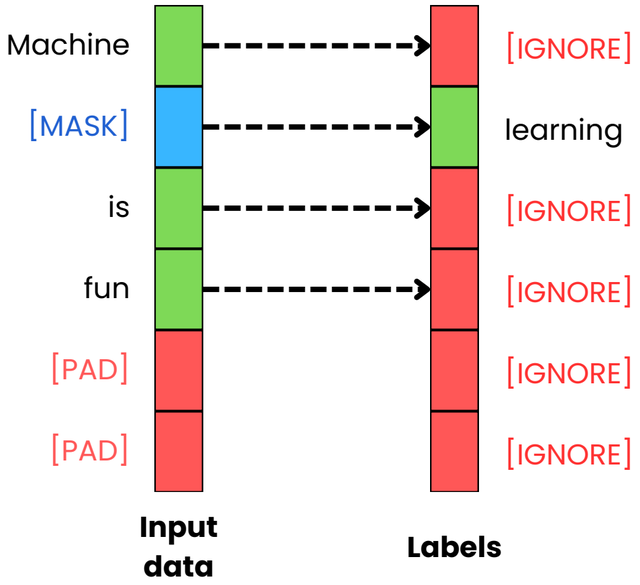
Masked Language Modeling



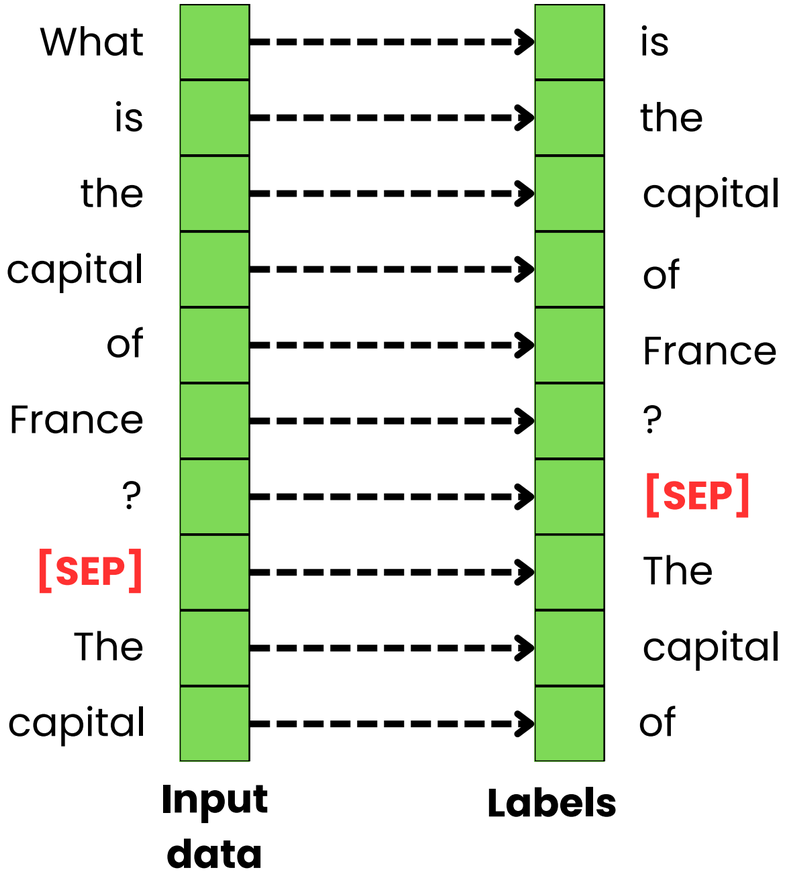
Sequence Prediction



Causal Language Modeling

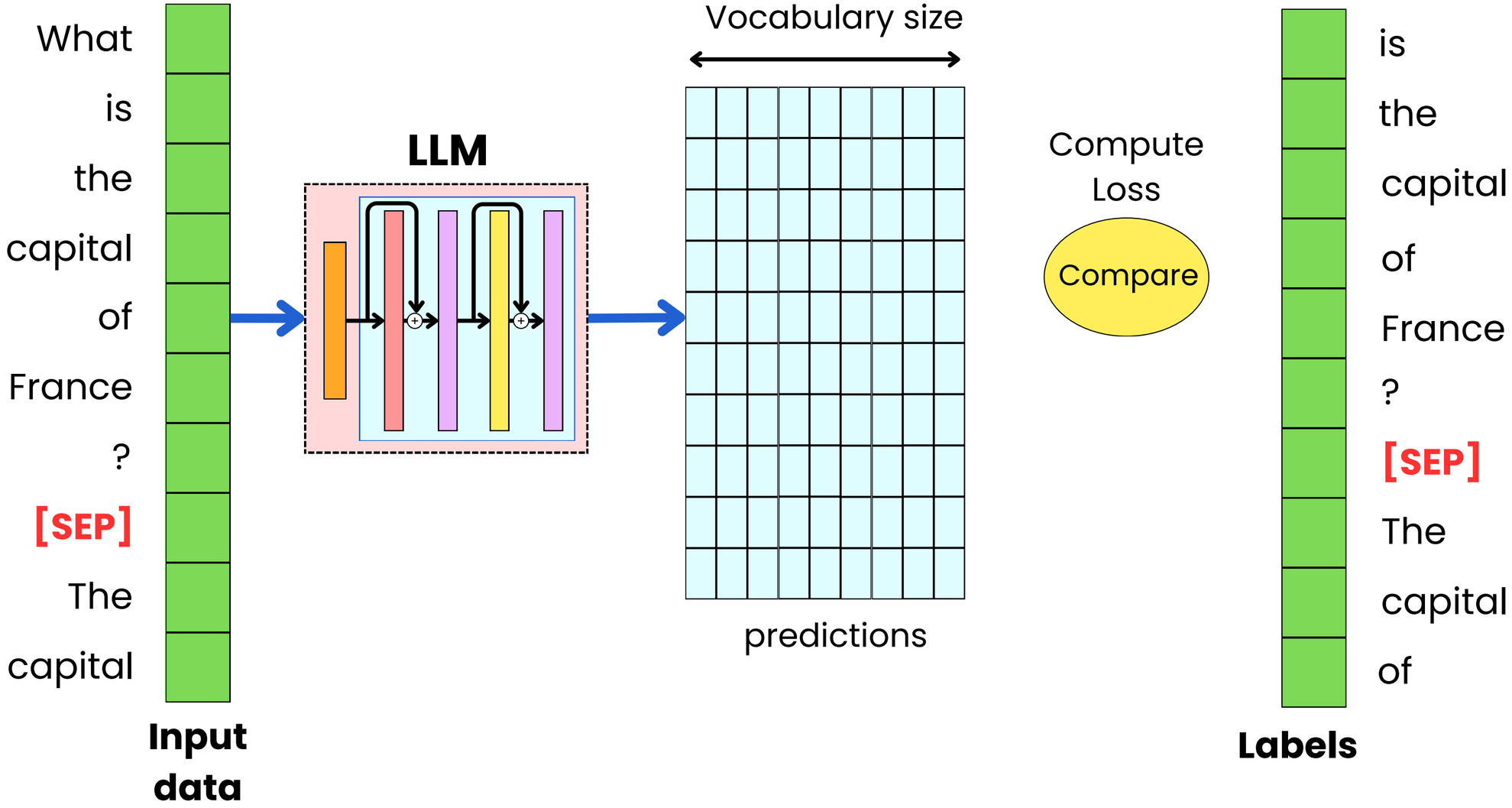


Masked Language Modeling

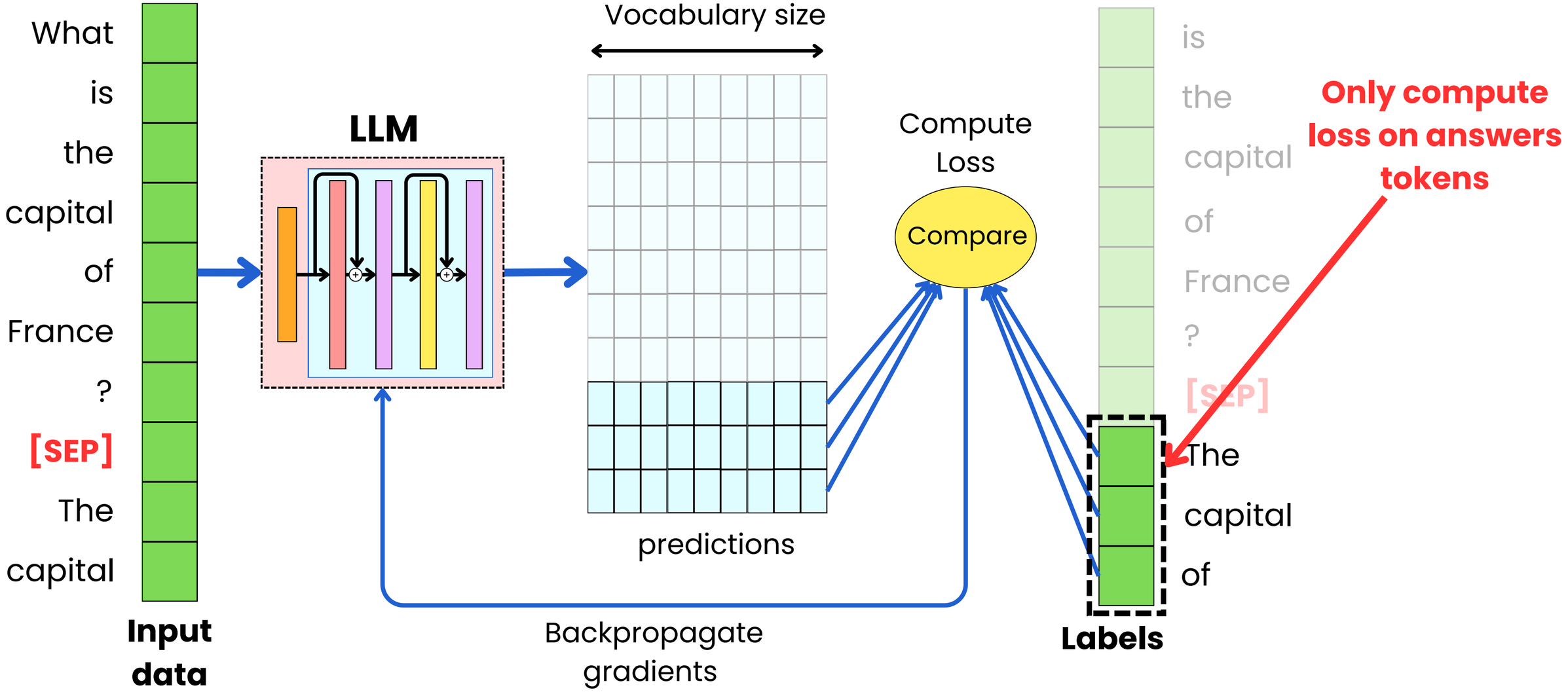


Sequence Prediction

Sequence Prediction

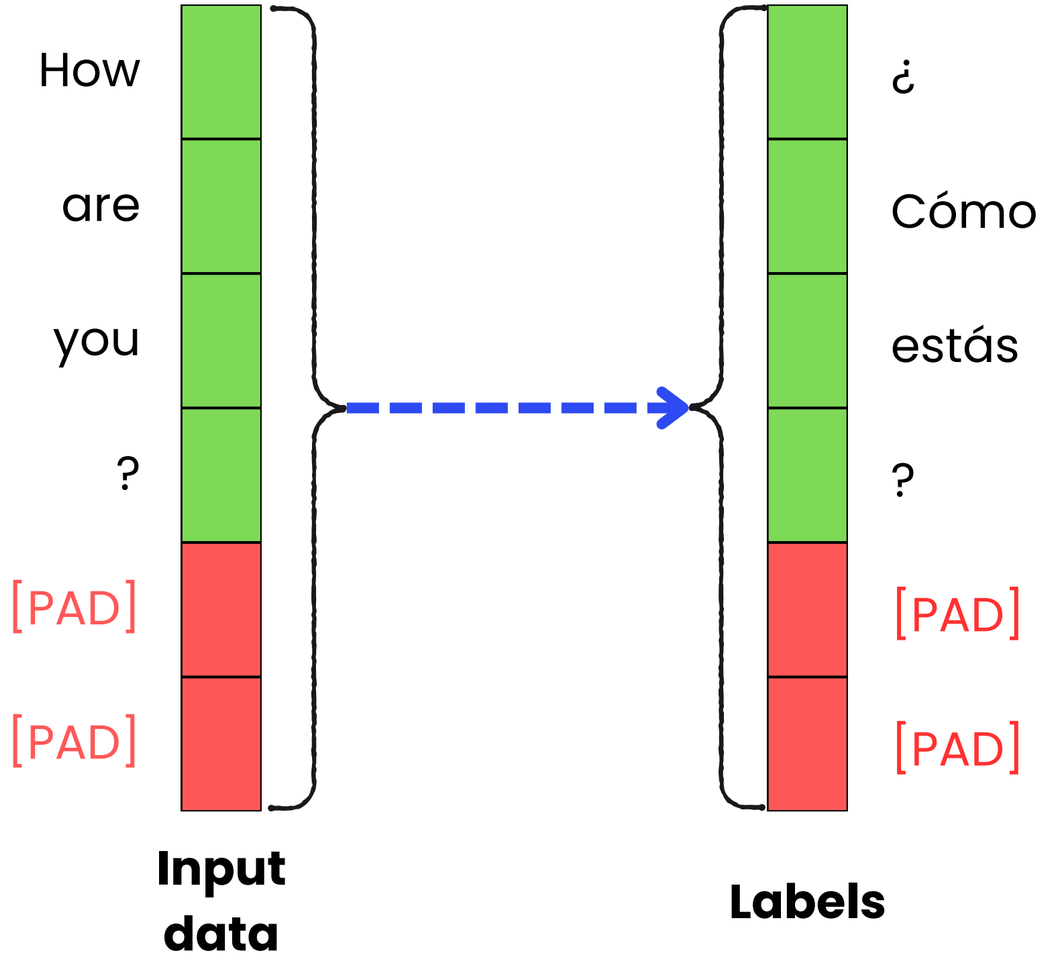


Sequence Prediction



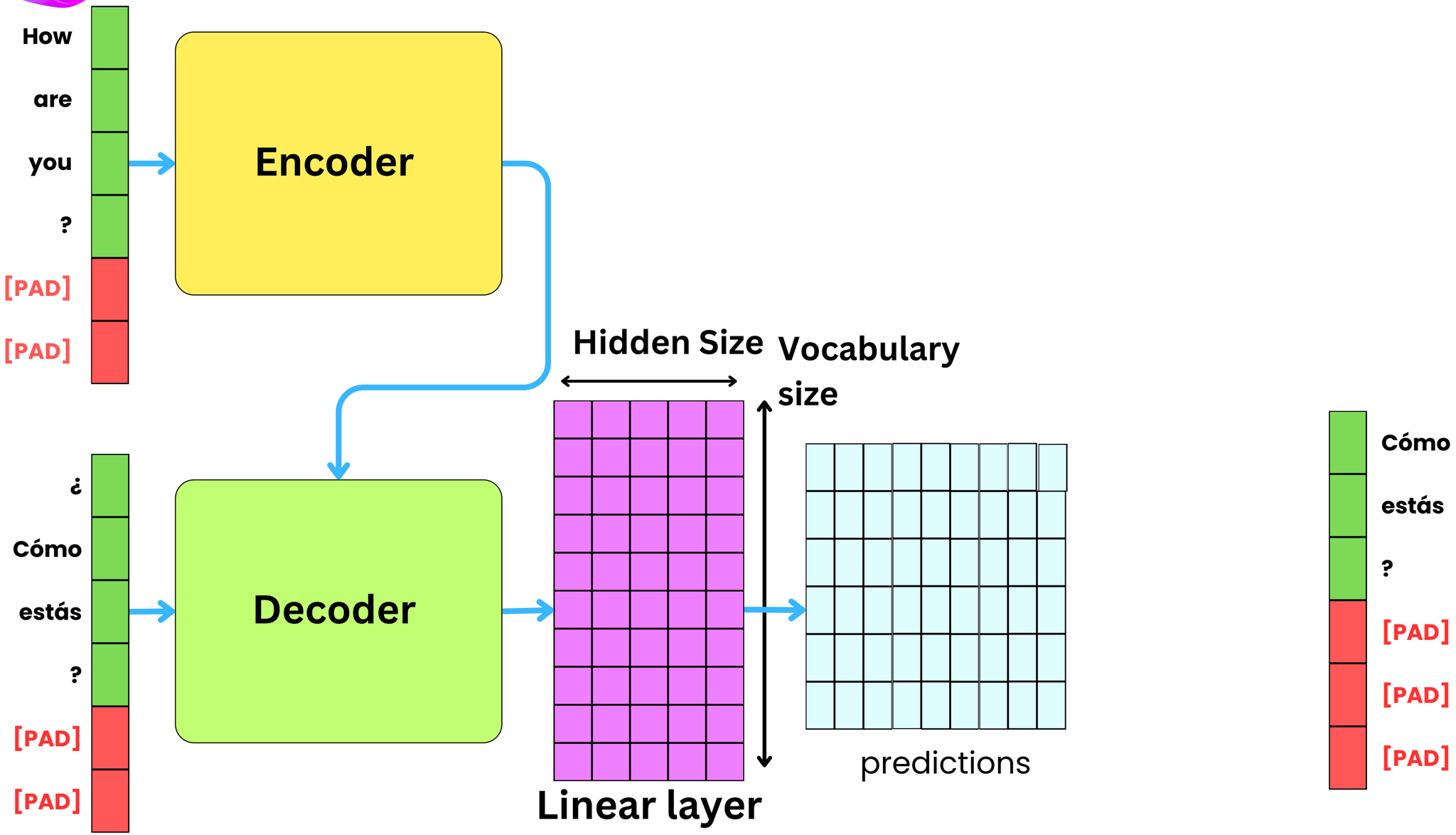
Sequence Prediction

- Machine translation
- Text summarization
- Question-answering
- Multimodal modeling

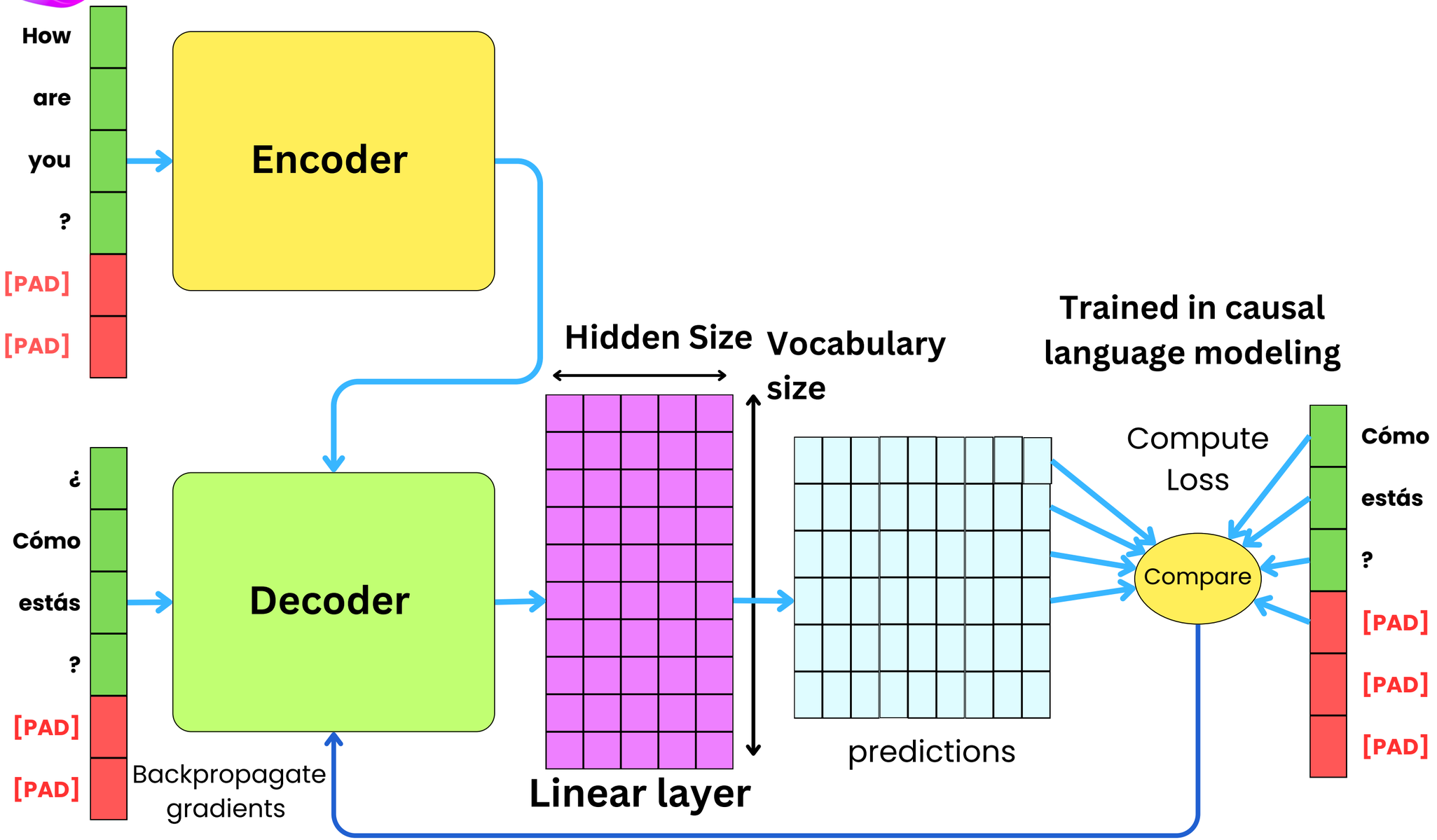


Sequence-to-sequence

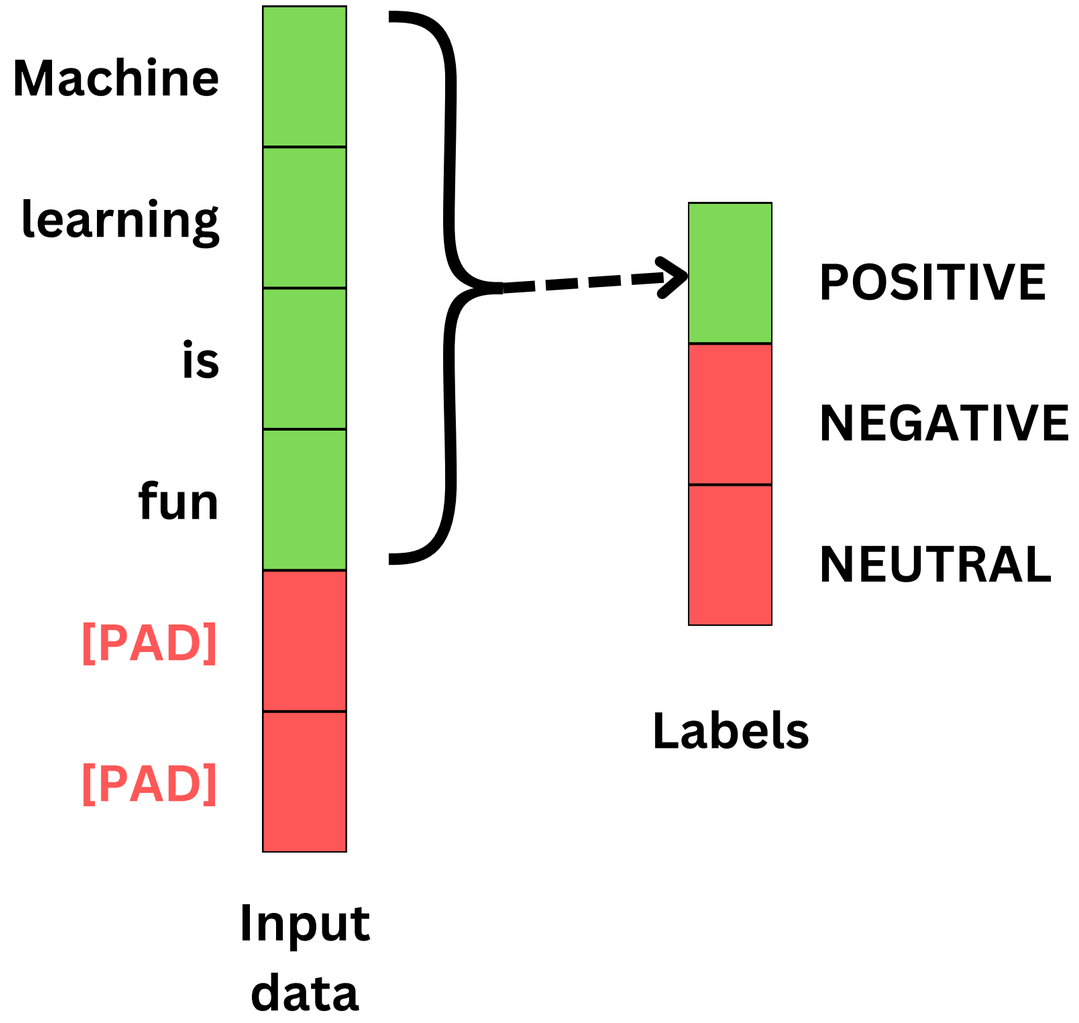
Sequence Prediction



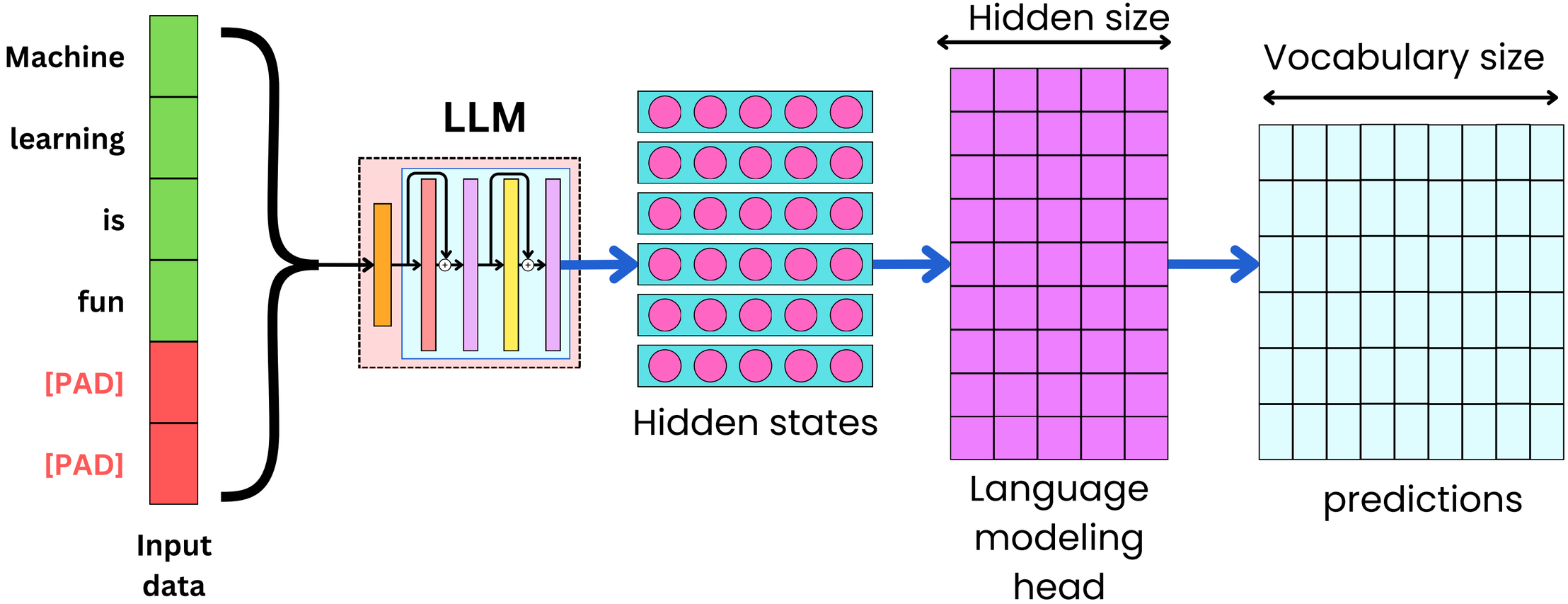
Sequence Prediction



Text Classification

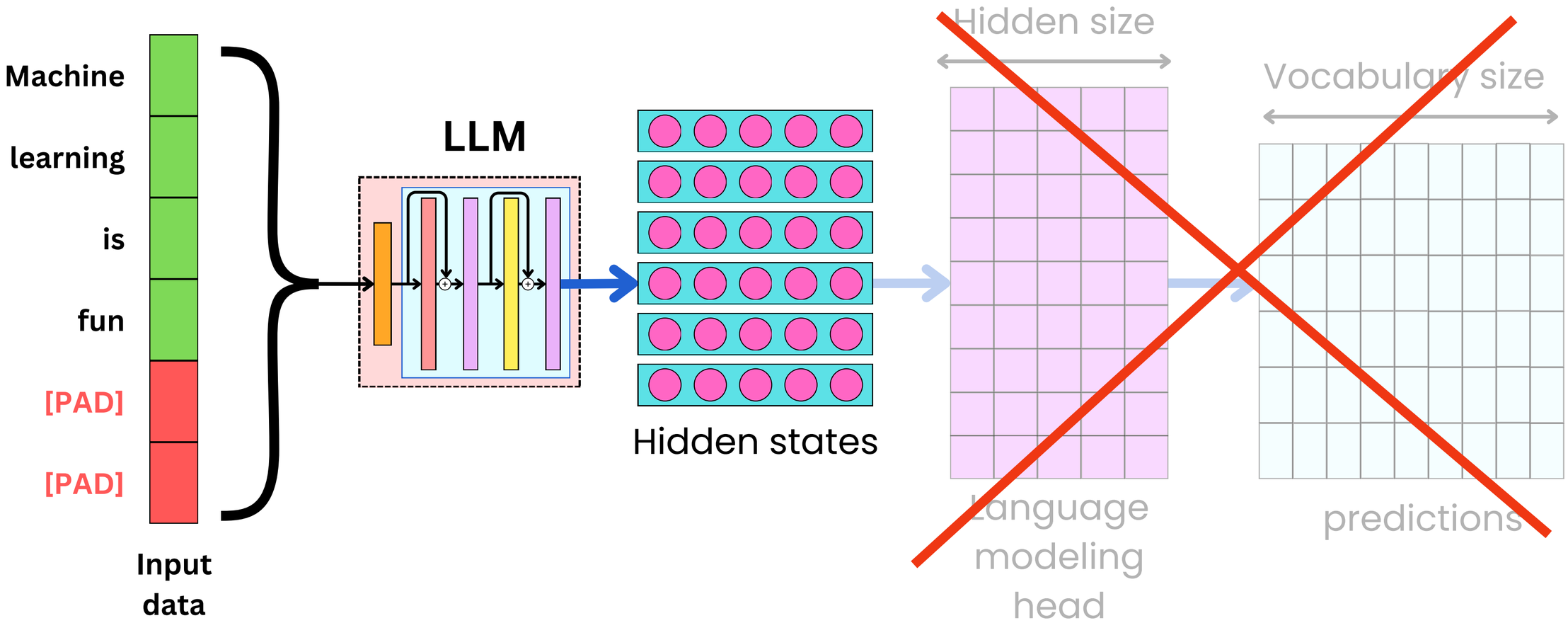


Text Classification



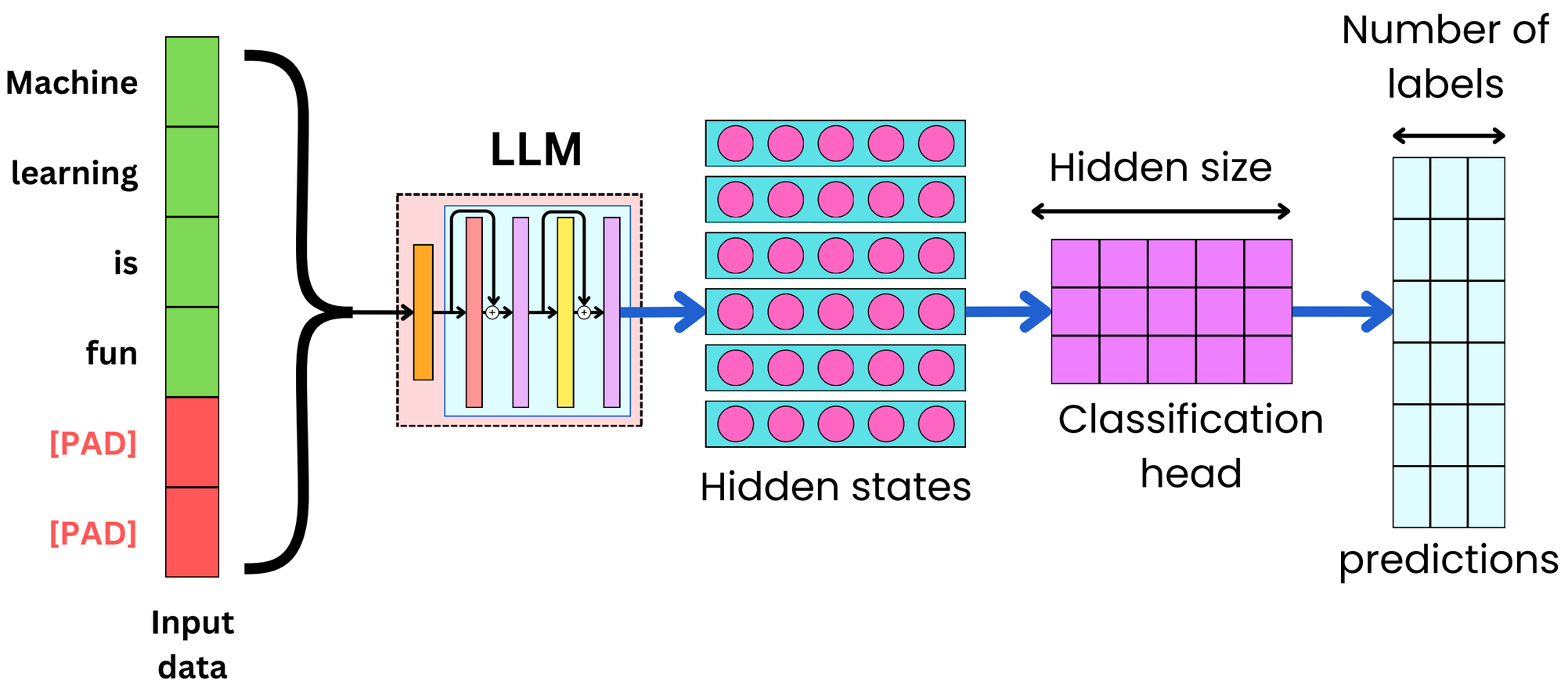
We start from a pre-trained LLM

Text Classification



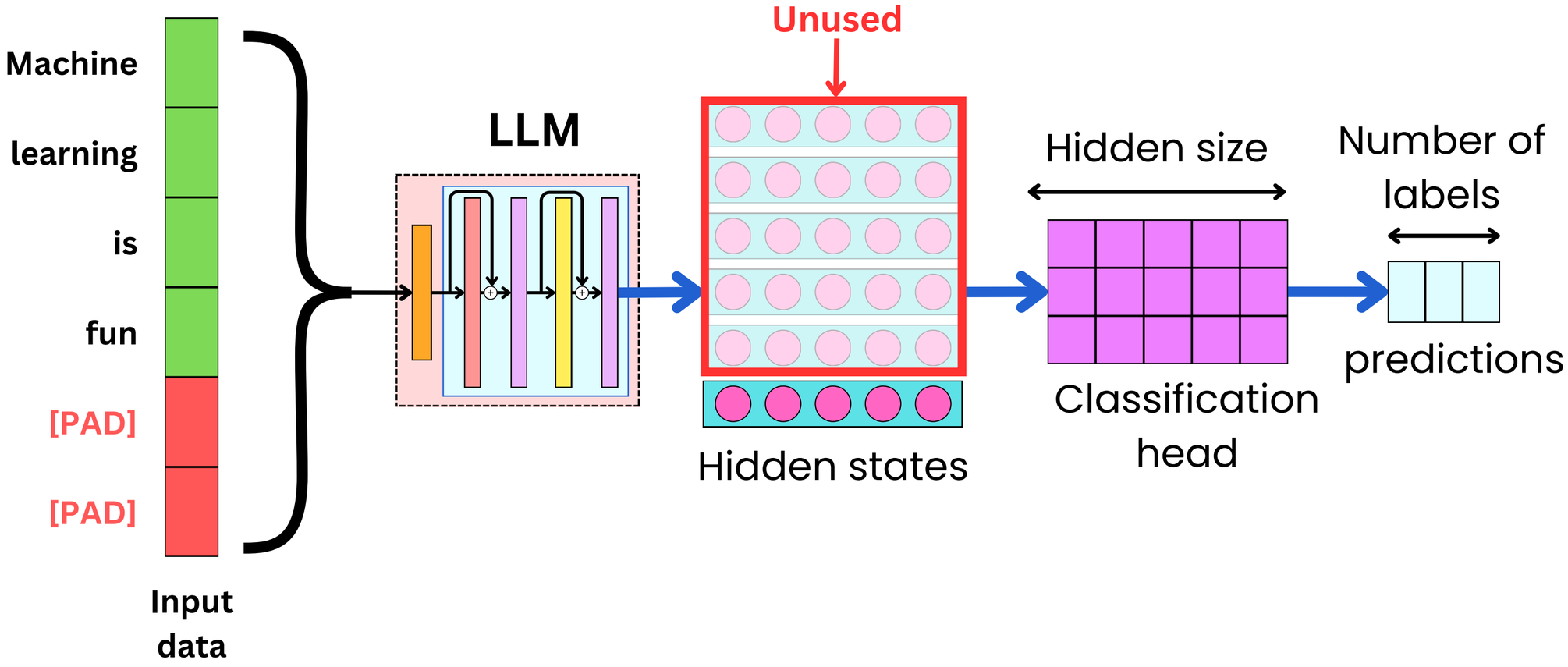
We remove the language modeling head

Text Classification



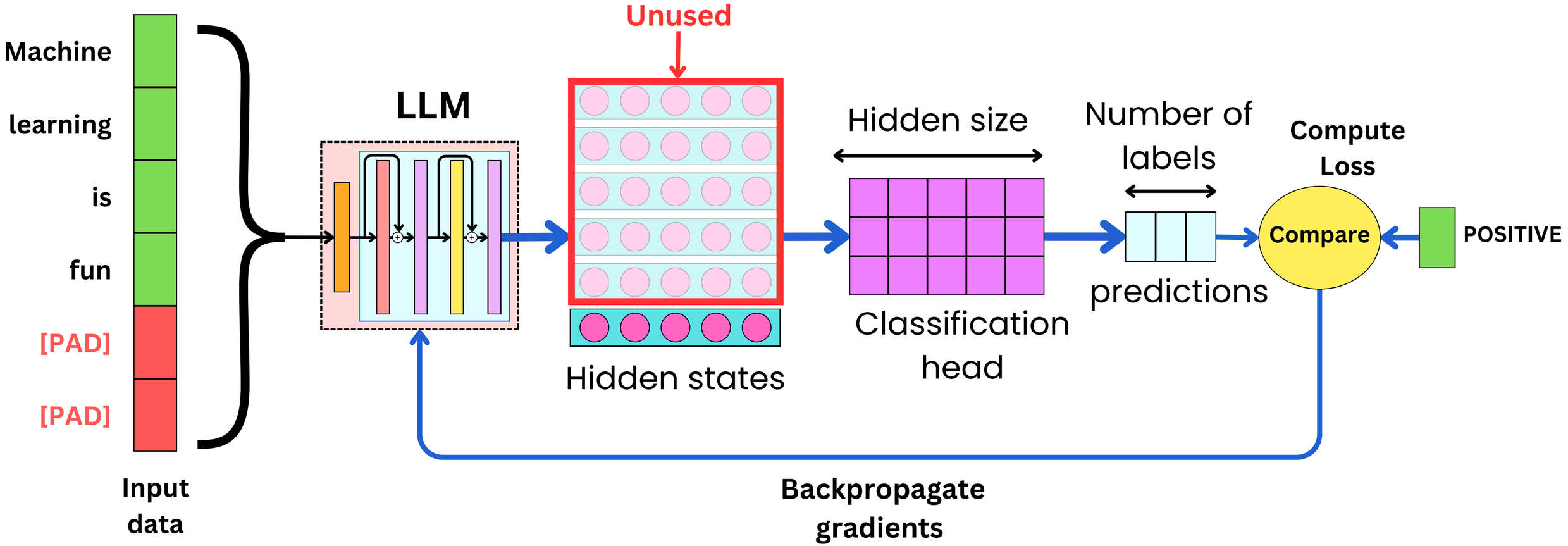
We replace the language modeling head by a classification head

Text Classification



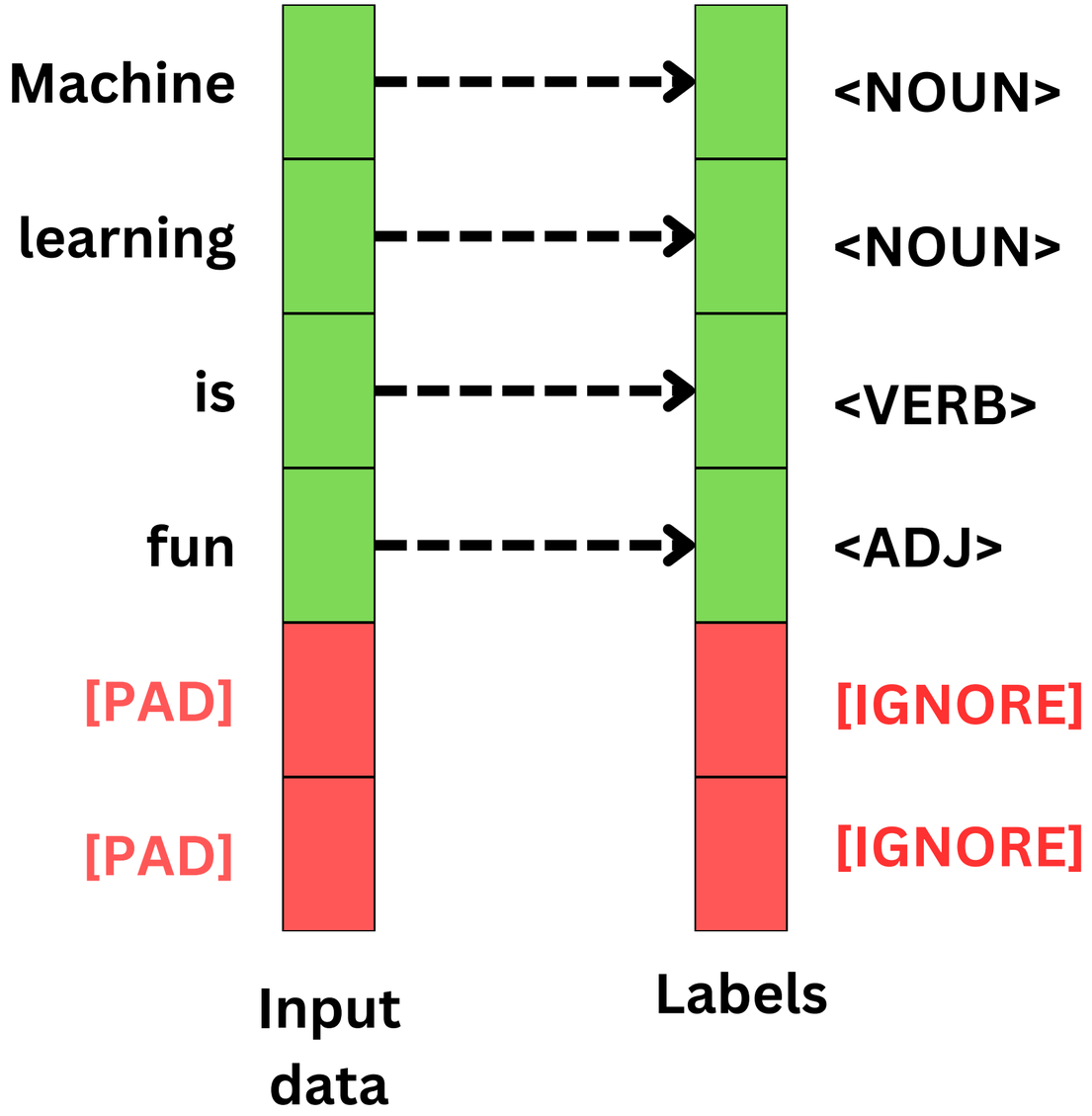
We only use ONE hidden state

Text Classification

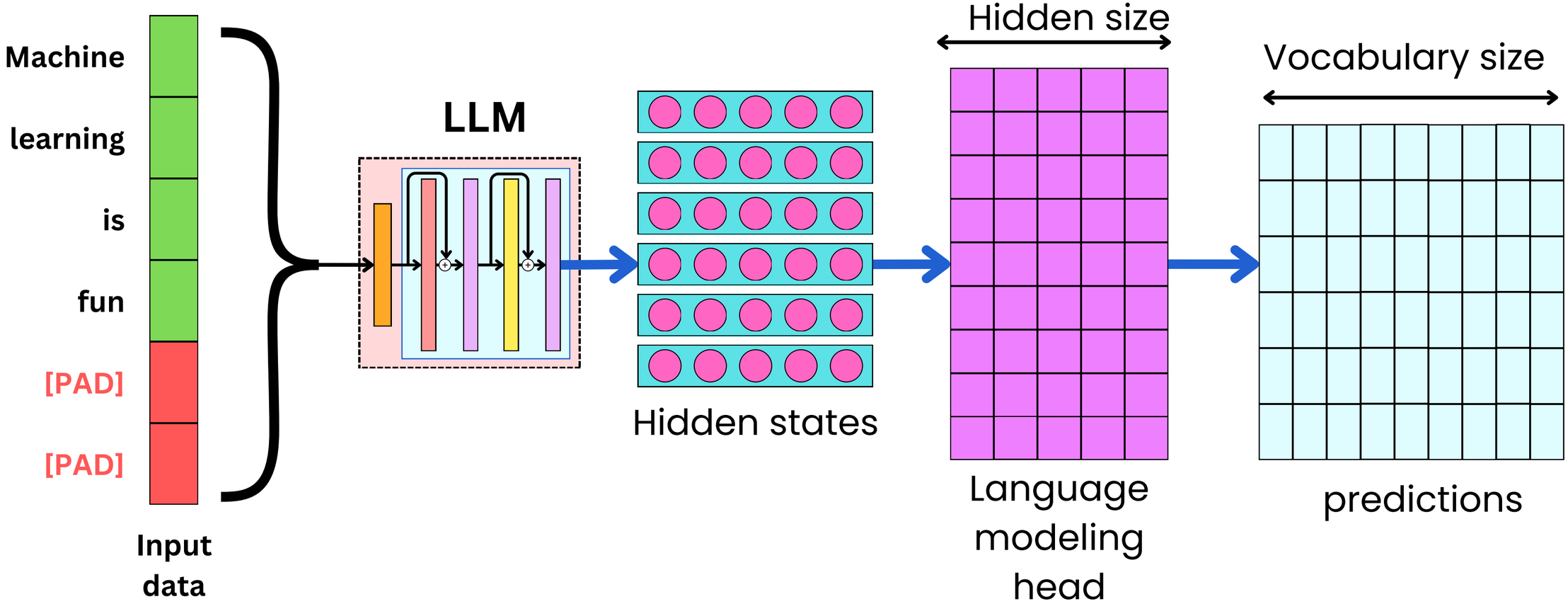


We can now compare the predictions to the targets

Token Classification

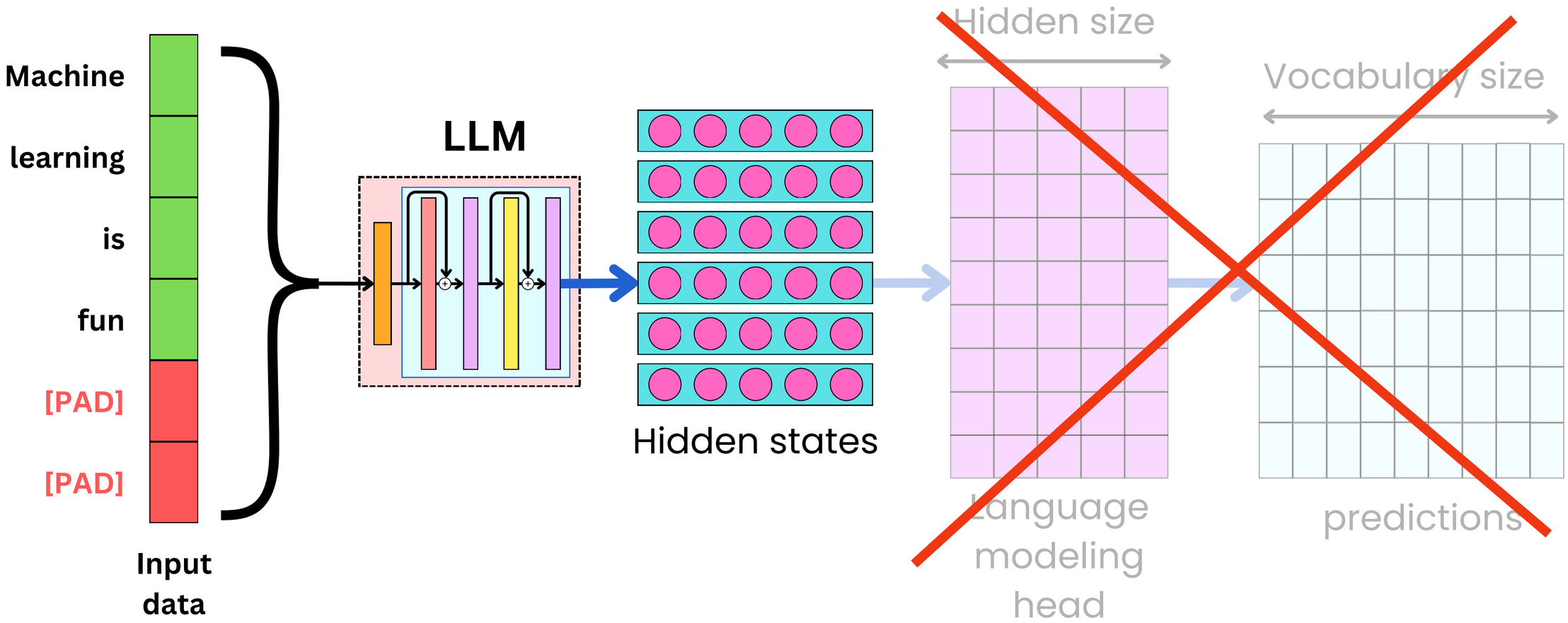


Token Classification



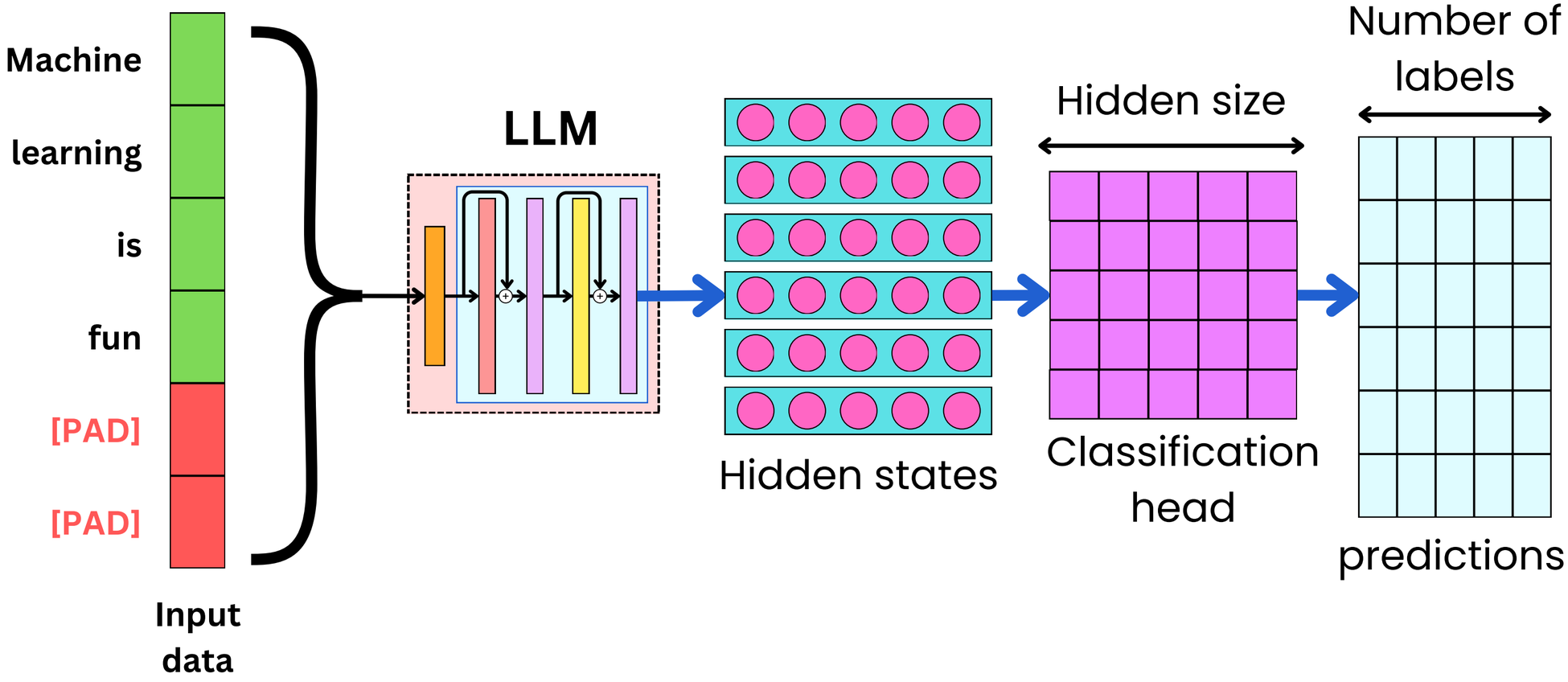
We start from a pre-trained LLM

Token Classification



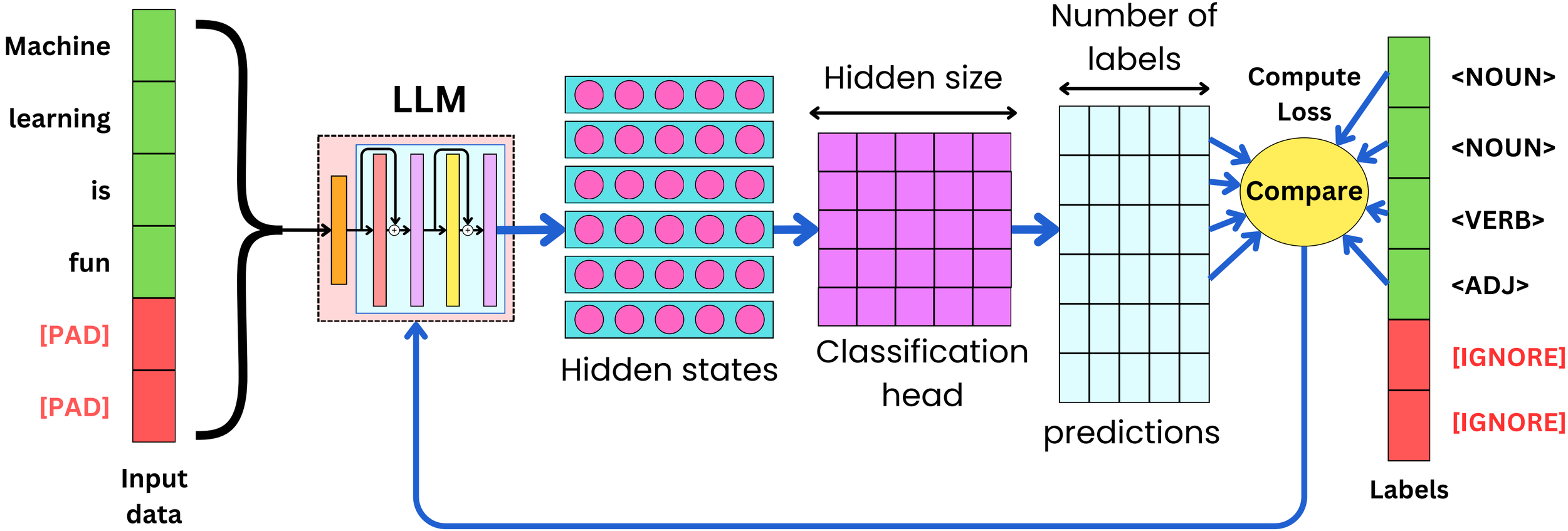
We remove the language modeling head

Token Classification

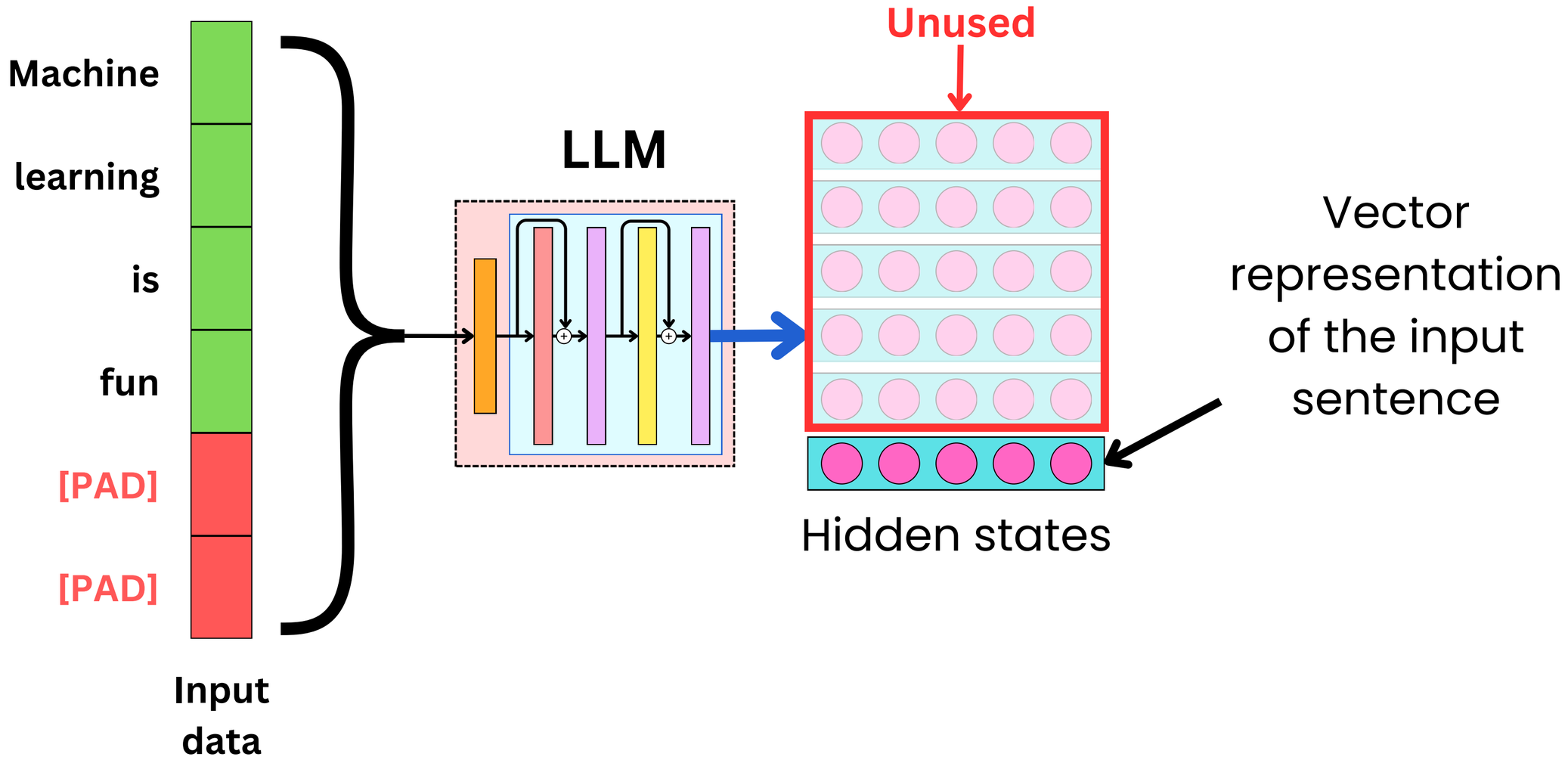


We replace the language modeling head by a classification head

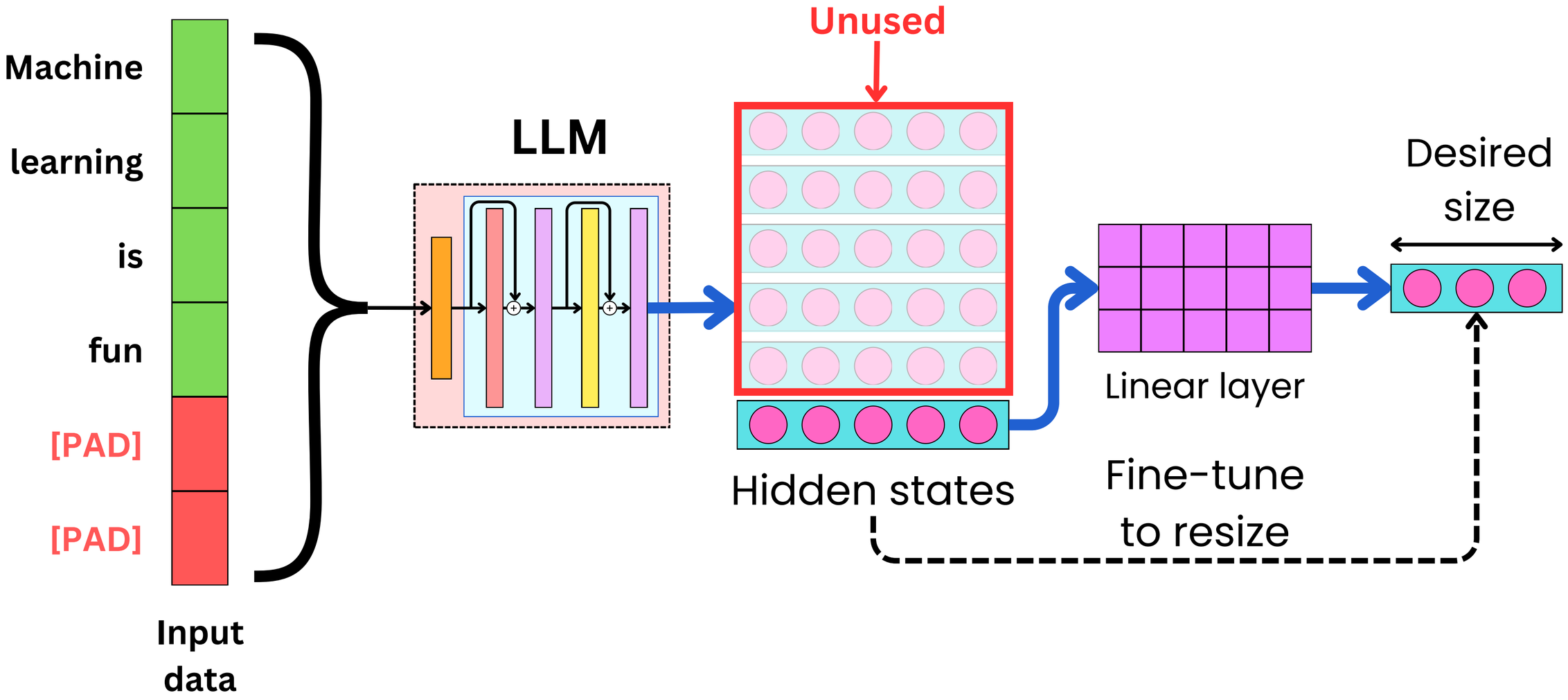
Token Classification



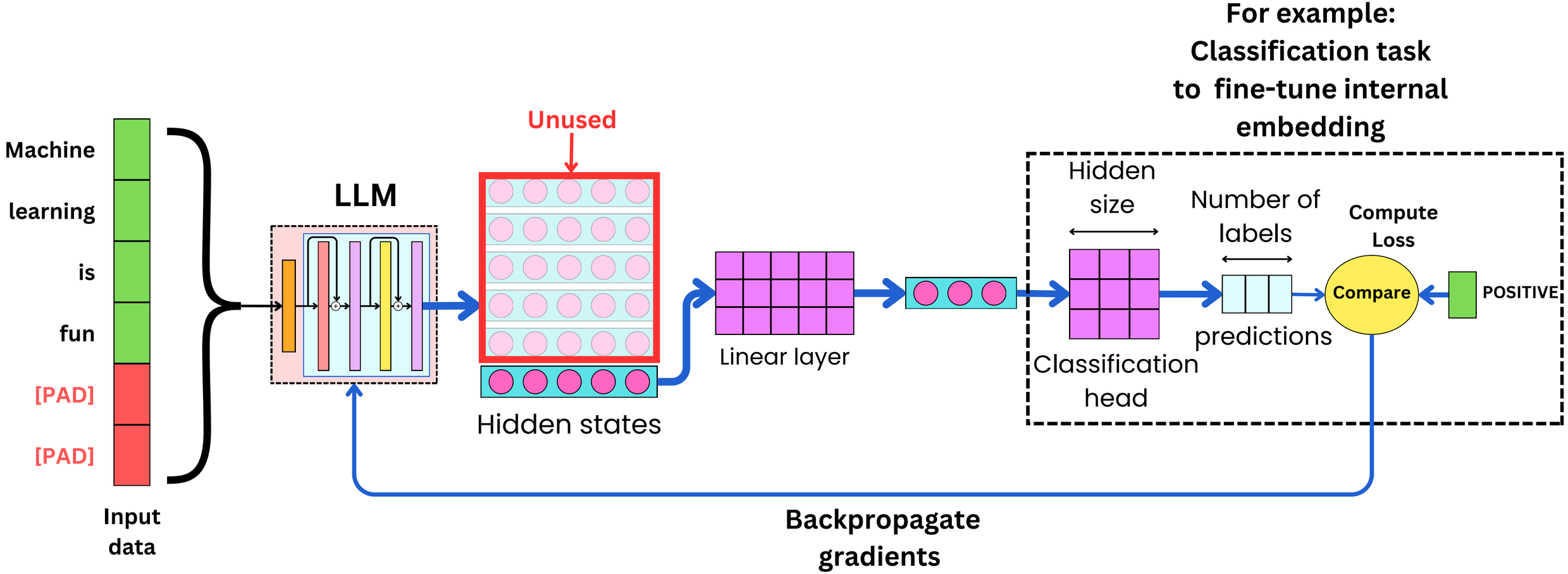
Text Encoding



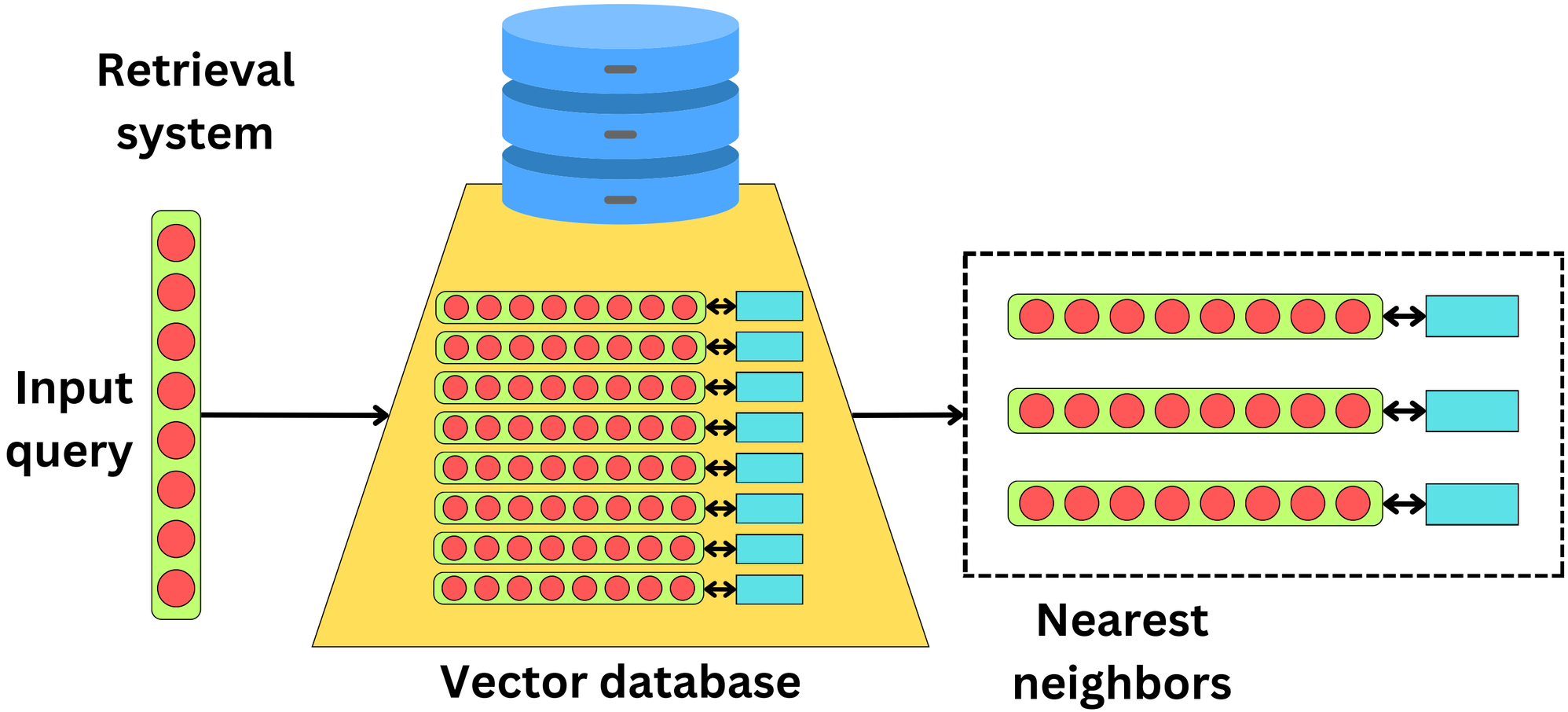
Text Encoding



Text Encoding

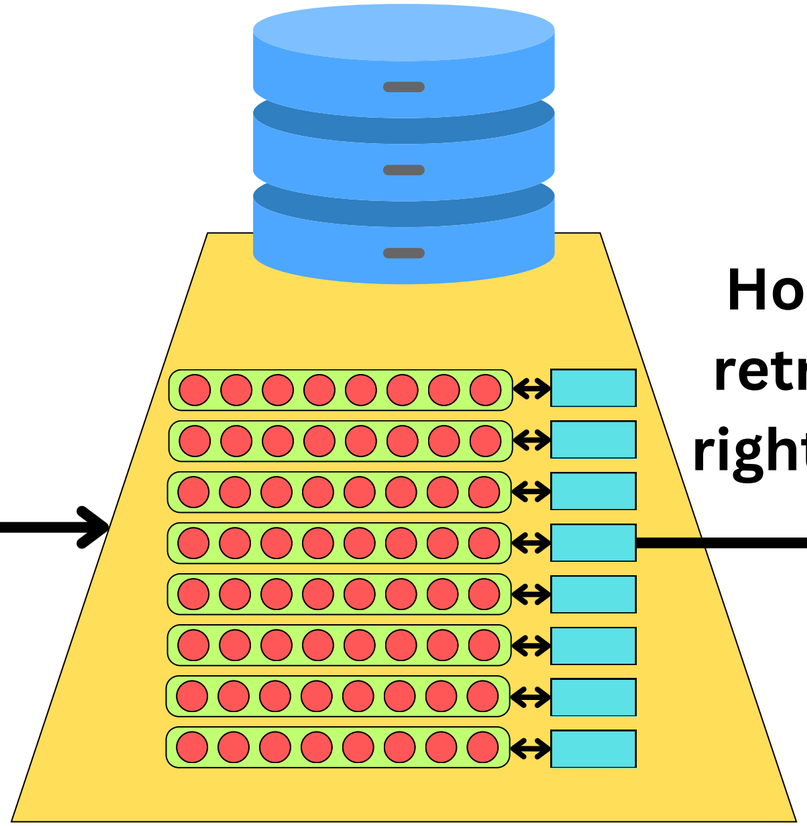


Text Encoding



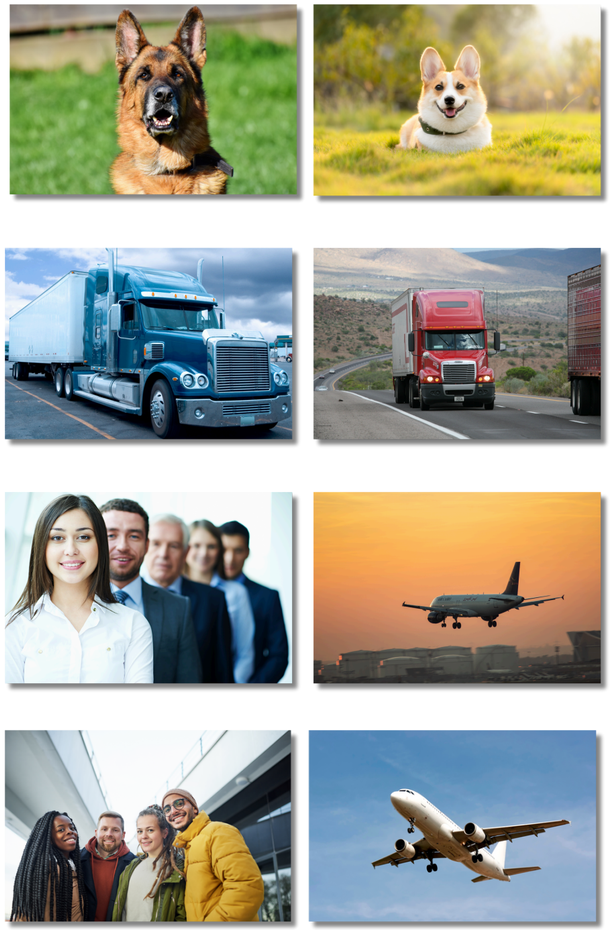
Text Encoding

"A DOG ON THE GRASS"

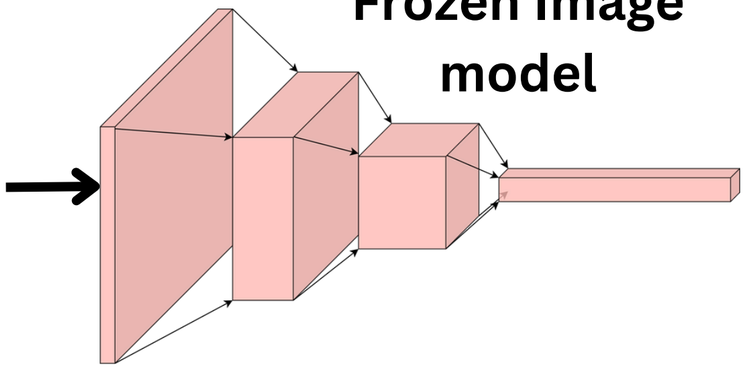


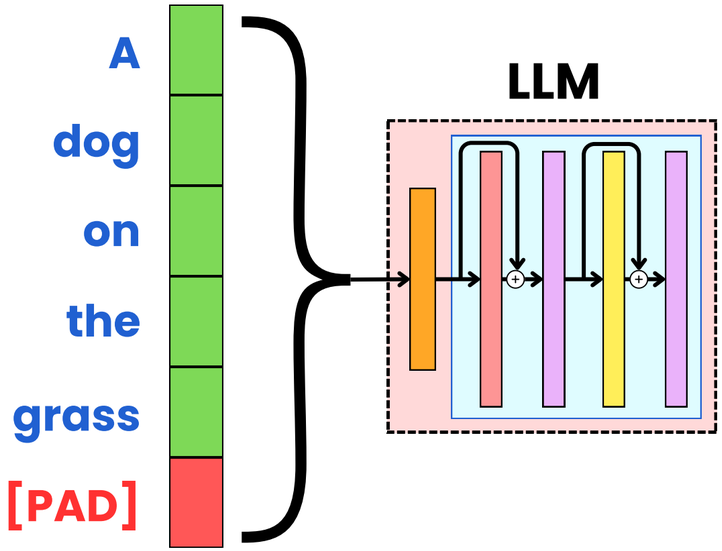
Vector database

How do we retrieve the right images?



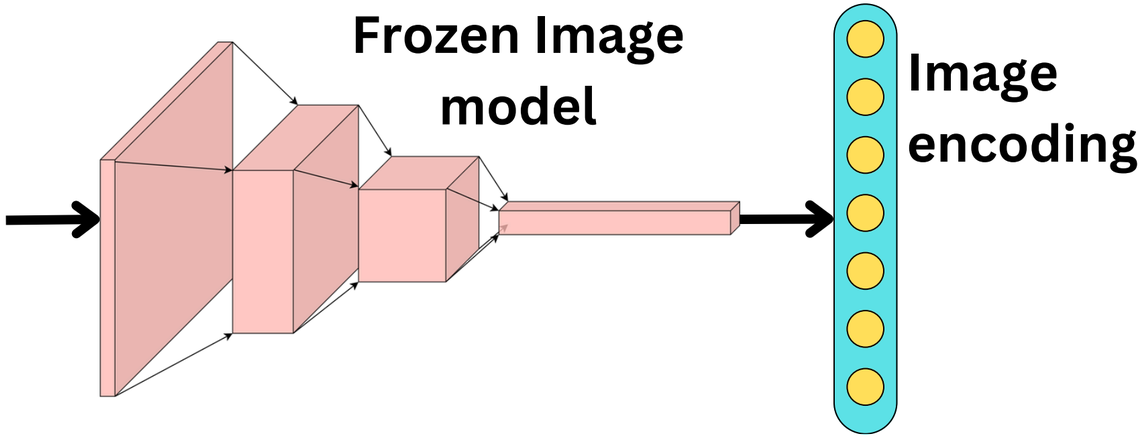
Text Encoding

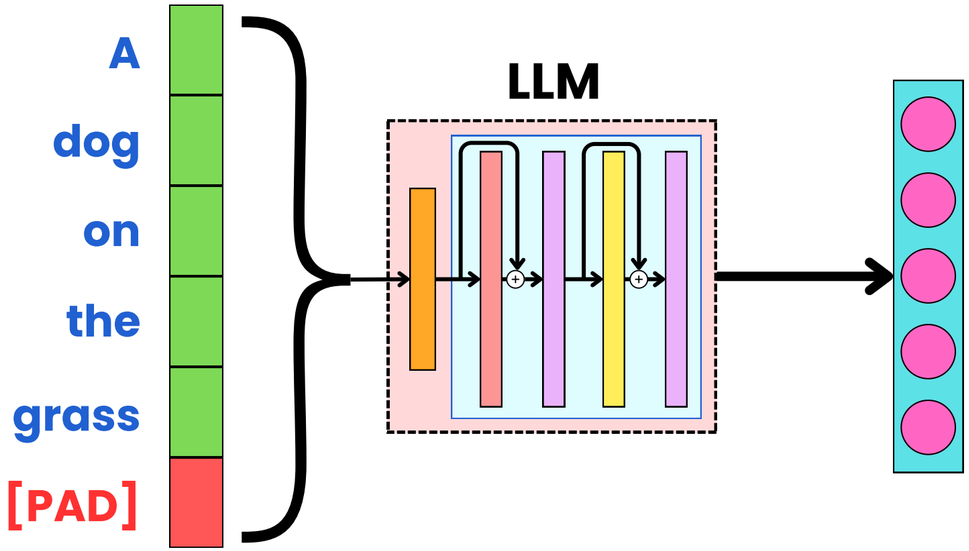




Input
prompt

Text Encoding





Input
prompt

Text Encoding

Frozen Image model

Image encoding

Train LLM to align with Computer Vision model

Compute Loss

A

dog

on

the

grass

[PAD]

Input prompt

LLM

Dot product

Compare

1

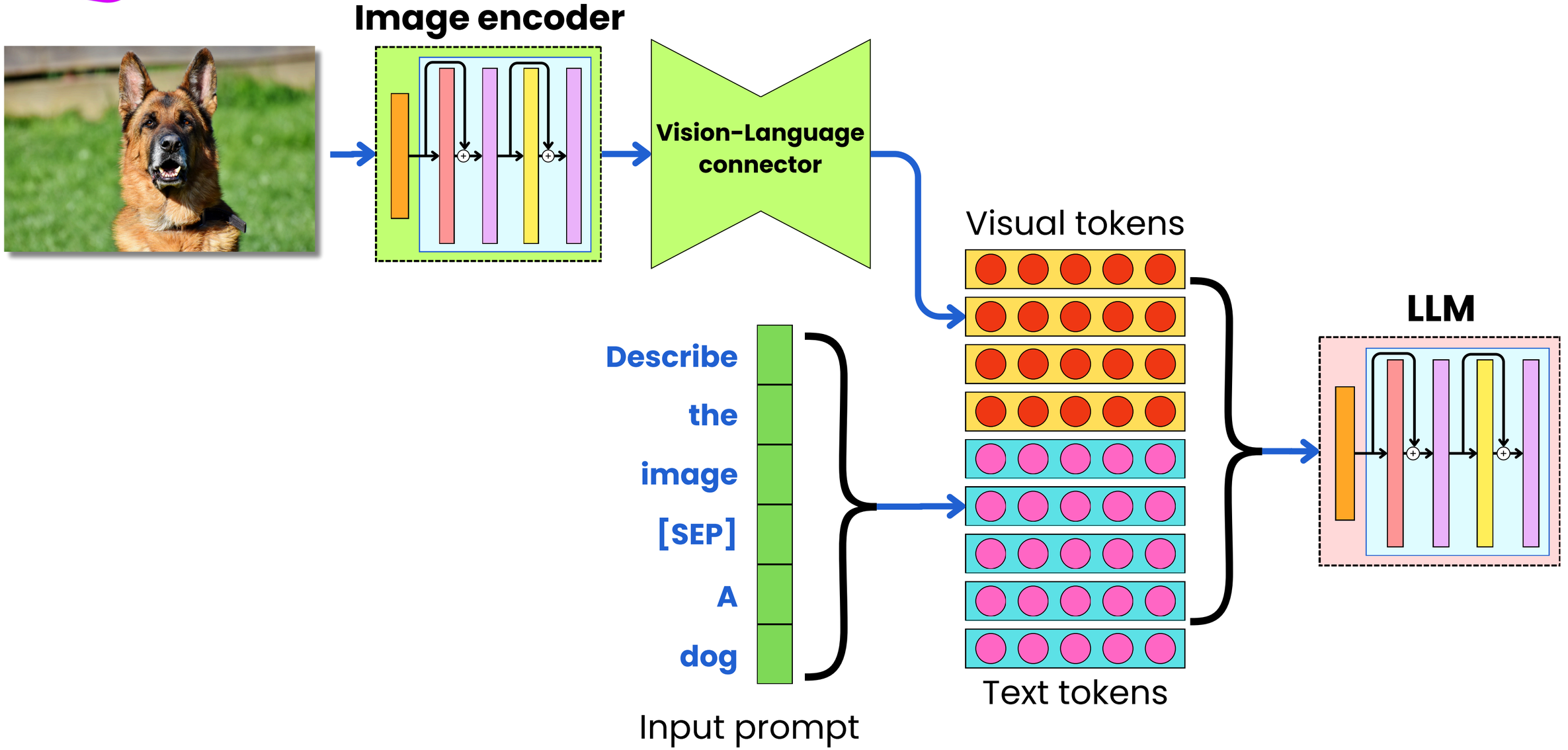
SIMILAR

Text encoding

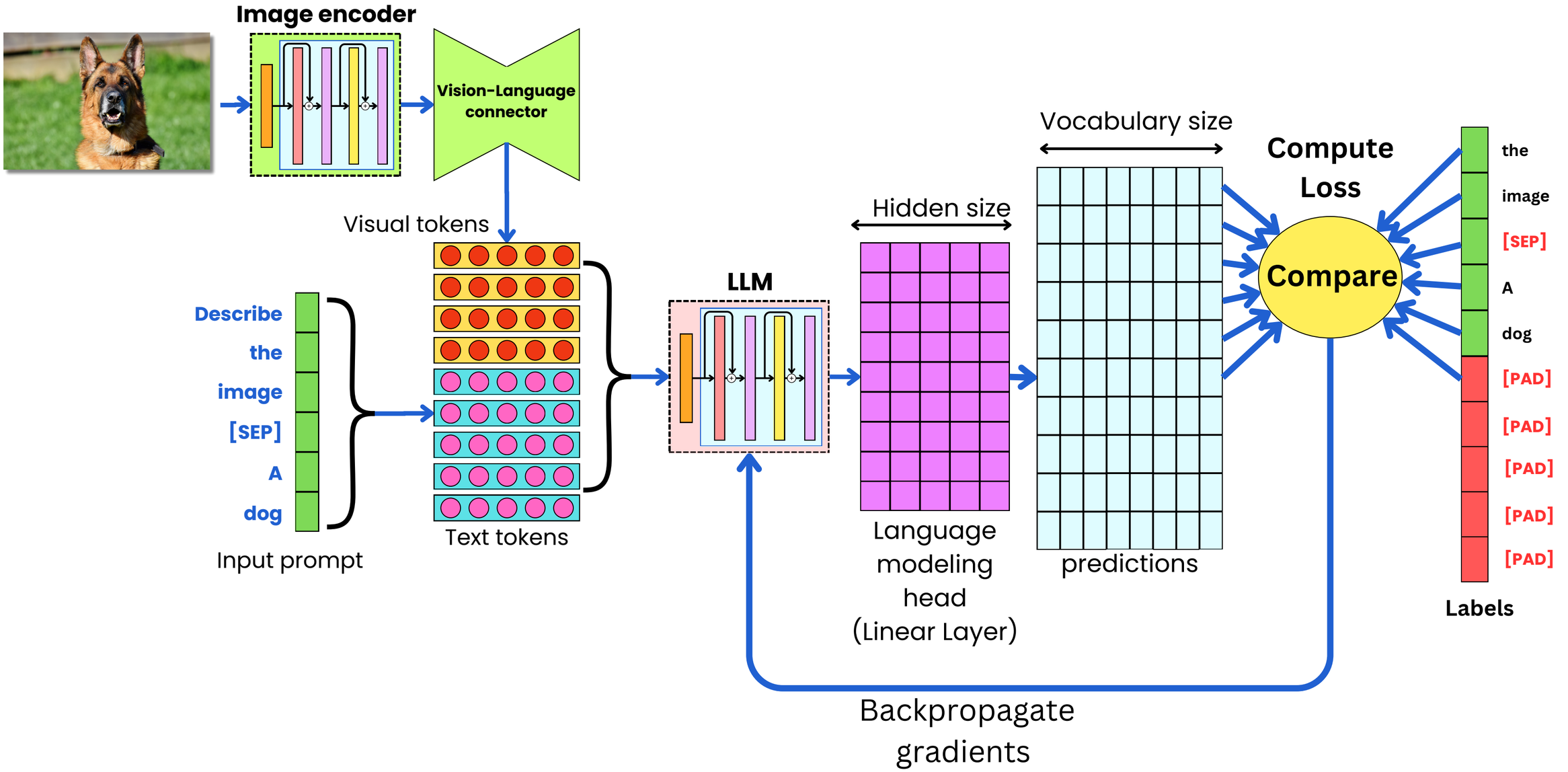
Backpropagate gradients

Multimodal Fine-Tuning

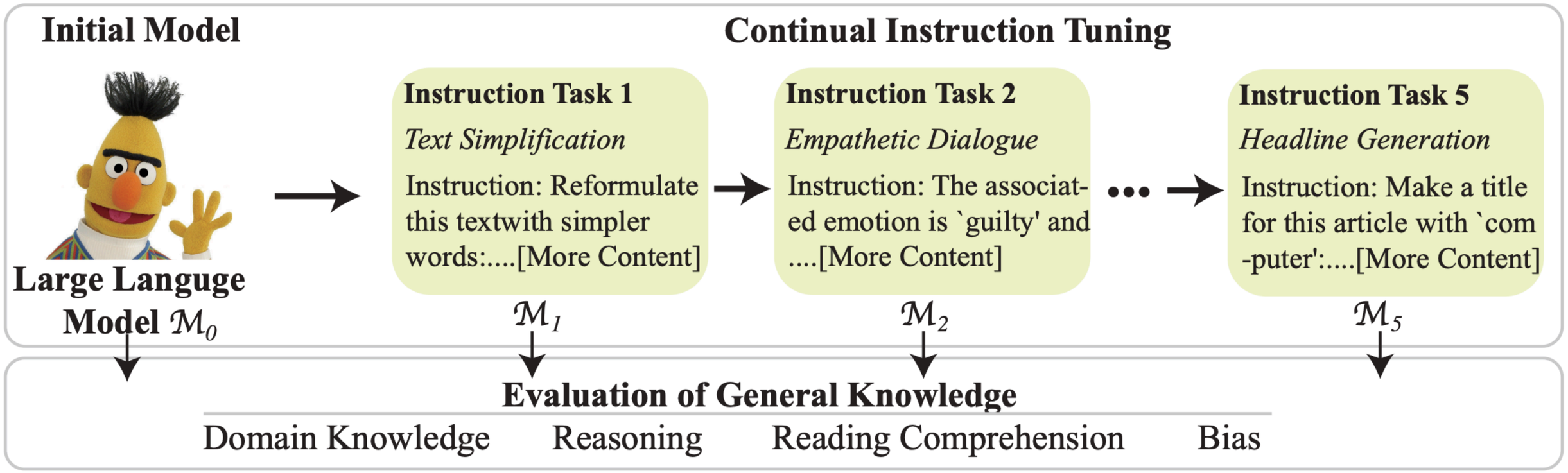
Multimodal Fine-Tuning



Multimodal Fine-Tuning

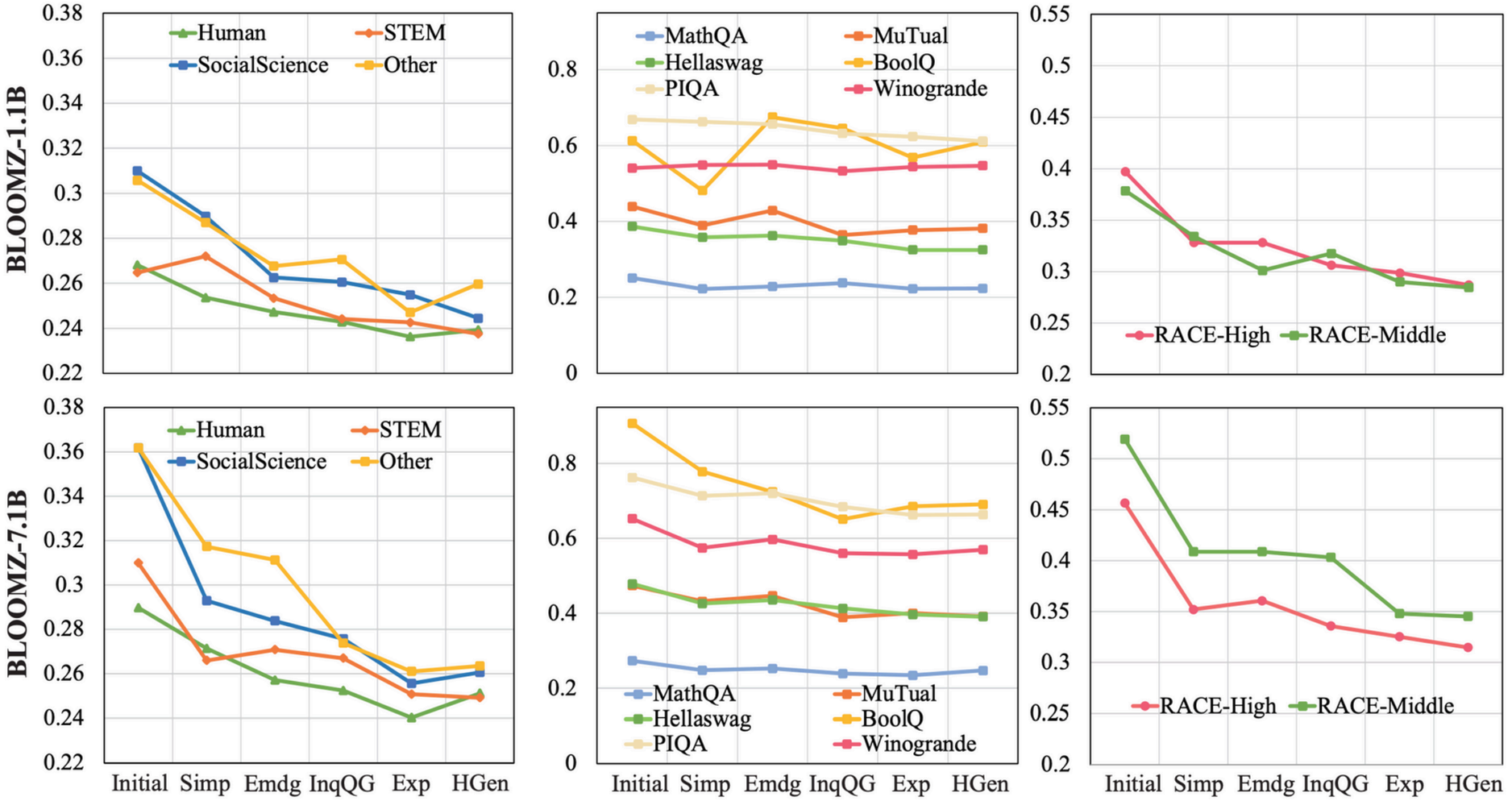


Catastrophic Forgetting



An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning
https://arxiv.org/pdf/2308.08747

Catastrophic Forgetting



An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning
https://arxiv.org/pdf/2308.08747

LoRA: Low-Rank Adaptation

The Gradient descent update

$$\theta_t \leftarrow \theta_{t-1} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{t-1}}$$

A trained model

$$\theta_T = \theta_0 - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_0} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_1} \ldots - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{T-1}}$$

LoRA: Low-Rank Adaptation

The Gradient descent update

$$\theta_t \leftarrow \theta_{t-1} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{t-1}}$$

A trained model

$$\theta_T = \theta_0 - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_0} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_1} \ldots - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{T-1}}$$

A fine-tuned model

$$\theta_F = \theta_0 - \alpha \sum_{t=0}^{T-1} \nabla_\theta \mathcal{L}|_{\theta=\theta_t} - \alpha \sum_{t=T}^{T+F} \nabla_\theta \mathcal{L}|_{\theta=\theta_t}$$

The trained model

The fine-tuning data

LoRA: Low-Rank Adaptation

The Gradient descent update

$$\theta_t \leftarrow \theta_{t-1} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{t-1}}$$

A trained model

$$\theta_T = \theta_0 - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_0} - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_1} \ldots - \alpha \nabla_\theta \mathcal{L}|_{\theta=\theta_{T-1}}$$

A fine-tuned model

$$\theta_F = \theta_0 - \alpha \sum_{t=0}^{T-1} \nabla_\theta \mathcal{L}|_{\theta=\theta_t} - \alpha \sum_{t=T}^{T+F} \nabla_\theta \mathcal{L}|_{\theta=\theta_t}$$

The trained model — The fine-tuning data

$$\theta_F = \theta_T + \Delta W$$

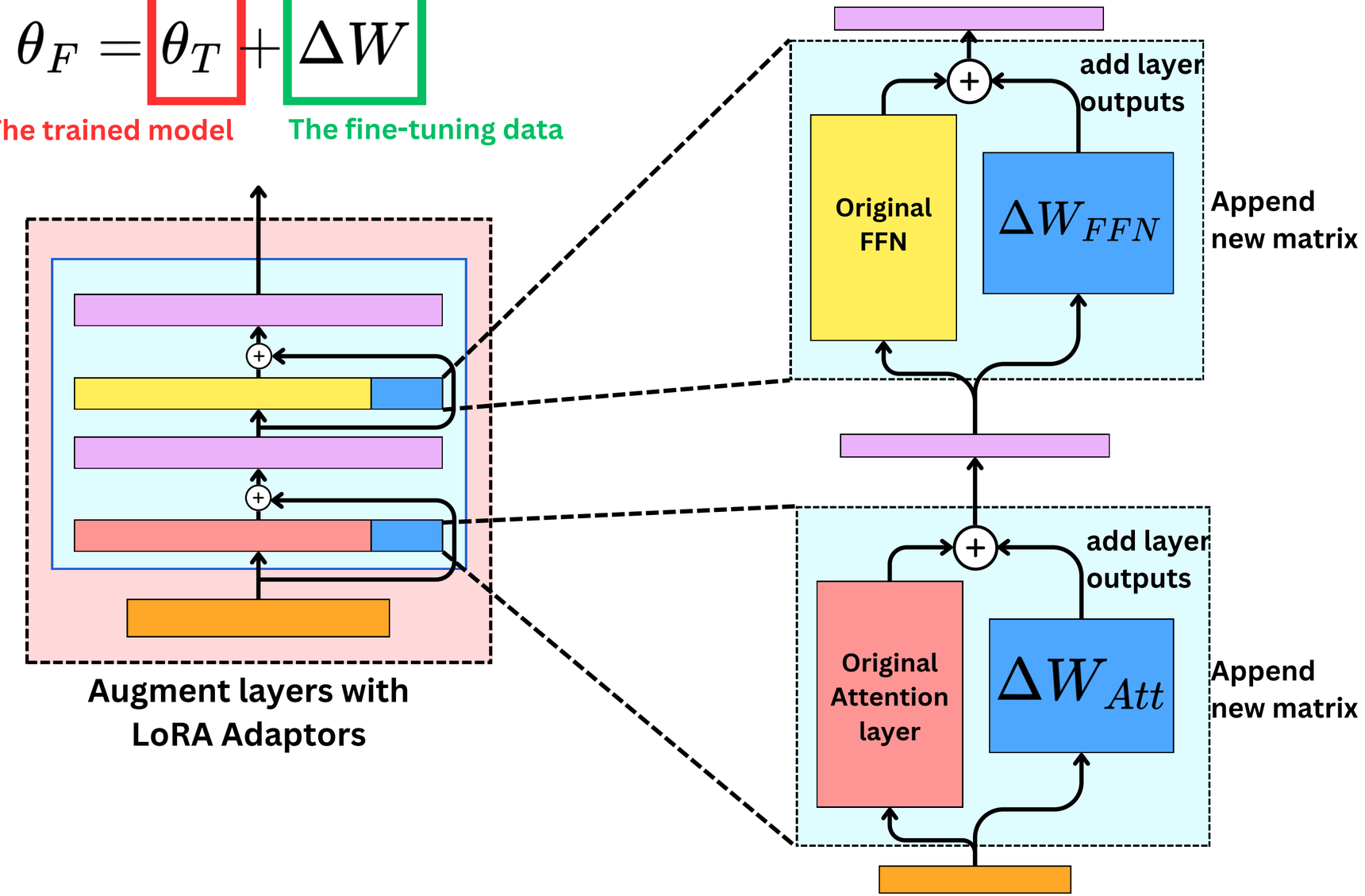
The trained model — The fine-tuning data

LoRA: Low-Rank Adaptation

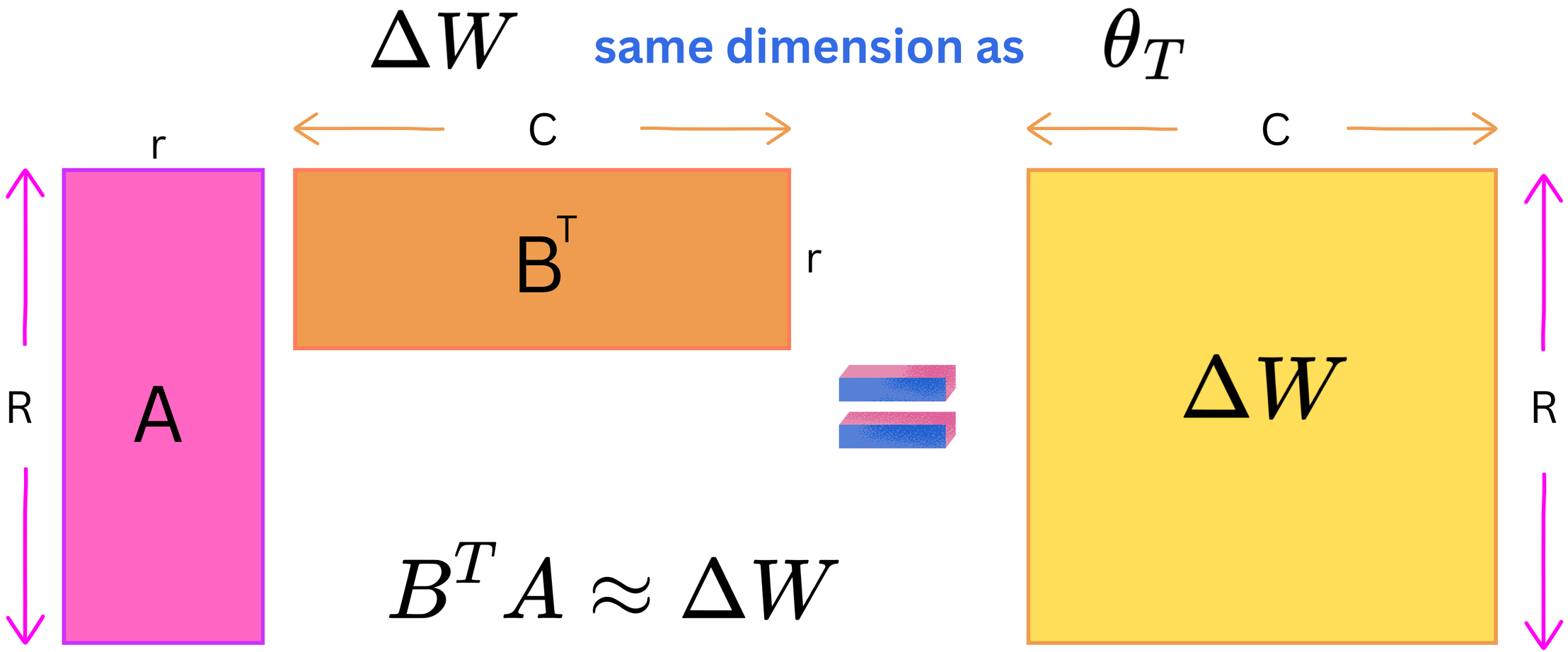
$$\theta_F = \theta_T + \Delta W$$

The trained model

The fine-tuning data



LoRA: Low-Rank Adaptation



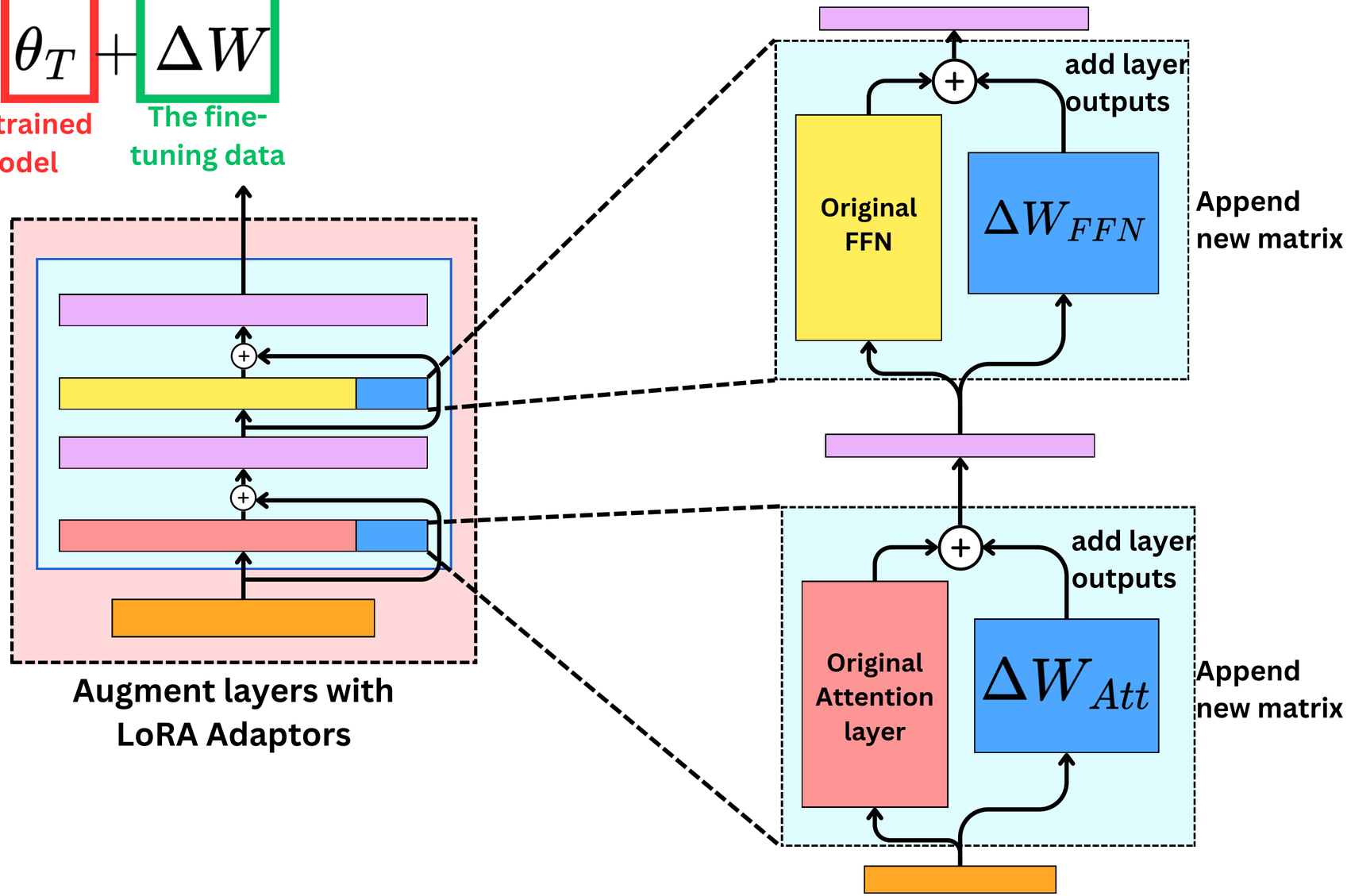
low-rank approximation

LoRA: Low-Rank Adaptation

$$\theta_F = \theta_T + \Delta W$$

The trained model

The fine-tuning data



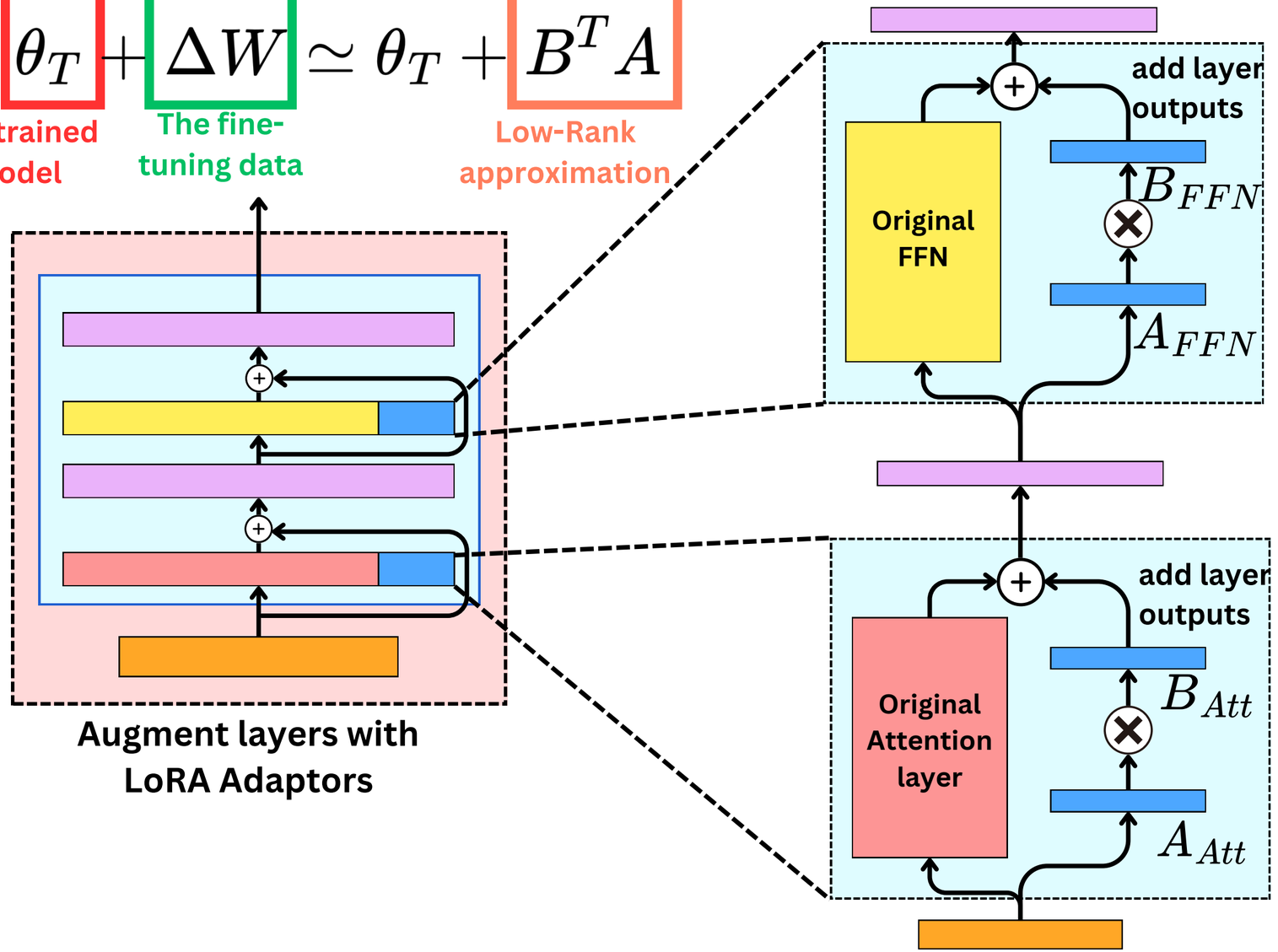
LoRA: Low-Rank Adaptation

$$\theta_F = \theta_T + \Delta W \simeq \theta_T + B^T A$$

The trained model

The fine-tuning data

Low-Rank approximation



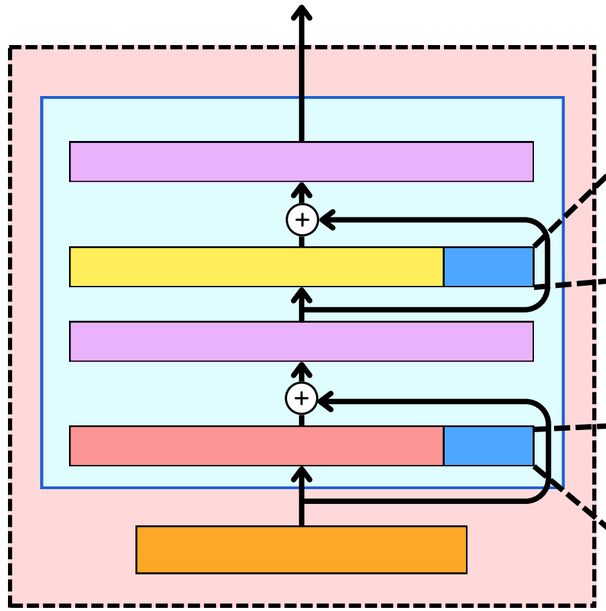
Augment layers with LoRA Adaptors

Forward pass

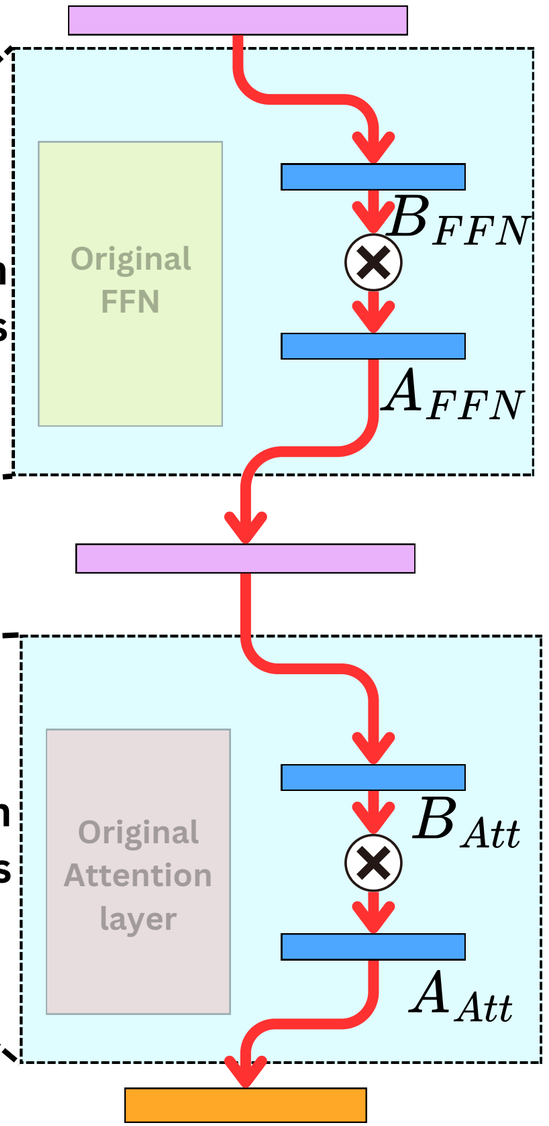
LoRA: Low-Rank Adaptation

$$A_t \leftarrow A_{t-1} - \alpha \nabla_A \mathcal{L}\big|_{A=A_{t-1}}$$

$$B_t \leftarrow B_{t-1} - \alpha \nabla_B \mathcal{L}\big|_{B=B_{t-1}}$$

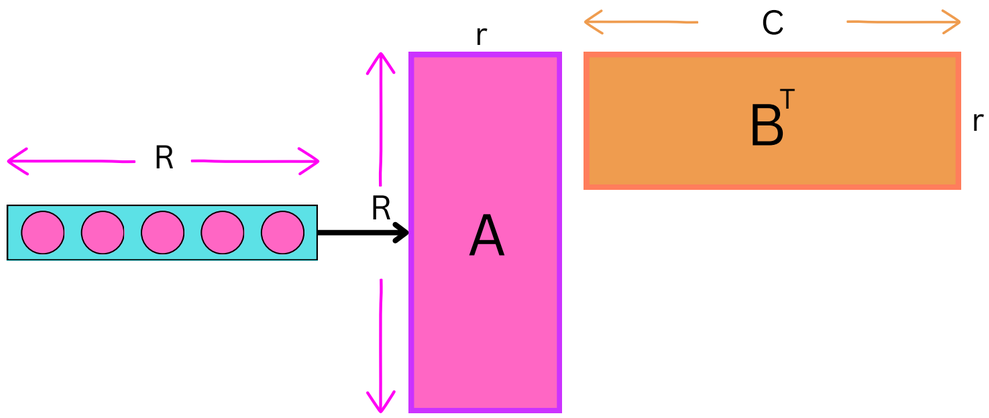


Augment layers with LoRA Adaptors

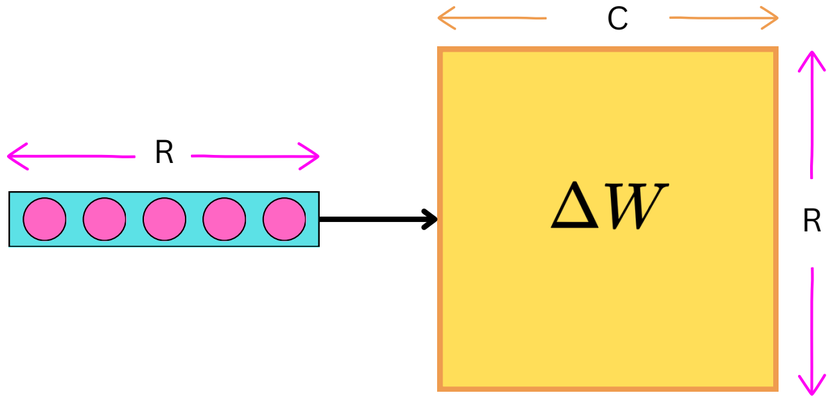


Backward pass

LoRA: Low-Rank Adaptation



- **R = 10K**
- **C = 20K**
- **r = 4**



Number of operations for Ax^T

$1 \times R \times r = 10,000 \times 4 = 40,000$

Number of operations for $B\left(Ax^T\right)$

$1 \times r \times C = 4 \times 20,000 = 80,000$

Total Number of operations

$40,000 + 80,000 = 120,000$

Number of elements

$40,000 + 80,000 = 120,000$

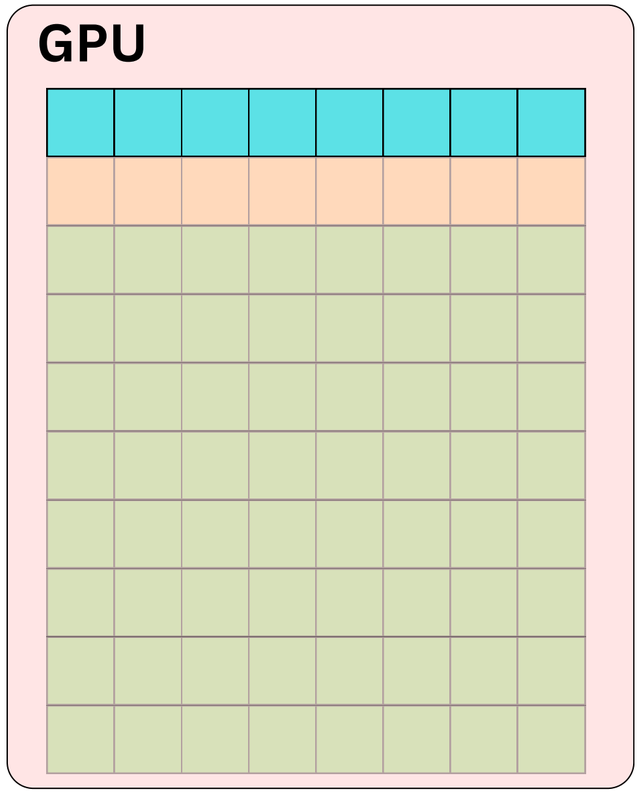
Number of operations

$1 \times R \times C = 10,000 \times 20,000 = 200,000,000$

Number of elements

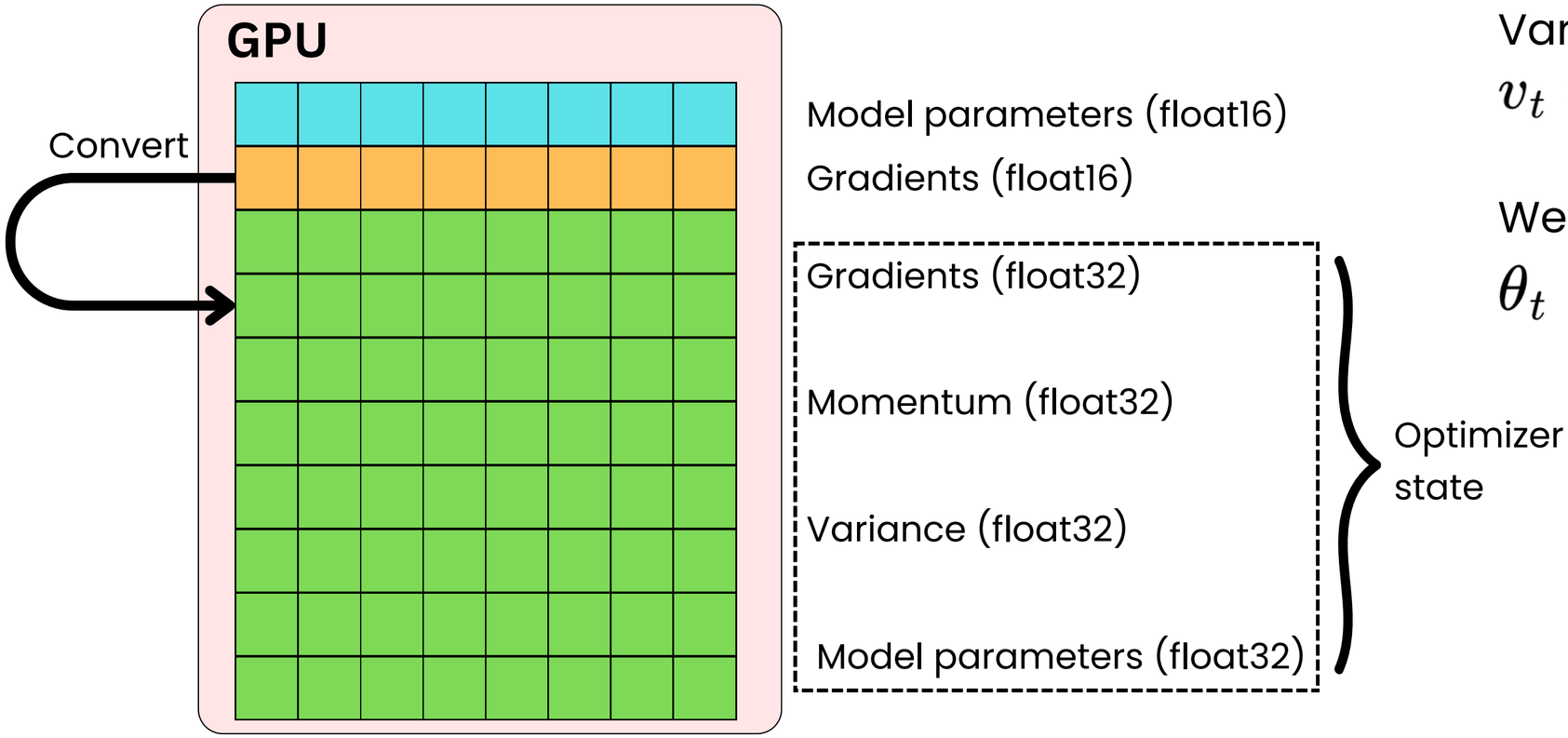
$1 \times R \times C = 10,000 \times 20,000 = 200,000,000$

Precision



Model parameters (float16)

Precision



Adam

Momentum

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla_\theta \mathcal{L}$$

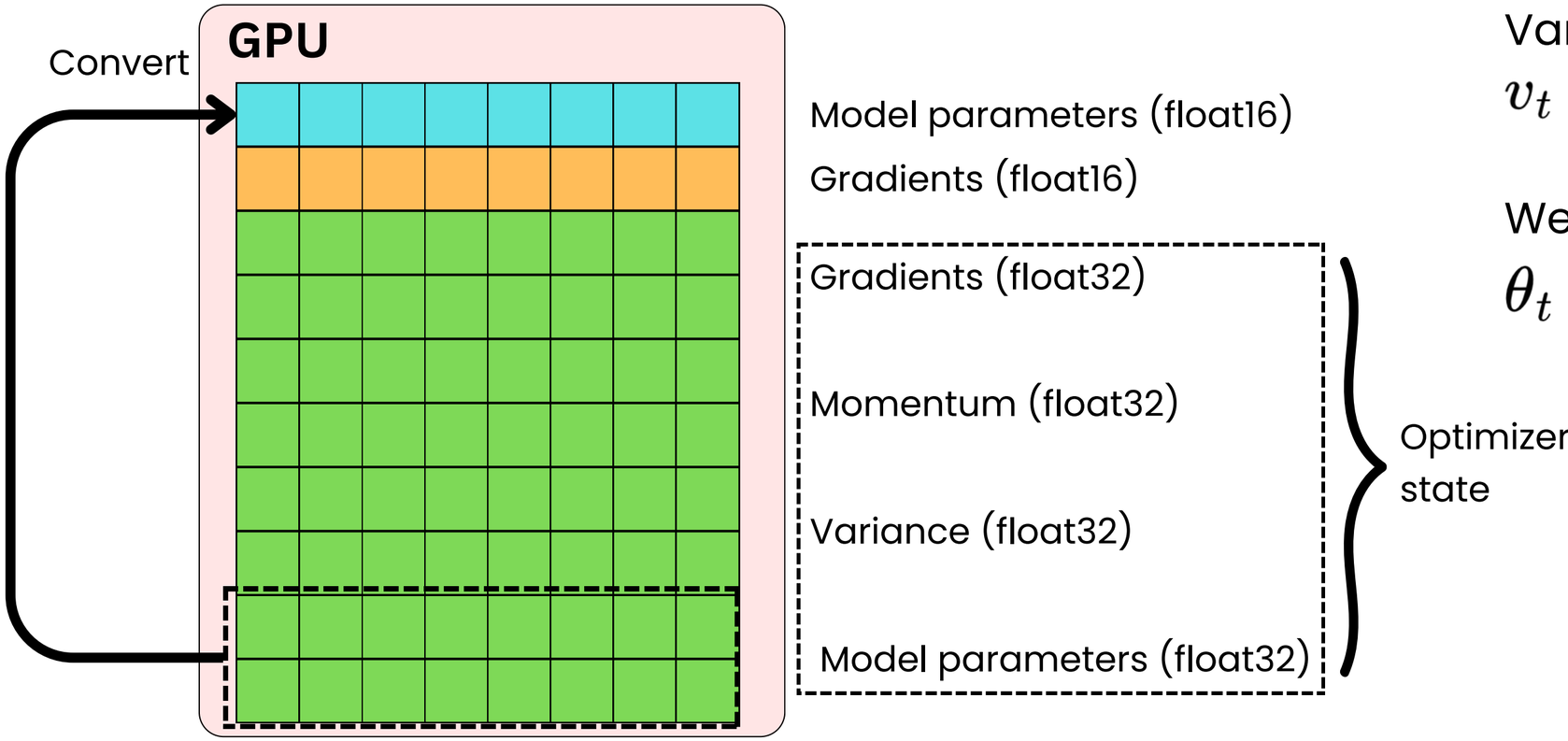
Variance

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla_\theta \mathcal{L})^2$$

Weights update

$$\theta_t = \theta_{t-1} - \frac{\mu}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Precision



Adam

Momentum

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla_\theta \mathcal{L}$$

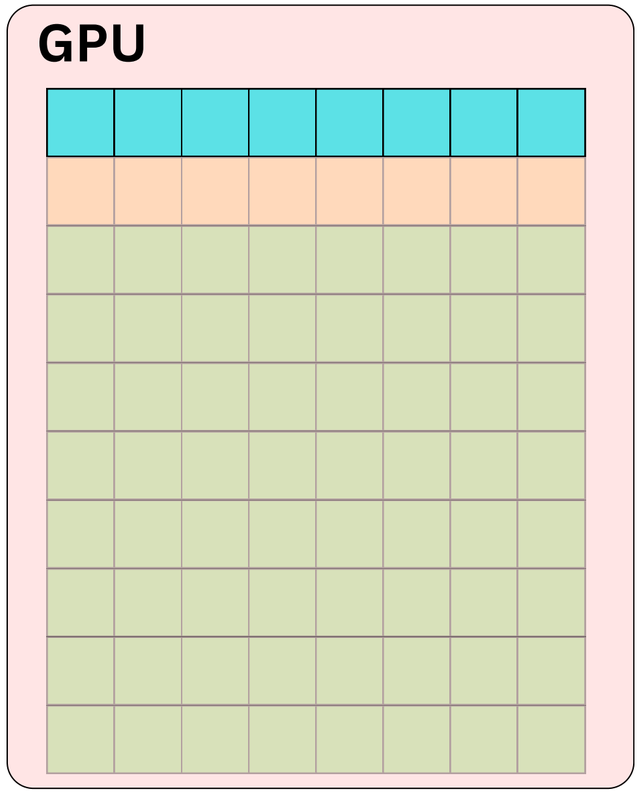
Variance

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla_\theta \mathcal{L})^2$$

Weights update

$$\theta_t = \theta_{t-1} - \frac{\mu}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

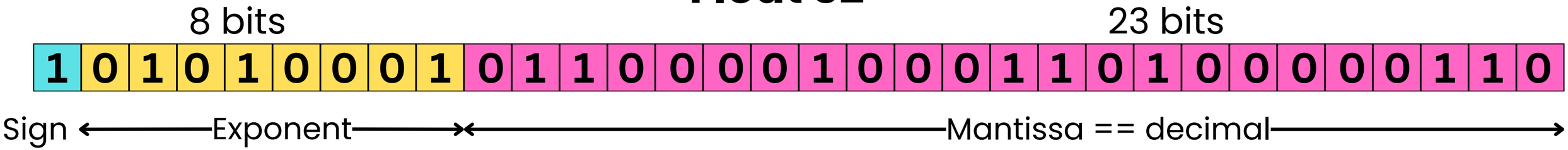
Precision



Model parameters (float16)

Precision

Float 32



$$(-1)^{sign} 2^{(exponent-127)} \times 1.mantissa$$

1 → **1**

0 1 0 1 0 0 0 1 → **81**

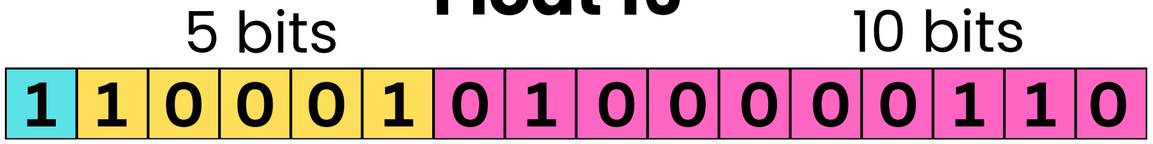
1 | 0 1 1 0 0 0 0 1 0 0 0 1 1 0 1 0 0 0 0 0 1 1 0 → **1.37930369377136230469**

$$(-1)^{1} 2^{(81-127)} \times 1.37930369377136230469$$

$$= -1.9601084e-14$$

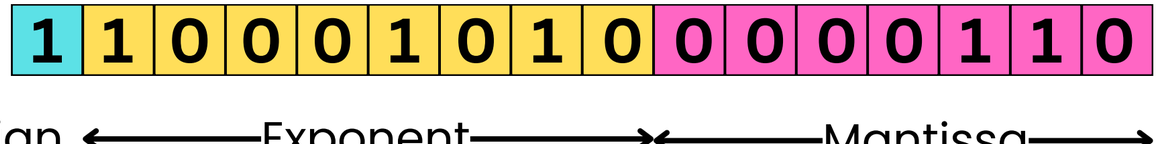
Precision

Float 16



$$(-1)^{sign} 2^{(exponent-15)} \times 1.mantissa$$

Brain Float 16



$$(-1)^{sign} 2^{(exponent-127)} \times 1.mantissa$$

Float 8

$$(-1)^{sign} 2^{(exponent-7)} \times 1.mantissa$$

Precision

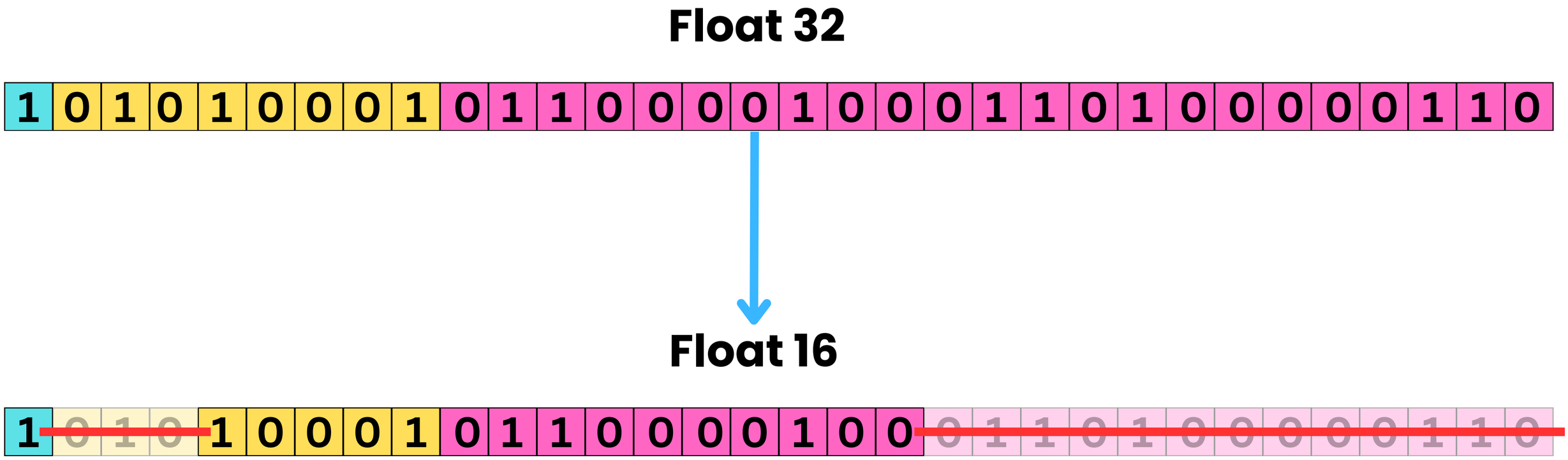
Float 32 $\left[-3.4 \times 10^{38}, 3.4 \times 10^{38}\right]$

Float 16 $\left[-6.55 \times 10^{4}, 6.55 \times 10^{4}\right]$

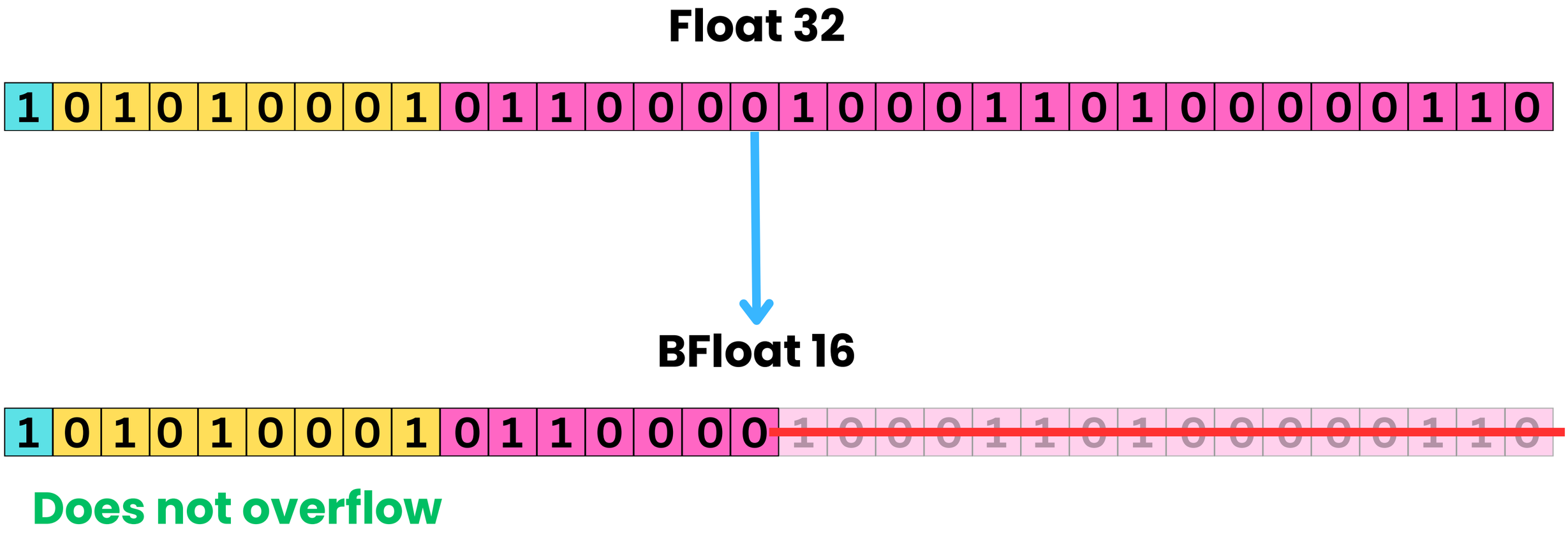
BFloat 16 $\left[-3.39 \times 10^{38}, 3.39 \times 10^{38}\right]$

Float 8 $\left[-240, 240\right]$

Converting Data Type

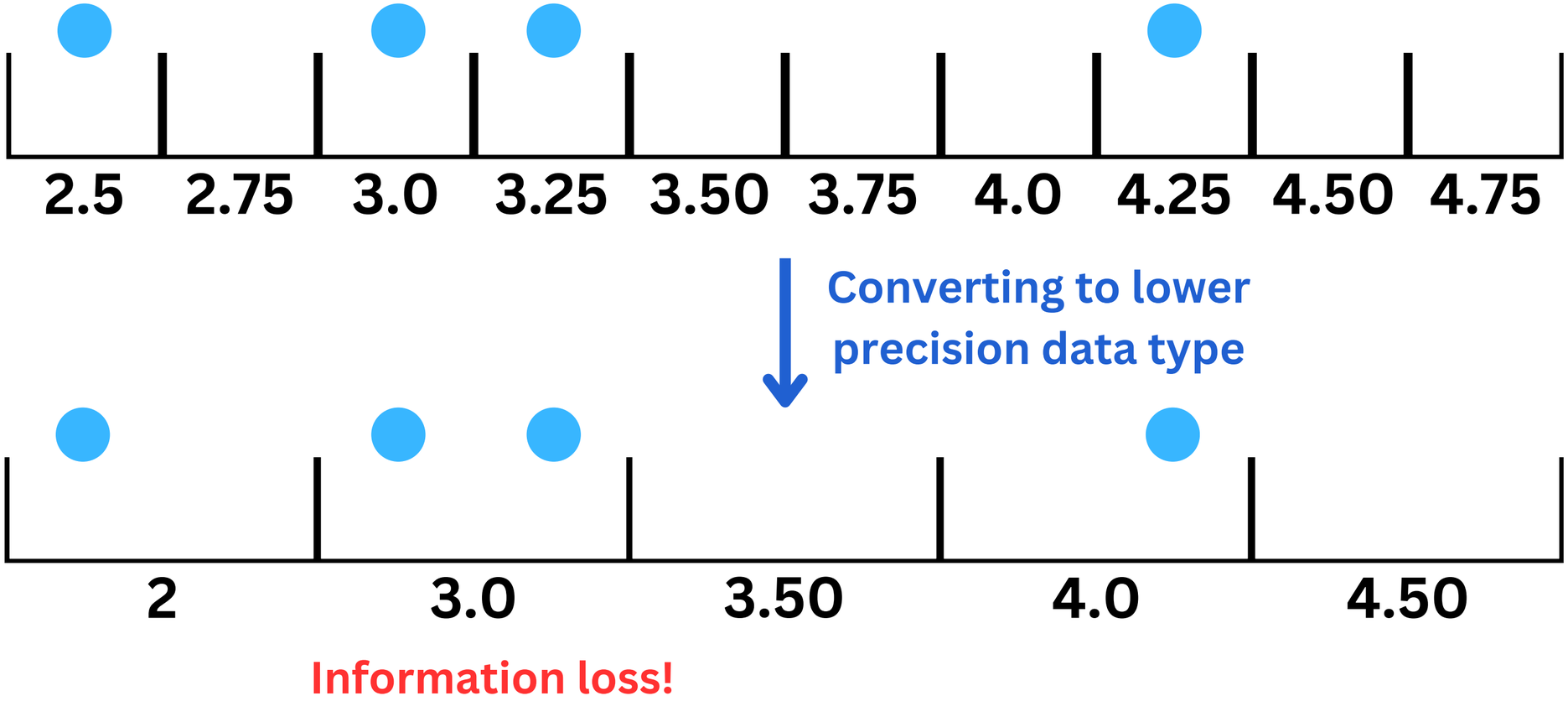


Converting Data Type

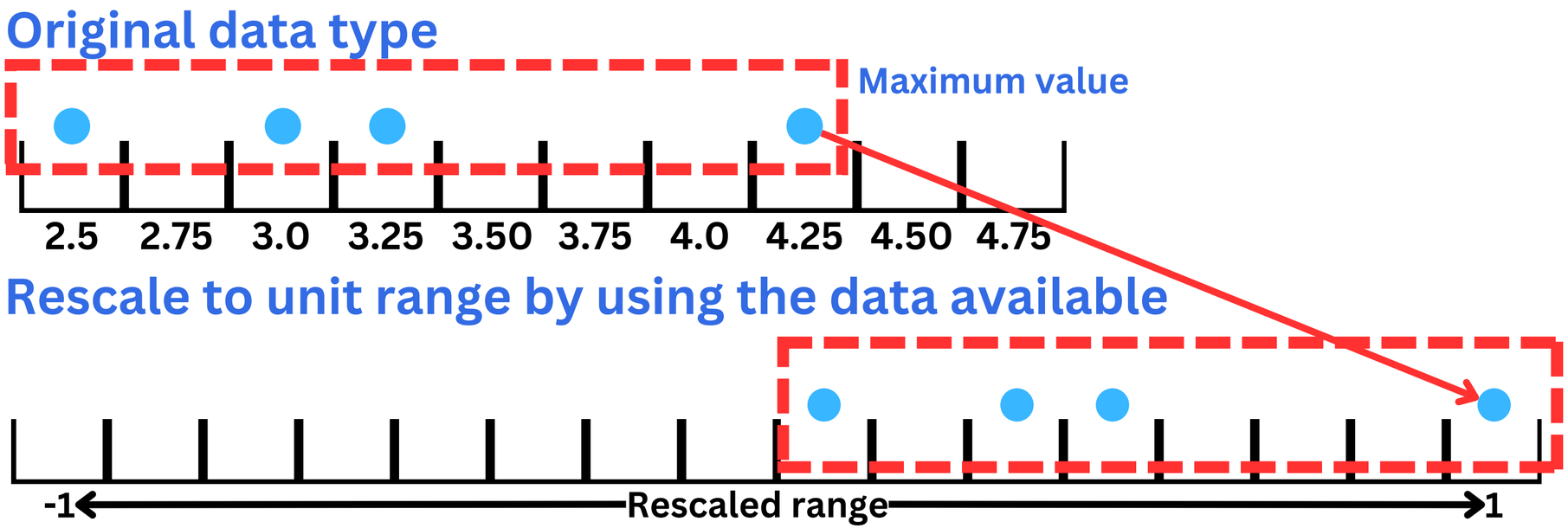


Does not overflow

Quantization

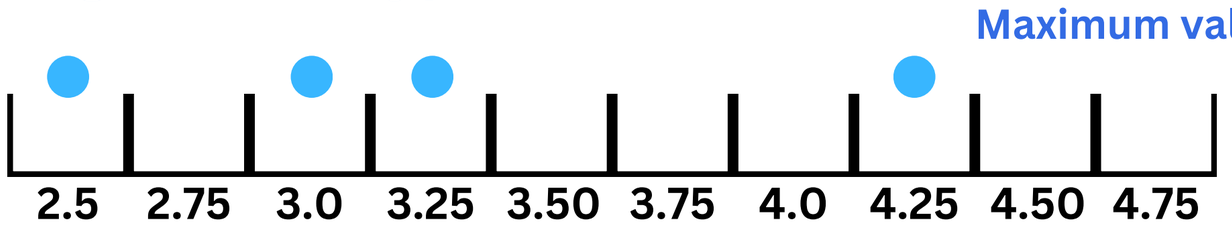


Quantization

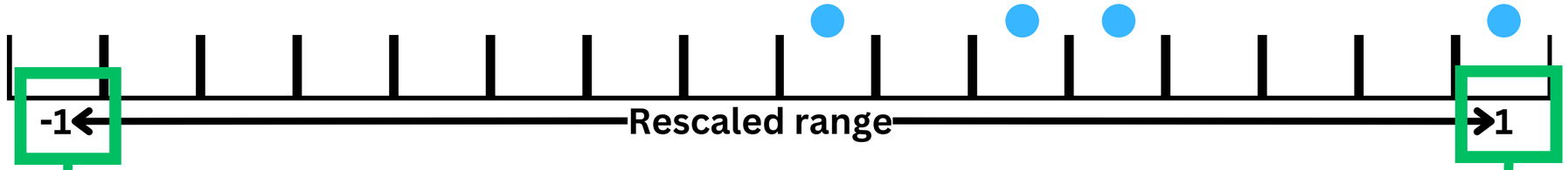


Quantization

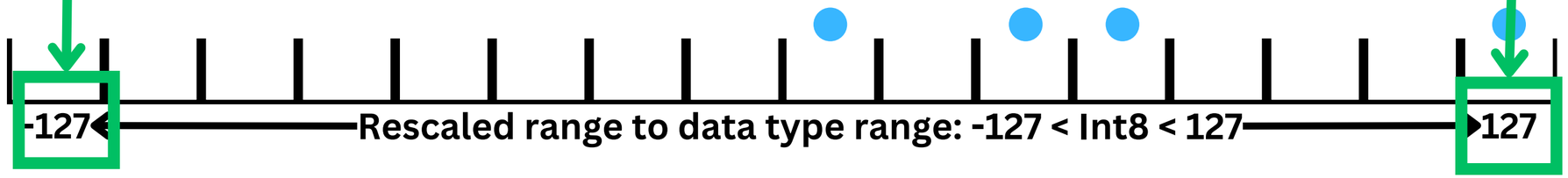
Original data type



Rescale to unit range by using the data available

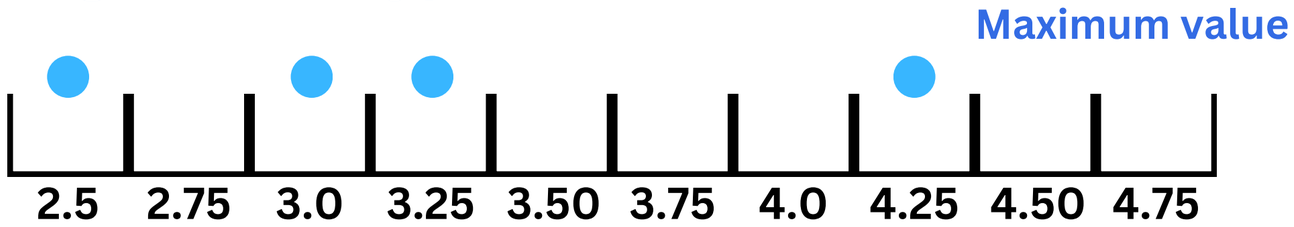


Rescale to the data type target range (e.g. Int8)



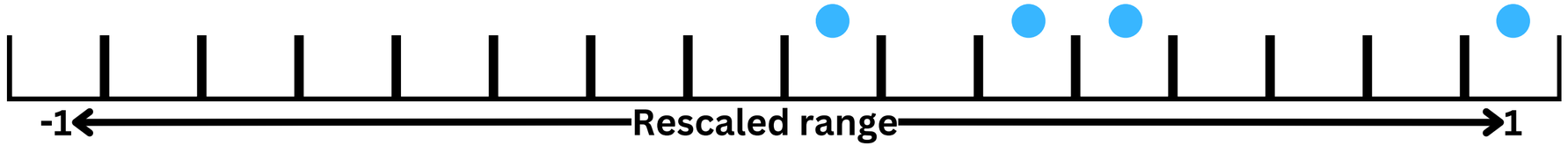
Quantization

Original data type

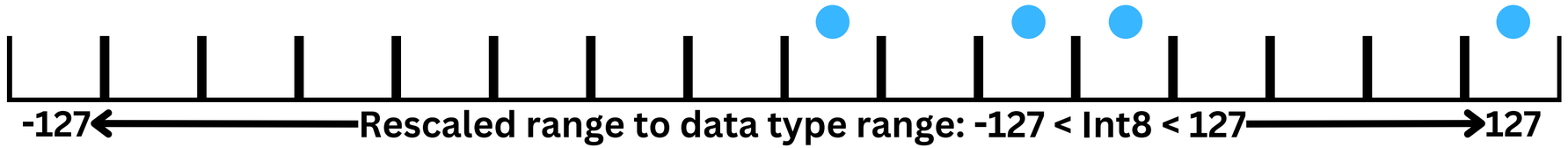


Adaptive data type conversion

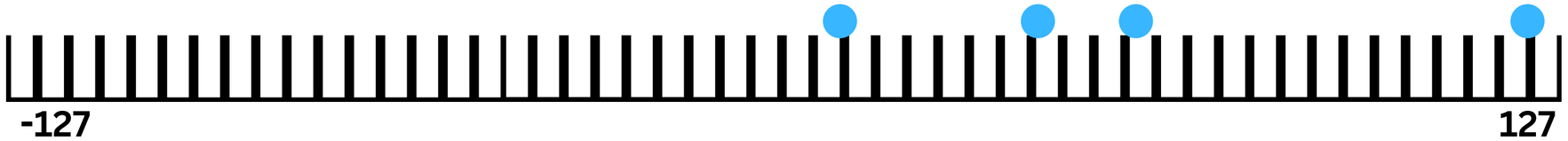
Rescale to unit range by using the data available



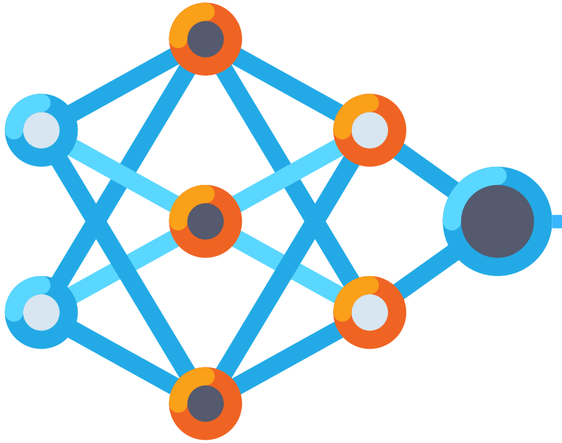
Rescale to the data type target range (e.g. Int8)



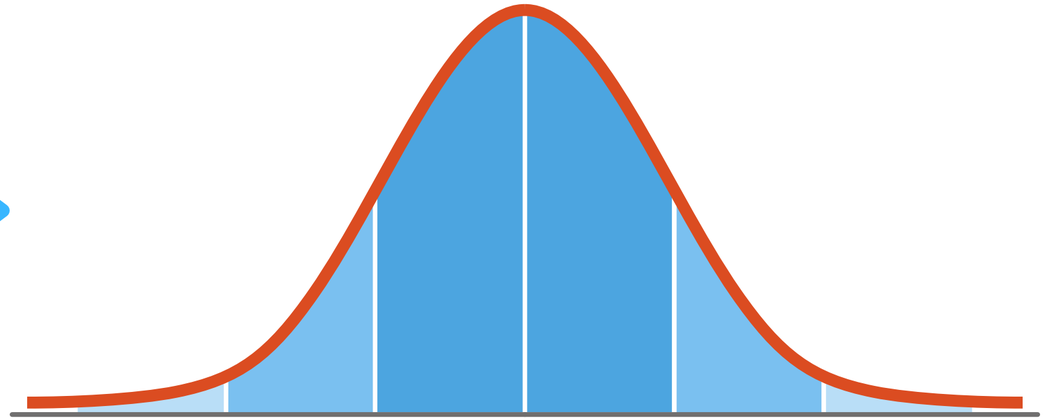
Convert to target data type (e.g. Int8)



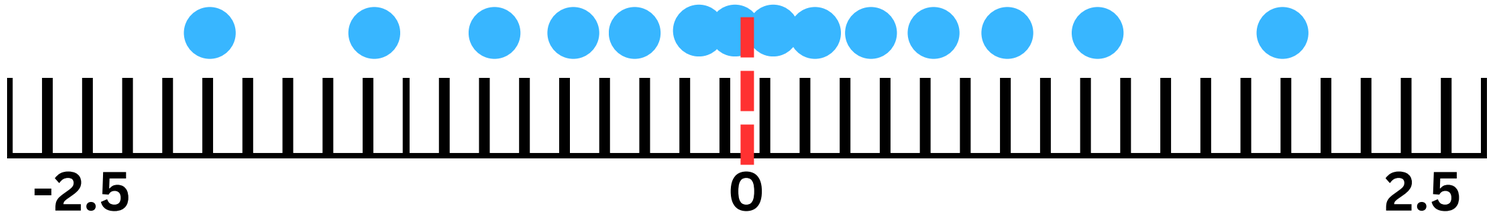
Quantization to 4-bits NormalFloat



Trained weights tend to be Normal distributed

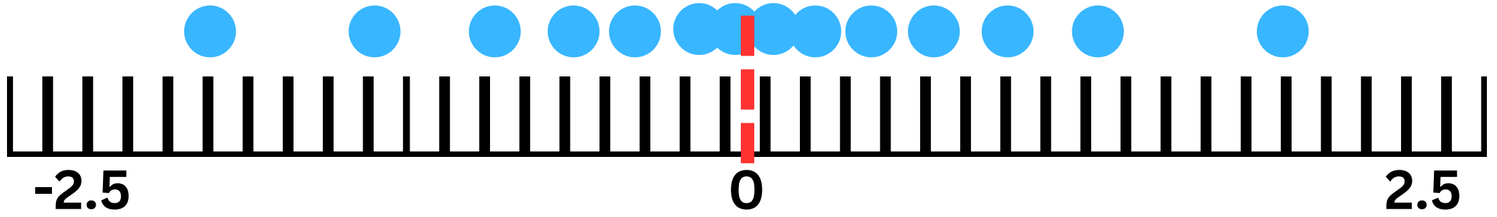


BFloat 16

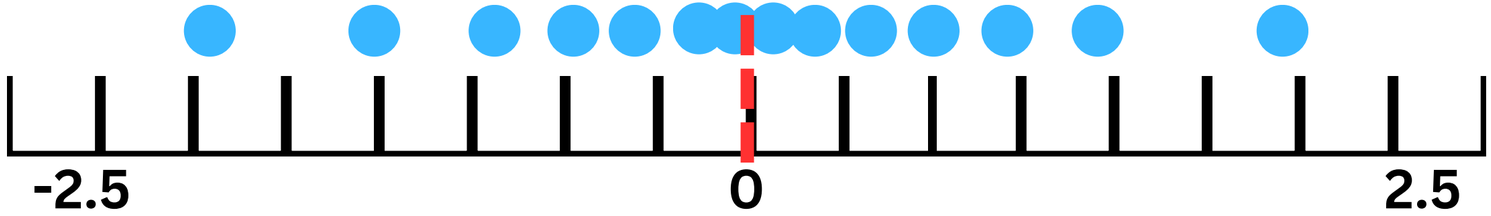


Quantization to 4-bits NormalFloat

BFloat 16



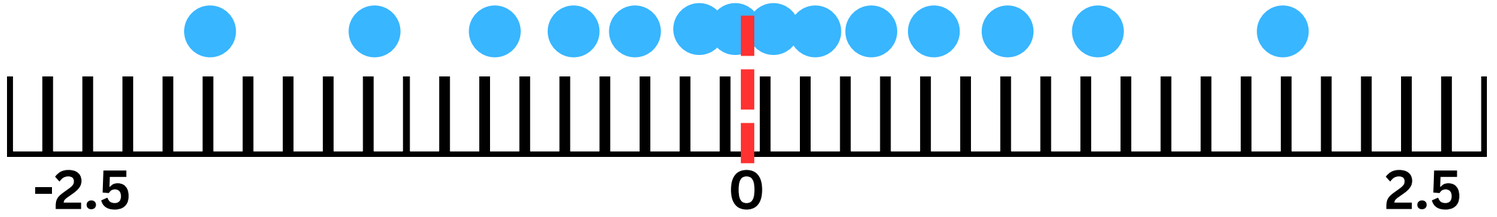
Naive 4-bits quantization



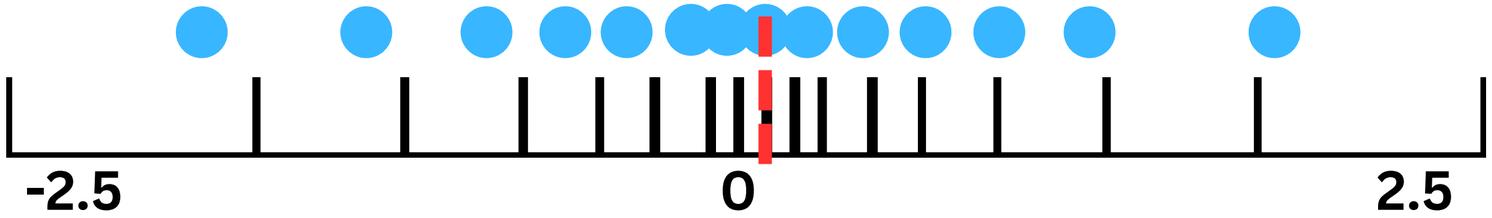
Many data points end up in the same bucket

Quantization to 4-bits NormalFloat

BFloat 16

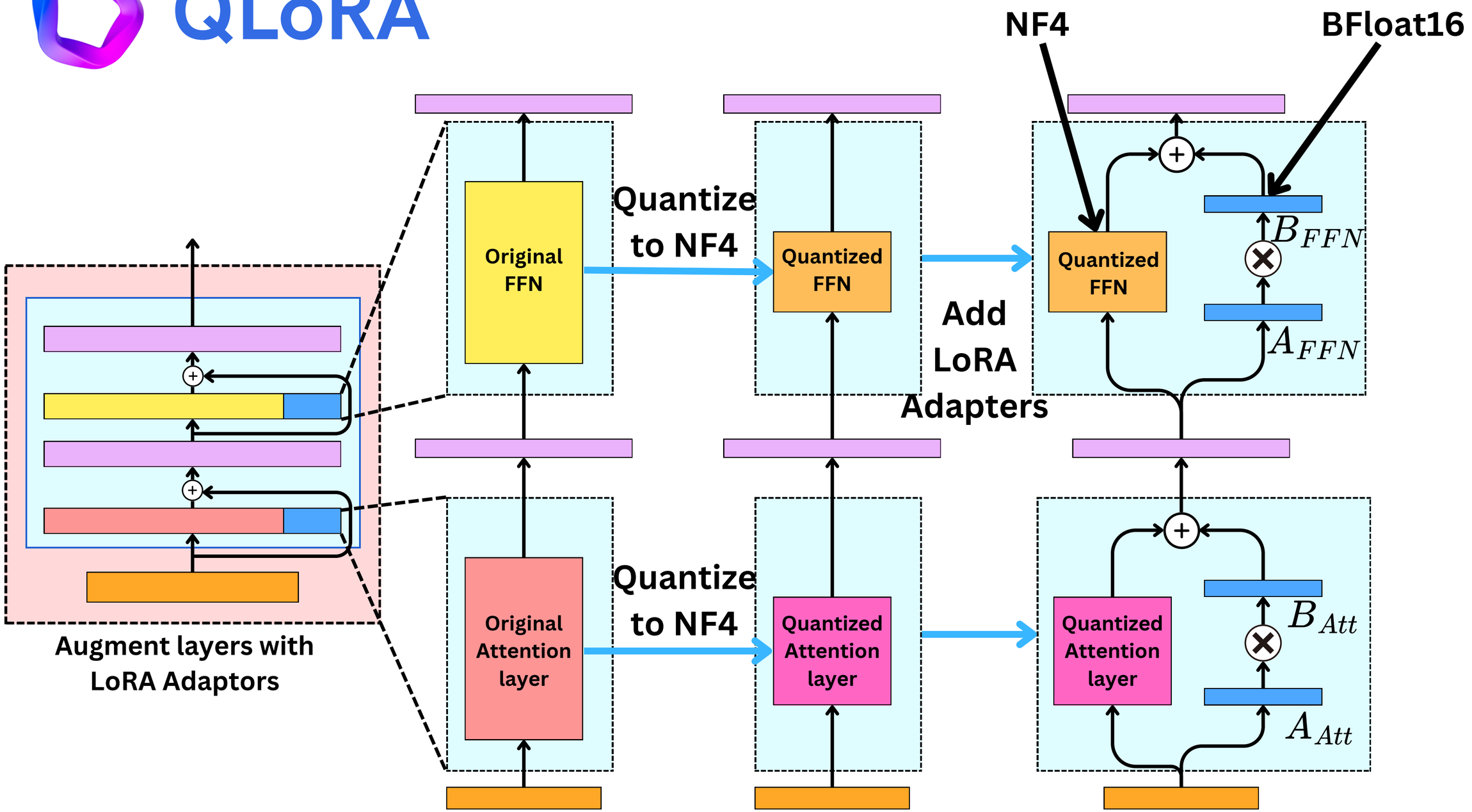


Normal distributed 4-bits quantization

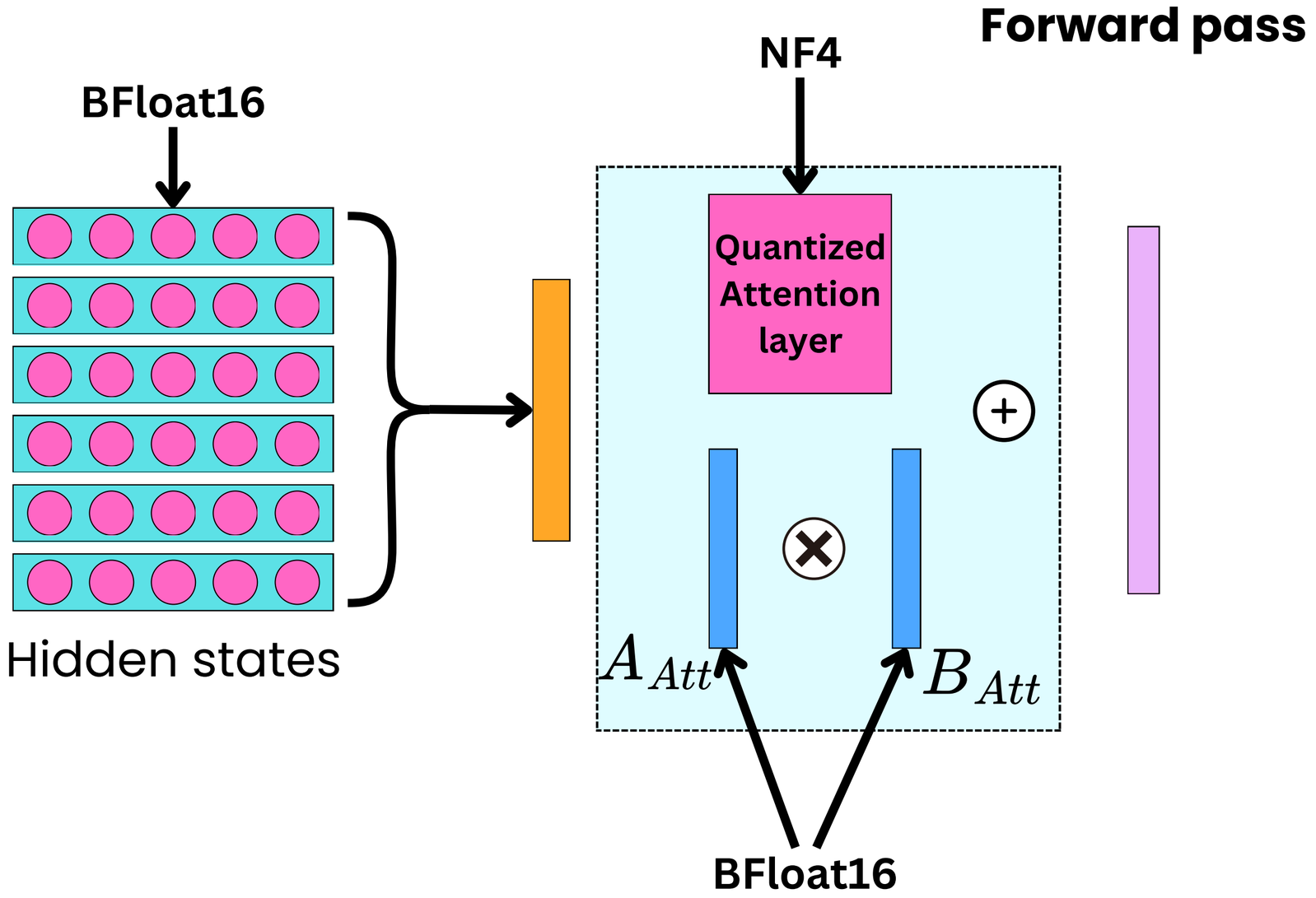


Less lossy

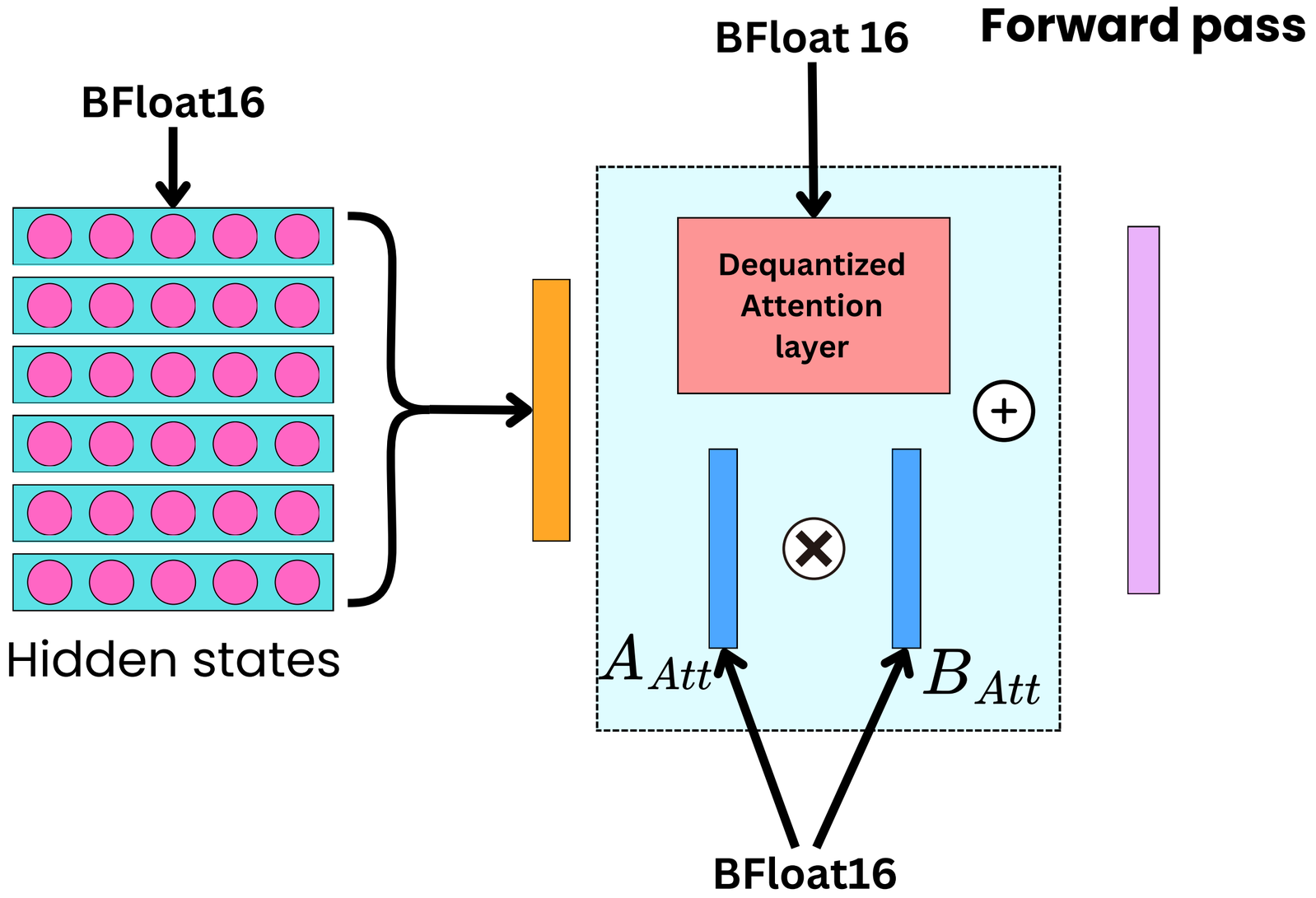
QLoRA


NF4
BFloat16
Original FFN
Quantize to NF4
Quantized FFN
Add LoRA Adapters
Quantized FFN
B_{FFN}
A_{FFN}
Augment layers with LoRA Adaptors
Original Attention layer
Quantize to NF4
Quantized Attention layer
Quantized Attention layer
B_{Att}
A_{Att}

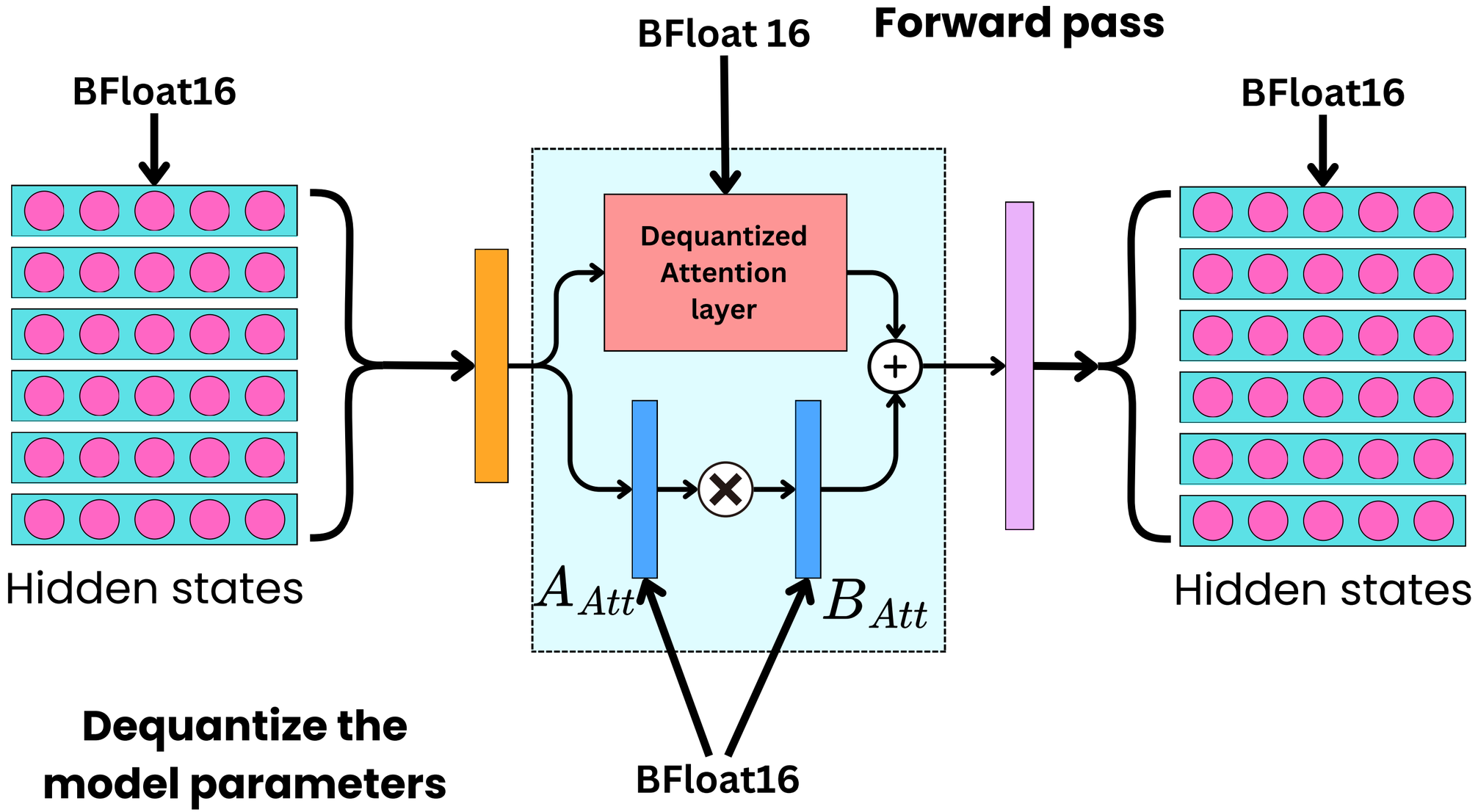
QLoRA


Forward pass
NF4
BFloat16
Quantized Attention layer
Hidden states
A_{Att}
B_{Att}
BFloat16

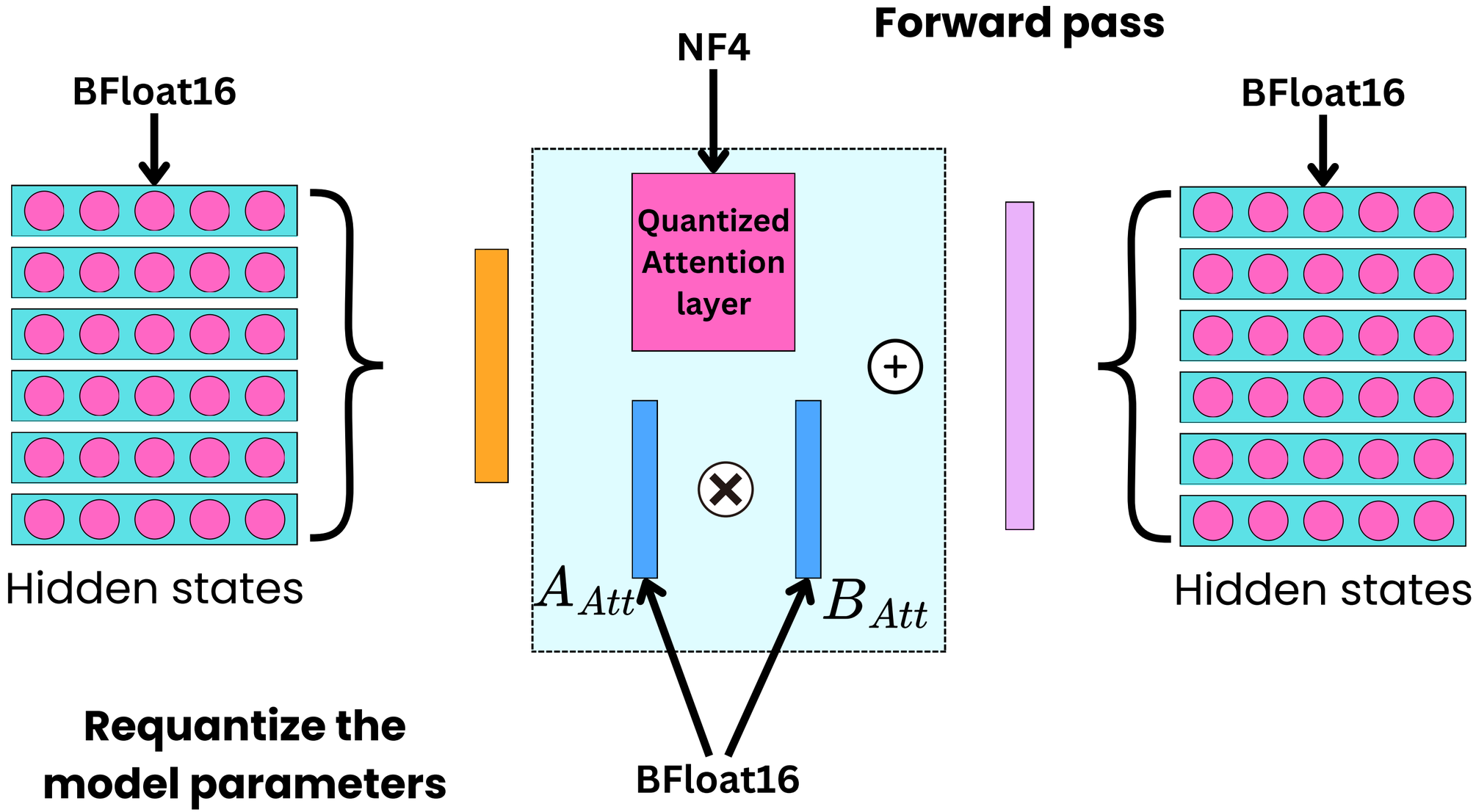
QLoRA


Forward pass
BFloat 16
BFloat16
Dequantized Attention layer
+
A_{Att}
B_{Att}
Hidden states
BFloat16

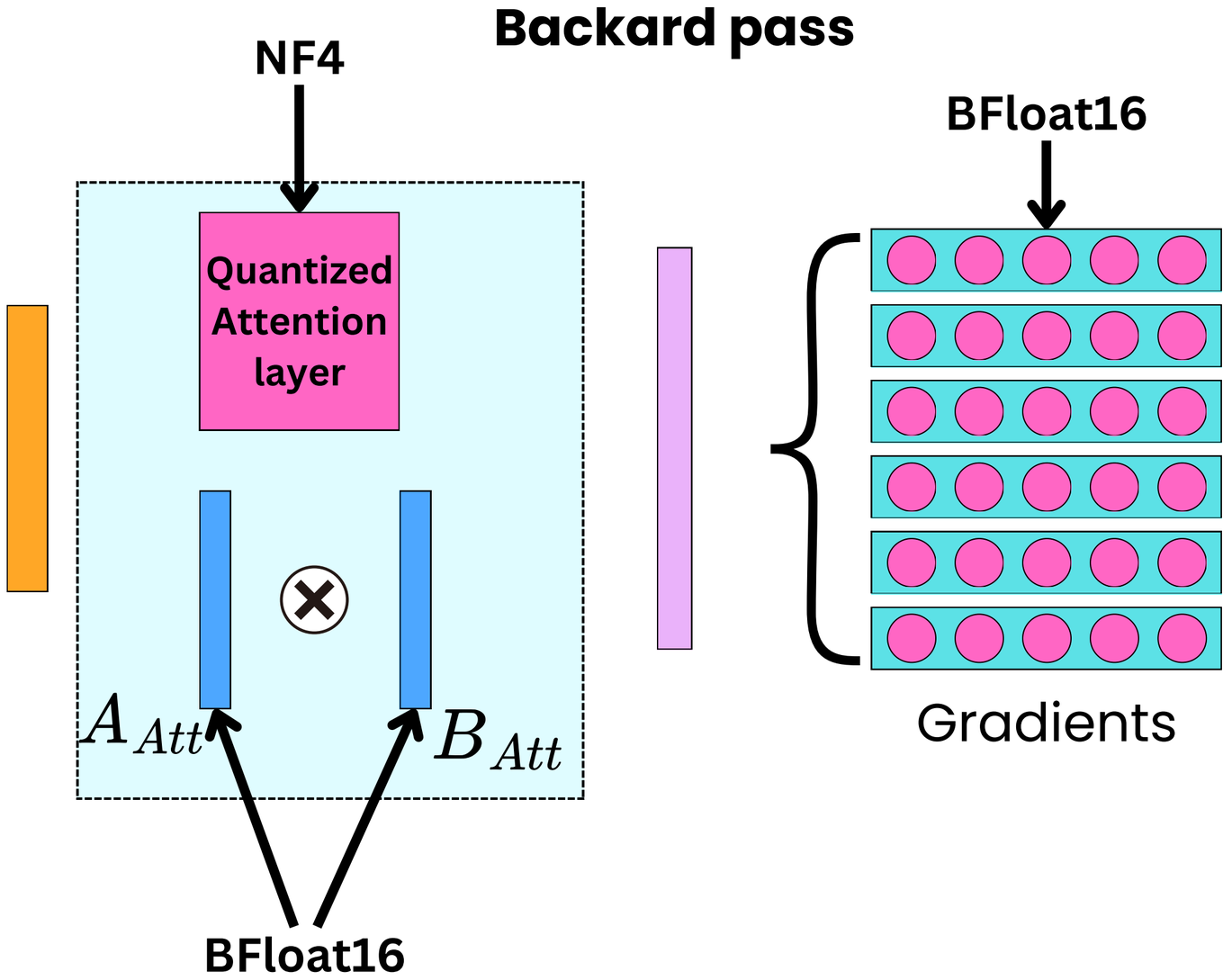
QLoRA


BFloat16
BFloat 16
Forward pass
BFloat16
Dequantized
Attention
layer
Hidden states
A_{Att}
B_{Att}
Hidden states
Dequantize the
model parameters
BFloat16

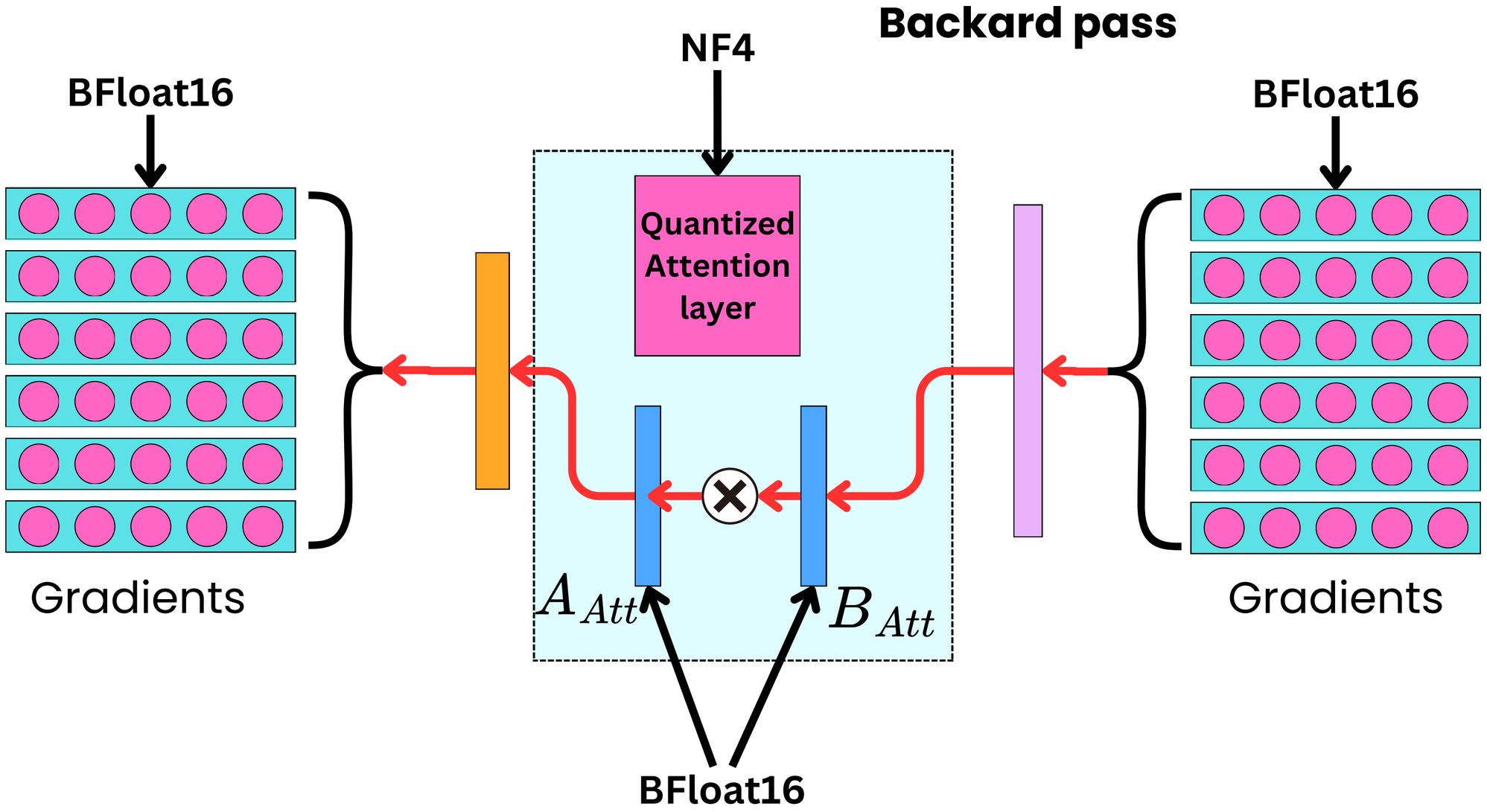
QLoRA


Forward pass
NF4
BFloat16
BFloat16
Quantized Attention layer
Hidden states
Hidden states
A_{Att}
B_{Att}
BFloat16
Requantize the model parameters

QLoRA


Backard pass
NF4
BFloat16
Quantized Attention layer
A_{Att}
B_{Att}
BFloat16
Gradients

QLoRA


Backard pass
NF4
BFloat16
BFloat16
Quantized Attention layer
Gradients
Gradients
A_{Att}
B_{Att}
BFloat16

QLoRA

- 4-bits NormalFloat quantization
- Double quantization (for the quantization constants)
- Paged Optimizer